UNIT I
Syllabus

Fundamentals of Natural Language Processing

Introduction to NLP: Definitions, Applications, Challenges, Linguistic Essentials: Syntax, Semantics, Pragmatics, Text Processing: Tokenization, Lemmatization, Stemming, Stopword Removal, Normalization, and N-gram Generation, POS Tagging and Named Entity Recognition, NLP Libraries: NLTK, SpaCy Overview.

STUDY MATERIAL

Prepared by **K.ANJANEYULU, M. Tech (CSE), MBA, PGDCA, M. Sc, B. Ed.,**
Assistant Professor,
Department of Computer Science & Engineering (AI&ML),
Sri Venkateswara College of Engineering & Technology,
RVS Nagar. Chittoor.
Phno: **(+91) – 9640985702, (+91) – 9398526101**
E-mail: **anji.karapakula@gmail.com**

1. NLP: Definitions, Applications, Challenges

Definition:

- Natural language processing (NLP) is a subfield of computer science and artificial intelligence (AI) that uses machine learning to enable computers to understand and communicate with human language.
- NLP enables computers and digital devices to recognize, understand and generate text and speech by combining computational linguistics, the rule-based modeling of human language together with statistical modeling, machine learning and deep learning.

Applications of NLP:

- Sentiment Analysis
- Machine Translation
- Chatbots and Virtual Assistants
- Information Extraction
- Text Summarization
- Speech Recognition
- Named Entity Recognition (NER)
- Topic Modeling

Sentiment Analysis

NLP systems analyze text data to determine the sentiment or emotion expressed within it. This is widely used in market research, social media monitoring, and customer feedback analysis to gauge public opinion and sentiment toward products, services, or brands.

Machine Translation

NLP enables automatic translation of text from one language to another. NLP systems like Google Translate and Microsoft Translator utilize advanced algorithms to accurately translate text, making cross-lingual communication seamless and efficient.

Chatbots and Virtual Assistants

NLP powers conversational interfaces, such as chatbots and virtual assistants, enabling human-like interactions between users and machines. These NLP-driven systems can understand user queries, provide relevant information, and perform tasks, such as scheduling appointments, ordering food, or booking tickets.

Information Extraction

NLP techniques are used to extract structured information from unstructured text data. This includes identifying entities, such as names, dates, and locations and relationships between them, facilitating tasks like document summarization, entity recognition, and knowledge graph construction.

Text Summarization

NLP systems generate concise summaries of lengthy documents or articles, helping users quickly grasp the main points without having to read the entire text. This is valuable in scenarios where time is limited or when dealing with

large volumes of textual data, such as news articles, research papers, or legal documents.

Speech Recognition

NLP enables machines to transcribe and understand human speech. Speech recognition systems like Siri, Google Assistant, and Amazon Alexa utilize NLP algorithms to convert spoken words into text, enabling hands-free interaction and voice-controlled commands.

Named Entity Recognition (NER)

NLP systems identify and classify named entities mentioned in text data, such as people, organizations, locations, dates, and numerical expressions. NER is used in various applications, including information retrieval, entity linking, and event extraction.

Topic Modeling

NLP techniques cluster and categorize text documents based on their underlying themes or topics. Topic modeling algorithms like Latent Dirichlet Allocation (LDA) and Non-negative Matrix Factorization (NMF) help uncover hidden patterns and structures within large collections of text data, aiding in document classification, content recommendation, and trend analysis.

Benefits of Natural Language Processing (NLP):

- ❖ Increased documentation efficiency & accuracy
- ❖ Capability to automatically create a summary of large & complex textual content
- ❖ Enables personal assistants like Alexa to interpret spoken words
- ❖ Enables the usage of chatbots for customer assistance
- ❖ Performing sentiment analysis is simpler
- ❖ Advanced analytics insights that were previously out of reach

❖ **Increased documentation efficiency & accuracy**

An NLP-generated document accurately summarizes any original text that humans can't automatically generate. Also, it can carry out repetitive tasks such as analyzing large chunks of data to improve human efficiency.

❖ **Capability to automatically create a summary of large & complex textual content**

Natural processing language can be used for simple text mining tasks such as extracting facts from documents, analyzing sentiment, or identifying named entities. Natural processing can also be used for more complex tasks, such as understanding human behaviors and emotions.

❖ **Enables personal assistants like Alexa to interpret spoken words**

NLP is useful for personal assistants such as Alexa, enabling the virtual assistant to understand spoken word commands. It also helps to quickly find relevant information from databases containing millions of documents in seconds.

❖ **Enables the usage of chatbots for customer assistance**

NLP can be used in chatbots and computer programs that use artificial intelligence to communicate with people through text or voice. The chatbot uses NLP to understand what the person is typing and respond appropriately.

They also enable an organization to provide 24/7 customer support across multiple channels.

❖ **Performing sentiment analysis is simpler**

Sentiment Analysis is a process that involves analyzing a set of documents (such as reviews or tweets) concerning their attitude or emotional state (e.g., joy, anger). Sentiment analysis can be used for categorizing and classifying social media posts or other text into several categories: positive, negative, or neutral.

❖ **Advanced analytics insights that were previously out of reach**

The recent proliferation of sensors and Internet-connected devices has led to an explosion in the volume and variety of data generated. As a result, many organizations leverage NLP to make sense of their data to drive better business decisions.

Challenges with Natural Language Processing (NLP):

- Misspellings
- Language Differences
- Innate Biases
- Words with Multiple Meanings
- Uncertainty and False Positives
- Training Data

❖ **Misspellings**

Natural languages are full of misspellings, typos, and inconsistencies in style. For example, the word "process" can be spelled as either "process" or "processing." The problem is compounded when you add accents or other characters that are not in your dictionary.

❖ **Language Differences**

An English speaker might say, "I'm going to work tomorrow morning," while an Italian speaker would say, "Domani Mattina vado al lavoro." Even though these two sentences mean the same thing, NLP won't understand the latter unless you translate it into English first.

❖ **Innate Biases**

Natural processing languages are based on human logic and data sets. In some situations, NLP systems may carry out the biases of their programmers or the data sets they use. It can also sometimes interpret the context differently due to innate biases, leading to inaccurate results.

❖ **Words with Multiple Meanings**

NLP is based on the assumption that language is precise and unambiguous. In reality, language is neither precise nor unambiguous. Many words have multiple meanings and can be used in different ways. For example, when we say "bark," it can either be dog bark or tree bark.

❖ **Uncertainty and False Positives**

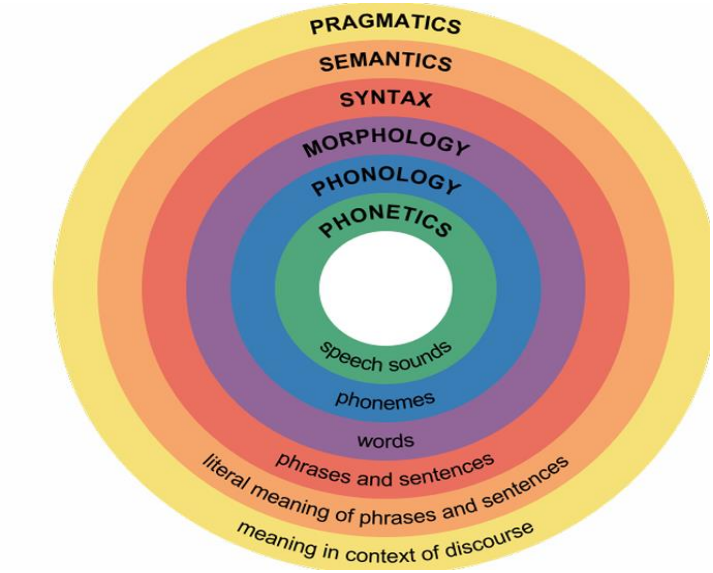
False positives occur when the NLP detects a term that should be understandable but can't be replied to properly. The goal is to create an NLP system that can identify its limitations and clear up confusion by using questions or hints.

❖ **Training Data**

One of the biggest challenges with natural processing language is inaccurate training data. The more training data you have, the better your results will be. If you give the system incorrect or biased data, it will either learn the wrong things or learn inefficiently.

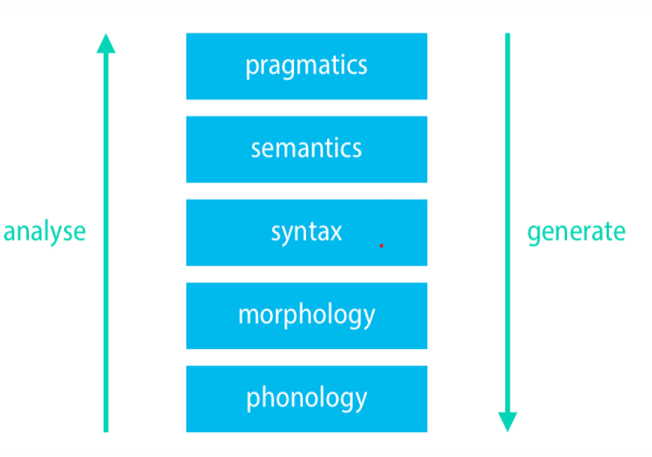
2. Linguistics essentials

- Linguistics is the scientific study of language, and in particular the relationship between **language form** and **language meaning**.
- Besides form and meaning, another important subject of study for linguistics is **how language is used** in context.
- The earliest activities in the documentation of language have been attributed to the 6th-century-BC Indian grammarian who wrote a formal description of the Sanskrit language.



- **Noam Chomsky**, sometimes called **"the father of modern linguistics",** is an American scientist who has started the development of a new framework for the study of language, called **generative linguistics**.
- The kind of structures that are studied in generative linguistics seem to be universal across languages.

levels of linguistic description



2.1 Syntax

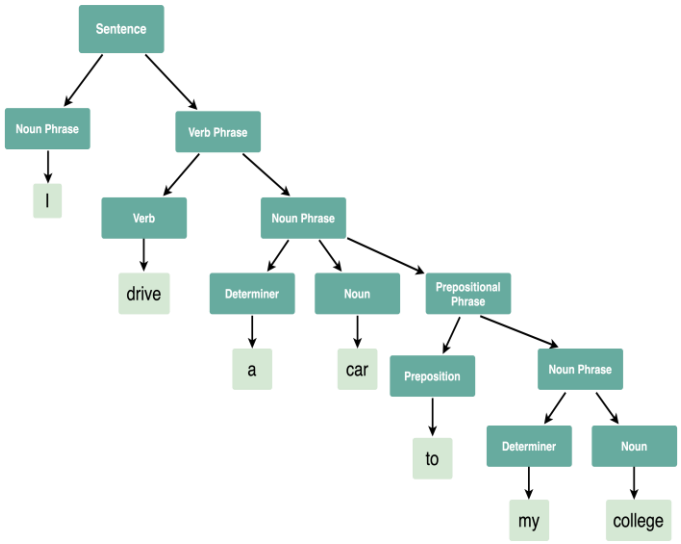
- Syntax studies the rules and costraints that govern how words can be organized into sentences.
- Connection with formal language theory and rewriting grammars.
- Differently from formal language theory, NLP systems need to be robust to input that does not follow the rules of grammar.
- One basic description of ***syntax*** is how different words such as Subject, Verbs, Nouns, Noun Phrases, etc. are sequenced in a sentence.
- Some of the syntactic categories of a natural language are as follows:

  - ✓ Sentence(S)
  - ✓ Noun Phrase(NP)
  - ✓ Determiner(Det)
  - ✓ Verb Phrase(VP)
  - ✓ Prepositional Phrase(PP)
  - ✓ Verb(V)
  - ✓ Noun(N)

Syntax Tree:

A Syntax tree or a parse tree is a tree representation of different syntactic categories of a sentence. It helps us to understand the syntactical structure of a sentence.

Example:
The syntax tree for the sentence given below is as follows:
"I drive a car to my college."



2.2 Semantics

- ❖ Semantics is the **study of the meaning of linguistic expressions** such as words, phrases, and sentences.
- ❖ The focus is on **what expressions conventionally/abstractly mean**, rather than on what they might mean in a particular context. The latter is the focus of pragmatics.
- ❖ Semantic Analysis is a subfield of Natural Language Processing (NLP) that attempts to **understand the meaning of Natural Language**.
- ❖ Semantic Analysis of Natural Language captures the meaning of the given text while taking into account context, logical structuring of sentences and grammar roles.

Parts of Semantic Analysis

Semantic Analysis of Natural Language can be classified into two broad parts:

1.Lexical Semantic Analysis: Lexical Semantic Analysis involves understanding the meaning of each word of the text individually. It basically refers to fetching the dictionary meaning that a word in the text is deputed to carry.

2. Compositional Semantics Analysis: Although knowing the meaning of each word of the text is essential, it is not sufficient to completely understand the meaning of the text.

For example, consider the following two sentences:

- ✓ **Sentence 1:** Students love SVCET.
- ✓ **Sentence 2:** SVCET loves Students.

Although both these sentences 1 and 2 use the same set of root words {student, love, SVCET}, they convey entirely different meanings.

Hence, under Compositional Semantics Analysis, we try to understand how combinations of individual words form the meaning of the text.

Tasks involved in Semantic Analysis (Semantic Analysis Process)

In order to understand the meaning of a sentence, the following are the major processes involved in Semantic Analysis:

1. Word Sense Disambiguation
2. Relationship Extraction

Word Sense Disambiguation:

- In Natural Language, the meaning of a word may vary as per its usage in sentences and the context of the text. Word Sense Disambiguation involves interpreting the meaning of a word based upon the context of its occurrence in a text.
- For example, the word 'Bark' may mean **'the sound made by a dog'** or **'the outermost layer of a tree.'**
- Likewise, the word 'rock' may mean '*a stone*' or '*a genre of music*' - hence, the accurate meaning of the word is highly dependent upon its context and usage in the text.
- Thus, the ability of a machine to overcome the ambiguity involved in identifying the meaning of a word based on its usage and context is called Word Sense Disambiguation.

Relationship Extraction:

Another important task involved in Semantic Analysis is Relationship Extracting.

It involves firstly identifying various entities present in the sentence and then extracting the relationships between those entities.

Elements of Semantic Analysis

Some of the critical elements of Semantic Analysis are:

- **Hyponymy:** Hyponymys refers to a term that is an instance of a generic term. They can be understood by taking class-object as an analogy. **For example:** '*Color*' is a hypernymy while '*grey*', '*blue*', '*red*', etc, are its hyponyms.
- **Homonymy:** Homonymy refers to two or more lexical terms with the same spellings but completely distinct in meaning.
  For example: '*Rose*' might mean '*the past form of rise*' or '*a flower*', - same spelling but different meanings; hence, '*rose*' is a homonymy.
- **Synonymy:** When two or more lexical terms that might be spelt distinctly have the same or similar meaning, they are called Synonymy. **For example**: *(Job, Occupation), (Large, Big), (Stop, Halt)*.
- **Antonymy:** Antonymy refers to a pair of lexical terms that have contrasting meanings - they are symmetric to a semantic axis.
  For example: *(Day, Night), (Hot, Cold), (Large, Small)*.
- **Polysemy:** Polysemy refers to lexical terms that have the same spelling but multiple closely related meanings.
  It differs from homonymy because the meanings of the terms need not be closely related in the case of homonymy.
  For example: '*man*' may mean '*the human species*' or '*a male human*' or '*an adult male human*' - since all these different meanings bear a close association, the lexical term '*man*' is a polysemy.

- **Meronomy:** Meronomy refers to a relationship wherein one lexical term is a constituent of some larger entity.
  For example: '*Wheel*' is a meronym of '*Automobile*'

Basic Units of Semantic System:

In order to accomplish Meaning Representation in Semantic Analysis, it is vital to understand the building units of such representations. The basic units of semantic systems are explained below:

1. **Entity:** An entity refers to a particular unit or individual in specific such as a person or a location.
   For example: Anji, Delhi, etc.
2. **Concept:** A Concept may be understood as a generalization of entities. It refers to a broad class of individual units.
   For example: Learning Portals, City, Students.
3. **Relations:** Relations help establish relationships between various entities and concepts.
   For example: 'SVCET.ORG is a Learning Portal', 'Delhi is a City.', etc.
4. **Predicate:** Predicates represent the verb structures of the sentences.

In Meaning Representation, we employ these basic units to represent textual information.

Approaches to Meaning Representations:

Now that we are familiar with the basic understanding of Meaning Representations, here are some of the most popular approaches to meaning representation:

- ✓ First-order predicate logic (FOPL)
- ✓ Semantic Nets
- ✓ Frames
- ✓ Conceptual dependency (CD)
- ✓ Rule-based architecture
- ✓ Case Grammar
- ✓ Conceptual Graphs

Semantic Analysis Techniques

Based upon the end goal one is trying to accomplish; Semantic Analysis can be used in various ways. Two of the most common Semantic Analysis techniques are:

1. **Text Classification**
2. **Text Extraction**

Text Classification

In-Text Classification, our aim is to label the text according to the insights we intend to gain from the textual data.

For example:

- In **Sentiment Analysis,** we try to label the text with the prominent emotion they convey. It is highly beneficial when analyzing customer reviews for improvement.

- In **Topic Classification**, we try to categories our text into some predefined categories.
  For example: Identifying whether a research paper is of Physics, Chemistry, Maths or Computers
- In **Intent Classification**, we try to determine the intent behind a text message.
  For example: Identifying whether an e-mail received at customer care service is a query, complaint or request.

Text Extraction

In-Text Extraction, we aim at obtaining specific information from our text.

For Example,

- In **Keyword Extraction**, we try to obtain the essential words that define the entire document.
- In **Entity Extraction**, we try to obtain all the entities involved in a document

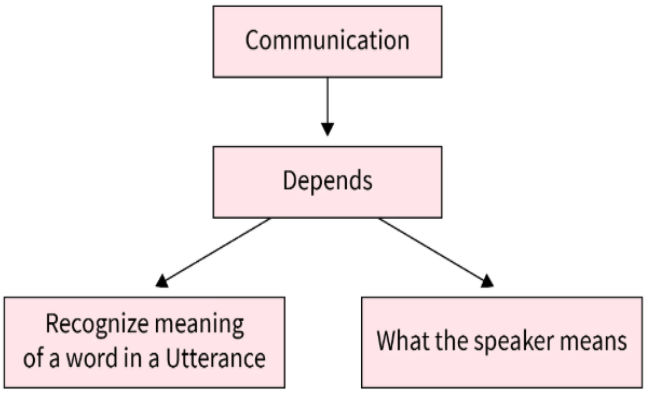
2.3 Pragmatics

- ❖ Pragmatics studies the way linguistic expressions with their semantic meanings are used for specific communicative goals.
- ❖ In contrast to semantics, pragmatics explicitly asks the question what an expression means in a given context.
- ❖ An important concept in pragmatics is **the speech act**, which describes an action performed through language.
- ❖ **Example :** Can you pass the Pen? is a requesting act

Difference Between Pragmatics and Semantics

Semantics is one area of linguistics, whereas pragmatics is another. Pragmatics focuses on **concerns of usage,** whereas Semantics is all about **questions of meaning**. It addresses the part of the meaning that depends on context.

Semantics examines the mystery of what signs mean. Pragmatics, on the other hand, examines how signs relate to their users and interpreters.



Criteria	Semantics	Pragmatics
Definition	The word semantics comes from the Greek word seme, which means to sigh. Another significant area connected to theoretical linguistics is semantics. It all comes down to understanding how language terms are meant.	Pragmatics understands the language's meaning but keeps the context in mind.
Focus	Meaning	Use of Language
Scope	Since it deals with only the meaning, narrow.	Since pragmatics deals with aspects beyond text, the scope is broad.
Meaning of an utterance	Context independent	Context Dependent
Governed by	General Rules	Principles

Pragmatics and NLP: Discourse Processing

While talking about Pragmatics in NLP, we've been going on about context and its importance in actually processing text.

There are 3 main types of context:

- **Discourse context:** Discourse context is where an utterance sits about a document, conversation, speech, etc.
- **Physical context:** Provides information about the physical aspect
- **Social context:** Social context is all about answering questions such as - who is speaking, who are they speaking to, and, what are they trying to achieve.

To understand this better, let's take two pairs of sentences.

"Arun hide my car keys. He was drunk."
And,
"Arun hide my car keys. He likes Apples."

In the above example, we have two pairs of statements. The first pair are related. It explains why the keys were hidden.

However, the second pair of statements are not related or do not have any meaningful connection.

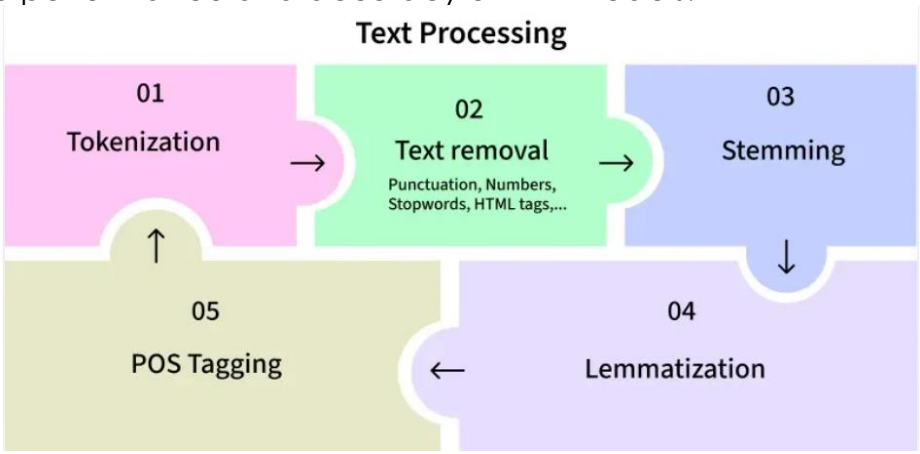
3. Text Preprocessing in NLP

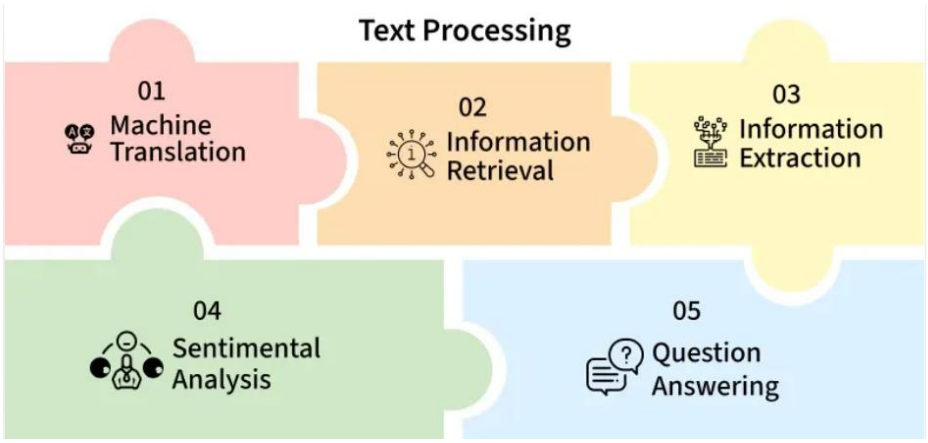
Text processing:

❖ Natural Language Processing (NLP) has advanced significantly and now plays an important role in multiple real-world applications like chatbots, search engines and sentiment analysis.

❖ An early step in any NLP workflow is **text preprocessing**, which prepares raw textual data for further analysis and modeling.

❖ Text processing involves cleaning and preparing raw text data for further analysis or model training. Proper text preprocessing can significantly impact the performance and accuracy of NLP models.





Text Preprocessing Techniques in NLP

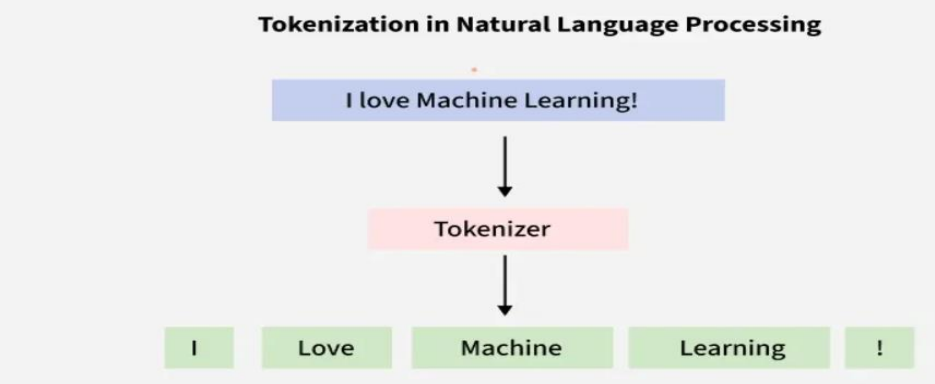
- **Regular Expressions:** Regular expressions (regex) is an important tool in text preprocessing for Natural Language Processing (NLP). They allow for efficient and flexible pattern matching and text manipulation.
- **Tokenization:** Tokenization is the process of breaking down text into smaller units such as words or sentences. This is an important step in NLP as it transforms raw text into a structured format that can be further analyzed.
- **Lemmatization and Stemming:** Lemmatization and stemming are techniques used in NLP to reduce words to their base or root forms. This

process is important for tasks like text normalization, information retrieval and text mining.

- **Parts of Speech (POS):** Parts of Speech (POS) tagging involves labeling each word in a sentence with its corresponding part of speech such as noun, verb, adjective etc. This information is crucial for many NLP applications, including parsing, information retrieval and text analysis.

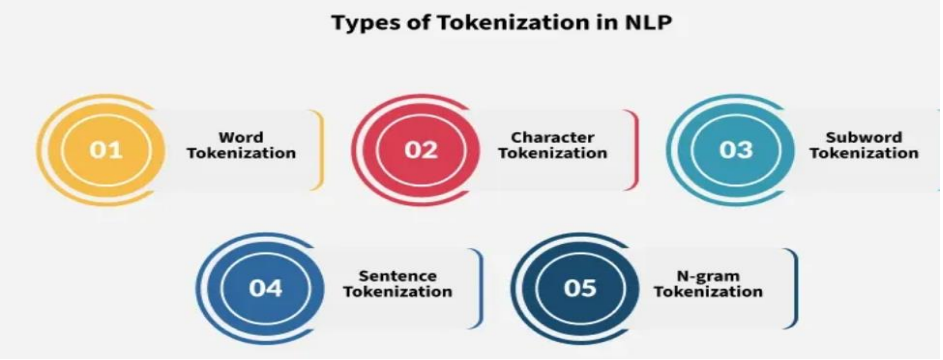
3.1 Tokenization in NLP

- Tokenization is a fundamental step in Natural Language Processing (NLP).
- It involves dividing a Textual input into smaller units known as tokens.
- These tokens can be in the form of words, characters, sub-words, or sentences.
- It helps in improving interpretability of text by different models.
- Let's understand How Tokenization Works.



- Uses a tokenizer to segment unstructured data and natural language text into distinct chunks of information, treating them as different elements.
- **Tokens:** Words or Sub-words in the context of natural language processing.
  Example: A word is a token in a sentence, A character is a token in a word, etc.
- Application: Multiple NLP tasks, text processing, language modelling, and machine translation.

Types of Tokenization



Tokenization can be classified into several types based on how the text is segmented. Here are some types of tokenization:

- Word Tokenization
- Character Tokenization
- Sub-word Tokenization
- Sentence Tokenization
- N-gram Tokenization

Word Tokenization

Word tokenization is the most commonly used method where text is divided into individual words. It works well for languages with clear word boundaries, like English.

For example: "Machine learning is fascinating" becomes:

Input before tokenization: *["Machine Learning is fascinating"]*
Output when tokenized by words: *["Machine", "learning", "is", "fascinating"]*

Character Tokenization

- In Character Tokenization, the textual data is split and converted to a sequence of individual characters.
- This is beneficial for tasks that require a detailed analysis, such as spelling correction or for tasks with unclear boundaries.
- It can also be useful for modelling character-level language.

Example:

Input before tokenization: *["You are helpful"]*
Output when tokenized by characters: *["Y", "o", "u", " ", "a", "r", "e", " ", "h", "e", "l", "p", "f", "u", "l"]*

Sub-word Tokenization

This strikes a balance between word and character tokenization by breaking down text into units that are larger than a single character but smaller than a full word. This is useful when dealing with morphologically rich languages or rare words.

Example

[“Btech”, “CSM”]
[“Btech”, “CAI”]
[“Time”, “table”]
[“Rain”, “coat”]
[“Grace”, “fully”]
[“Run”, “way”]

Sub-word tokenization helps to handle out-of-vocabulary words in NLP tasks and for languages that form words by combining smaller units.

Sentence Tokenization: Sentence tokenization is also a common technique used to make a division of paragraphs or large set of sentences into separated sentences as tokens. This is useful for tasks requiring individual sentence analysis or processing.

Example:
***Input before tokenization**: ["Artificial Intelligence is an emerging technology. Machine learning is fascinating. Computer Vision handles images. "]*
***Output when tokenized by sentences** ["Artificial Intelligence is an emerging technology.", "Machine learning is fascinating.", "Computer Vision handles images."]*

N-gram Tokenization
N-gram tokenization splits words into fixed-sized chunks (size = n) of data.

Example:
***Input before tokenization**: ["Machine learning is powerful"]*
***Output when tokenized by bigrams**: [('Machine', 'learning'), ('learning', 'is'), ('is', 'powerful')]*

Techniques for Tokenization
We can also implement tokenization using following methods and libraries:

- **Spacy:** Spacy is NLP library that provide robust tokenization capabilities.
- **BERT tokenizer:** BERT uses Word Piece tokenizer, which is a type of sub-word tokenizer for tokenizing input text. Using regular expressions allows for more fine-grained control over tokenization, and you can customize the pattern based on your specific requirements.
- **Byte-Pair Encoding:** Byte Pair Encoding (BPE) is a data compression algorithm that has also found applications in the field of natural language processing, specifically for tokenization. It is a Sub-word Tokenization technique that works by iteratively merging the most frequent pairs of consecutive bytes (or characters) in a given corpus.
- **Sentence Piece:** Sentence Piece is another sub-word tokenization algorithm commonly used for natural language processing tasks. It is designed to be language-agnostic and works by iteratively merging frequent sequences of characters or sub words in a given corpus.

3.2 Lemmatization

Python - Lemmatization Approaches with Examples

Lemmatization:

- lemmatization is a lot more powerful.
- It looks beyond word reduction and considers a language's full vocabulary to apply a morphological analysis to words, aiming to remove inflectional endings only and to return the base or dictionary form of a word, which is known as the lemma.
- For clarity, look at the following examples given below:

Original Word	--->	Root Word (lemma)	Feature
meeting	--->	meet	(core-word extraction)

was ---> be (tense conversion to present tense)
mice ---> mouse (plural to singular)

TIP: Always convert your text to lowercase before performing any NLP task including lemmatizing.

Various Approaches to Lemmatization:

9 different approaches to perform Lemmatization along with multiple examples and code implementations.

1. WordNet
2. WordNet (with POS tag)
3. TextBlob
4. TextBlob (with POS tag)
5. spaCy
6. TreeTagger
7. Pattern
8. Gensim
9. Stanford CoreNLP

1. Wordnet Lemmatizer

Wordnet is a publicly available lexical database of over 200 languages that provides semantic relationships between its words. It is one of the earliest and most commonly used lemmatizer technique.

- It is present in the nltk library in python.
- Wordnet links words into semantic relations. (eg. synonyms)
- It groups synonyms in the form of synsets.
  - **synsets :** a group of data elements that are semantically equivalent.

How to use:

1. Download nltk package : In our anaconda prompt or terminal, type: pip install nltk
2. Download Wordnet from nltk : In our python console, do the following :

```
import nltk
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
```

2. Wordnet Lemmatizer (with POS tag)

In the above approach, we observed that Wordnet results were not up to the mark. Words like 'sitting', 'flying' etc remained the same after lemmatization. This is because these words are treated as a noun in the given sentence rather than a verb. To overcome come this, we use POS (Part of Speech) tags.
We add a tag with a particular word defining its type (verb, noun, adjective etc).

For Example,

Word	+	Type (POS tag)	--->	Lemmatized Word
driving	+	verb 'v'	--->	drive
dogs	+	noun 'n'	--->	dog

3. TextBlob

TextBlob is a python library used for processing textual data. It provides a simple API to access its methods and perform basic NLP tasks.
Download TextBlob package : In your anaconda prompt or terminal, type:
pip install textblob

4. TextBlob (with POS tag)

Same as in Wordnet approach without using appropriate POS tags, we observe the same limitations in this approach as well. So, we use one of the more powerful aspects of the TextBlob module the 'Part of Speech' tagging to overcome this problem.

5. spaCy

spaCy is an open-source python library that parses and "understands" large volumes of text. Separate models are available that cater to specific languages (English, French, German, etc.).

6. TreeTagger

The TreeTagger is a tool for annotating text with part-of-speech and lemma information. The TreeTagger has been successfully used to tag over 25 languages and is adaptable to other languages if a manually tagged training corpus is available.

Word	**POS**	**Lemma**
the	DT	the
TreeTagger	NP	TreeTagger
is	VBZ	be
easy	JJ	easy
to	TO	to

Word	POS	Lemma
use	VB	use
.	SENT	.

7. Pattern

Pattern is a Python package commonly used for **web mining**, natural language processing, machine learning, and network analysis. It has many useful NLP capabilities. It also contains a special feature which we will be discussing below.

8. Gensim

Gensim is designed to handle large text collections using data streaming. Its lemmatization facilities are based on the pattern package we installed above.

- gensim.utils.lemmatize() function can be used for performing Lemmatization. This method comes under the utils module in python.
- We can use this lemmatizer from pattern to extract UTF8-encoded tokens in their base form=lemma.
- Only considers nouns, verbs, adjectives, and adverbs by default (all other lemmas are discarded).

Example:

Word ---> Lemmatized Word
are/is/being ---> be
saw ---> see

9. Stanford CoreNLP

CoreNLP enables users to derive linguistic annotations for text, including token and sentence boundaries, parts of speech, named entities, numeric and time values, dependency and constituency parses, sentiment, quote attributions, and relations.

- CoreNLP is our one stop shop for natural language processing in Java!
- CoreNLP currently supports 6 languages, including Arabic, Chinese, English, French, German, and Spanish.

How to use:

1. Get JAVA 8 : Download Java 8 (as per your OS) and install it.

2. Get Stanford_coreNLP package :
   2.1) Download Stanford_CoreNLP and unzip it.
   2.2) Open terminal

(a) go to the directory where you extracted the above file by doing
cd C:\Users\...\stanford-corenlp-4.1.0 on terminal

(b) then, start your Stanford CoreNLP server by executing the following command on terminal:
java -mx4g -cp "*" edu.stanford.nlp.pipeline.StanfordCoreNLPServer -annotators "tokenize, ssplit, pos, lemma, parse, sentiment" -port 9000 -timeout 30000
(leave your terminal open as long as you use this lemmatizer)

3. Download Stanford CoreNLP package: Open your anaconda prompt or terminal, type.

3.3 Stemming

Stemming:

- Simplifying words to their most basic form is called stemming, and it is made easier by stemmers or stemming algorithms.
- For example, "chocolates" becomes "chocolate" and "retrieval" becomes "retrieve." This is crucial for pipelines for natural language processing, which use tokenized words that are acquired from the first stage of dissecting a document into its constituent words.
- Stemming in natural language processing reduces words to their base or root form, aiding in text normalization for easier processing. This technique is crucial in tasks like text classification, information retrieval, and text summarization.
- While beneficial, stemming has drawbacks, including potential impacts on text readability and occasional inaccuracies in determining the correct root form of a word.

Types of Stemmer in NLTK

Python NLTK contains a variety of stemming algorithms, including several types. Some of them are:

1. Porter's Stemmer
2. Lovins Stemmer
3. Dawson Stemmer
4. Krovetz Stemmer
5. Xerox Stemmer
6. N-Gram Stemmer
7. Snowball Stemmer
8. Lancaster Stemmer
9. Regexp Stemmer

1. Porter's Stemmer

- It is one of the most popular stemming methods proposed in 1980. It is based on the idea that the suffixes in the English language are made up of a combination of smaller and simpler suffixes.

- This stemmer is known for its speed and simplicity. The main applications of Porter Stemmer include data mining and Information retrieval. However, its applications are only limited to English words.
- The algorithms are fairly lengthy in nature and are known to be the oldest stemmer.
- **Example:** EED -> EE means "if the word has at least one vowel and consonant plus EED ending, change the ending to EE" as 'agreed' becomes 'agree'.

Implementation of Porter Stemmer

```
from nltk.stem import PorterStemmer

# Create a Porter Stemmer instance
porter_stemmer = PorterStemmer()

# Example words for stemming
words = ["running", "jumps", "happily", "running", "happily"]

# Apply stemming to each word
stemmed_words = [porter_stemmer.stem(word) for word in words]

# Print the results
print("Original words:", words)
print("Stemmed words:", stemmed_words)
```

Output:

```
Original words: ['running', 'jumps', 'happily', 'running', 'happily']
Stemmed words: ['run', 'jump', 'happili', 'run', 'happili']
```

- Advantage: It produces the best output as compared to other stemmers and it has less error rate.
- Limitation: Morphological variants produced are not always real words.

2. Lovins Stemmer

It is proposed by Lovins in 1968, that removes the longest suffix from a word then the word is recorded to convert this stem into valid words. **Example:** sitting -> sitt -> sit

- Advantage: It is fast and handles irregular plurals like 'teeth' and 'tooth' etc.
- Limitation: It is time consuming and frequently fails to form words from stem.

3. Dawson Stemmer

It is an extension of Lovins stemmer in which suffixes are stored in the reversed order indexed by their length and last letter.

- Advantage: It is fast in execution and covers more suffices.
- Limitation: It is very complex to implement.

4. Krovetz Stemmer

It was proposed in 1993 by Robert Krovetz. Following are the steps: 1) Convert the plural form of a word to its singular form.

2) Convert the past tense of a word to its present tense and remove the suffix 'ing'.

Example: 'children' -> 'child'

- Advantage: It is light in nature and can be used as pre-stemmer for other stemmers.
- Limitation: It is inefficient in case of large documents.

5. Xerox Stemmer

Capable of processing extensive datasets and generating valid words, it has a tendency to over-stem, primarily due to its reliance on lexicons, making it language-dependent. This constraint implies that its effectiveness is limited to specific languages.

Example:

'children' -> 'child'
'understood' -> 'understand'
'whom' -> 'who'
'best' -> 'good'

6. N-Gram Stemmer

- The algorithm, aptly named n-grams (typically n=2 or 3), involves breaking words into segments of length n and then applying statistical analysis to identify patterns.
- An n-gram is a set of n consecutive characters extracted from a word in which similar words will have a high proportion of n-grams in common.
  Example: 'INTRODUCTIONS' for n=2 becomes : *I, IN, NT, TR, RO, OD, DU, UC, CT, TI, IO, ON, NS, S*

Advantage: It is based on string comparisons and it is language dependent.
Limitation: It requires space to create and index the n-grams and it is not time efficient.

7. Snowball Stemmer

- The Snowball Stemmer, compared to the Porter Stemmer, is multi-lingual as it can handle non-English words.
- It supports various languages and is based on the 'Snowball' programming language, known for efficient processing of small strings.
- The Snowball stemmer is way more aggressive than Porter Stemmer and is also referred to as Porter2 Stemmer.
- Because of the improvements added when compared to the Porter Stemmer, the Snowball stemmer is having greater computational speed.

8. Lancaster Stemmer

- The Lancaster stemmers are more aggressive and dynamic compared to the other two stemmers. The stemmer is really faster, but the algorithm is really confusing when dealing with small words. But they are not as efficient as Snowball Stemmers.
- The Lancaster stemmers save the rules externally and basically uses an iterative algorithm.

9. Regexp Stemmer

- The Regexp Stemmer, or Regular Expression Stemmer, is a stemming algorithm that utilizes regular expressions to identify and remove suffixes from words. \
- It allows users to define custom rules for stemming by specifying patterns to match and remove.
- This method provides flexibility and control over the stemming process, making it suitable for specific applications where custom rule-based stemming is desired.

Applications of Stemming

1. Stemming is used in information retrieval systems like search engines.
2. It is used to determine domain vocabularies in domain analysis.
3. To display search results by indexing while documents are evolving into numbers and to map documents to common subjects by stemming.
4. Sentiment Analysis, which examines reviews and comments made by different users about anything, is frequently used for product analysis, such as for online retail stores. Before it is interpreted, stemming is accepted in the form of the text-preparation mean.
5. A method of group analysis used on textual materials is called document clustering or text clustering. Important uses of it include subject extraction, automatic document structuring, and quick information retrieval.

Disadvantages in Stemming

There are mainly two errors in stemming –

Over-stemming:

- Over-stemming in natural language processing occurs when a stemmer produces incorrect root forms or non-valid words.
- This can result in a loss of meaning and readability.
- For instance, "arguing" may be stemmed to "argu," losing meaning.
- To address this, choosing an appropriate stemmer, testing on sample text, or using lemmatization can mitigate over-stemming issues. Techniques like semantic role labelling and sentiment analysis can enhance context awareness in stemming.

Under-stemming:

- Under-stemming in natural language processing arises when a stemmer fails to produce accurate root forms or reduce words to their base form.
- This can result in a loss of information and hinder text analysis.
- For instance, stemming "arguing" and "argument" to "argu" may lose meaning.
- To mitigate under-stemming, selecting an appropriate stemmer, testing on sample text, or opting for lemmatization can be beneficial.
- Techniques like semantic role labelling and sentiment analysis enhance context awareness in stemming.

3.4 Stopword Removal

Stop Words:

- ❖ A stop word is a commonly used word (such as "the", "a", "an", or "in") that a search engine has been programmed to ignore, both when indexing entries for searching and when retrieving them as the result of a search query.
- ❖ We would not want these words to take up space in our database or take up valuable processing time.
- ❖ For this, we can remove them easily, by storing a list of words that we consider to stop words.
- ❖ NLTK(Natural Language Toolkit) in Python has a list of stopwords stored in 16 different languages.
- ❖ we can find them in the nltk_data directory.

Sample text with Stop Words	Without Stop Words
GeeksforGeeks – A Computer Science Portal for Geeks	GeeksforGeeks , Computer Science, Portal ,Geeks
Can listening be exhausting?	Listening, Exhausting
I like reading, so I read	Like, Reading, read

Types of Stopwords

- ❖ Stopwords are frequently occurring words in a language that are frequently omitted from natural language processing (NLP) tasks due to their low significance for deciphering textual meaning.
- ❖ The particular list of stopwords can change based on the language being studied and the context.
- ❖ The following is a broad list of stopword categories:
  - ➢ **Common Stopwords:** These are the most frequently occurring words in a language and are often removed during text preprocessing. Examples: "the," "is," "in," "for," "where," "when," "to," "at," etc.
  - ➢ **Custom Stopwords:** Depending on the specific task or domain, additional words may be considered as stopwords. These could be domain-specific terms that don't contribute much to the overall meaning. For example, in a medical context, words like "patient" or "treatment" might be considered as custom stopwords.
  - ➢ **Numerical Stopwords:** Numbers and numeric characters may be treated as stopwords in certain cases, especially when the analysis is focused on the meaning of the text rather than specific numerical values.
  - ➢ **Single-Character Stopwords:** Single characters, such as "a," "I," "s," or "x," may be considered stopwords, particularly in cases where they don't convey much meaning on their own.

- **Contextual Stopwords:** Words that are stopwords in one context but meaningful in another may be considered as contextual stopwords. For instance, the word "will" might be a stopword in the context of general language processing but could be important in predicting future events.

Stop Words Removal with NLTK

In **natural language processing (NLP)**, **stopwords** are frequently filtered out to enhance text analysis and computational efficiency.
Eliminating stopwords can improve the accuracy and relevance of NLP tasks by drawing attention to the more important words, or content words.
Checking English Stopwords List
An English stopwords list typically includes common words that carry little semantic meaning and are often excluded during text analysis.
To check the list of stopwords, type the following commands in the python shell.

```
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')
print(stopwords.words('english'))
```

Output:

```
['i', 'me', 'my', 'myself', 'we', 'our', 'ours',
'ourselves', 'you', "you're", "you've", "you'll", "you'd", 'your', 'yours', 'yourself',
'yourselves', 'he', 'him', 'his', 'himself', 'she', "she's", 'her', 'hers', 'herself',
'it', "it's", 'its', 'itself', 'they', 'them', 'their', 'theirs', 'themselves',
 'what', 'which', 'who', 'whom', 'this', 'that', "that'll", 'these', 'those', 'am', 'is', 'are',
 'was', 'were', 'be', 'been', 'being', 'have', 'has', 'had', 'having', 'do', 'does', 'did',
'doing',
 'a', 'an', 'the', 'and', 'but', 'if', 'or', 'because', 'as', 'until', 'while', 'of', 'at', 'by', 'for',
'with', 'about', 'against', 'between', 'into', 'through', 'during', 'before', 'after',
'above', 'below', 'to',
'from', 'up', 'down', 'in', 'out', 'on', 'off', 'over', 'under', 'again', 'further', 'then',
'once',
'here', 'there', 'when', 'where', 'why', 'how', 'all', 'any', 'both', 'each', 'few',
'more', 'most',
'other', 'some', 'such', 'no', 'nor', 'not', 'only', 'own', 'same', 'so', 'than', 'too',
'very',
's', 't', 'can', 'will', 'just', 'don', "don't", 'should', "should've", 'now', 'd', 'll', 'm', 'o',
're', 've', 'y',
 'ain', 'aren', "aren't", 'couldn', "couldn't", 'didn', "didn't", 'doesn', "doesn't",
'hadn', "hadn't", 'hasn', "hasn't", 'haven', "haven't", 'isn', "isn't", 'ma', 'mightn',
 "mightn't", 'mustn', "mustn't", 'needn', "needn't", 'shan', "shan't", 'shouldn',
"shouldn't",
'wasn', "wasn't", 'weren', "weren't", 'won', "won't", 'wouldn', "wouldn't"]
```

Removing common stop words from the tokens.

- Imports the list of English stopwords from nltk.corpus.stopwords.
- Downloads the stopwords corpus using nltk.download('stopwords').
- Stores all English stopwords (like "the", "is", "and", etc.) in a set called stop_words for fast lookup.
- Iterates over each document in tokenized_corpus and removes all stopwords.
- Saves the cleaned, non-stopword tokens into filtered_corpus.
- Prints the a list of documents.

```
from nltk.corpus import stopwords
nltk.download('stopwords')

stop_words = set(stopwords.words('english'))
filtered_corpus = [[word for word in doc if word not in stop_words] for doc in tokenized_corpus]
print(filtered_corpus)
```

Output:

[nltk_data] Downloading package stopwords to /root/nltk_data...
[['cant', 'wait', 'new', 'season', 'favorite', 'show'], ['covid', 'pandemic', 'affected', 'millions', 'people', 'worldwide'], ['us', 'stocks', 'fell', 'friday', 'news', 'rising', 'inflation'], ['htmlbodywelcome', 'websitebodyhtml'], ['python', 'great', 'programming', 'language']]
[nltk_data] Unzipping corpora/stopwords.zip

3.5 Text normalization

- Text normalization is a key step in natural language processing (NLP). It involves cleaning and preprocessing text data to make it consistent and usable for different NLP tasks.
- The process includes a variety of techniques, such as case normalization, punctuation removal, stop word removal, stemming, and lemmatization.

Steps to carry out text normalization in NLP

1. Case Normalization
2. Punctuation Removal
3. Stop Word Removal
4. Stemming
5. Lemmatization
6. Tokenization
7. Replacing synonyms and Abbreviation to their full form to normalize the text in NLP
8. Removing numbers and symbol to normalize the text in NLP
9. Removing any remaining non-textual elements to normalize the text in NLP

1. Case Normalization

Case normalization is converting all text to lowercase or uppercase to standardize the text. This technique is useful when working with text data that contains a mix of uppercase and lowercase letters.

Example text normalization

Input: "The quick BROWN Fox Jumps OVER the lazy dog."
Output: "the quick brown fox jumps over the lazy dog."

Text normalization code in Python

```
text = "The quick BROWN Fox Jumps OVER the lazy dog."
text = text.lower()
print(text)
```

2. Punctuation Removal

Punctuation removal is the process of removing special characters and punctuation marks from the text. This technique is useful when working with text data containing many punctuation marks, which can make the text harder to process.

Example text normalization

Input: "The quick BROWN Fox Jumps OVER the lazy dog!!!"
Output: "The quick BROWN Fox Jumps OVER the lazy dog"

Text normalization code in Python

```
import string
text = "The quick BROWN Fox Jumps OVER the lazy dog!!!"
text = text.translate(text.maketrans("", "", string.punctuation))
print(text)
```

3. Stop Word Removal

Stop word removal is the process of removing common words with little meaning, such as "the" and "a". This technique is useful when working with text data containing many stop words, which can make the text harder to process.

Example text normalization

Input: "The quick BROWN Fox Jumps OVER the lazy dog."
Output: "quick BROWN Fox Jumps OVER lazy dog."

Text normalization code in Python

```
from nltk.corpus import stopwords
text = "The quick BROWN Fox Jumps OVER the lazy dog."
stop_words = set(stopwords.words("english"))
words = text.split()
filtered_words = [word for word in words if word not in stop_words]
text = " ".join(filtered_words)
print(text)
```

4. Stemming

Stemming is reducing words to their root form by removing suffixes and prefixes, such as "running" becoming "run". This method is helpful when working with text data that has many different versions of the same word, which can make the text harder to process.

Example text normalization

Input: "running,runner,ran"
Output: "run,run,run"

5. Lemmatization

Lemmatization is reducing words to their base form by considering the context in which they are used, such as "running" becoming "run". This technique is similar to stemming, but it is more accurate as it considers the context of the word.

Example text normalization
Input: "running,runner,ran"
Output: "run,runner,run"

6. Tokenization

Tokenization is the process of breaking text into individual words or phrases, also known as "tokens". This technique is useful when working with text data that needs to be analyzed at the word or phrase level, such as in text classification or language translation tasks.

Example text normalization
Input: "The quick BROWN Fox Jumps OVER the lazy dog."
Output: ["The", "quick", "BROWN", "Fox", "Jumps", "OVER", "the", "lazy", "dog."]

7. Replacing synonyms and Abbreviation to their full form to normalize the text in NLP

This technique is useful when working with text data that contains synonyms or abbreviations that need to be replaced by their full form.

Example text normalization
Input: "I'll be there at 2pm"
Output: "I will be there at 2pm"

8. Removing numbers and symbol to normalize the text in NLP

This technique is useful when working with text data that contain numbers and symbols that are not important for the NLP task.

Example text normalization
Input: "I have 2 apples and 1 orange #fruits"
Output: "I have apples and orange fruits"

9. Removing any remaining non-textual elements to normalize the text in NLP

Removing any remaining non-textual elements such as HTML tags, URLs, and email addresses This technique is useful when working with text data that contains non-textual elements such as HTML tags, URLs, and email addresses that are not important for the NLP task.

Example text normalization

Input: "Please visit example.com for more information or contact me at info@example.com"
Output: "Please visit for more information or contact me at "

Keyword normalization techniques in NLP:

- The above steps for normalising text in NLP can also all be used on a list of keywords or phrases.

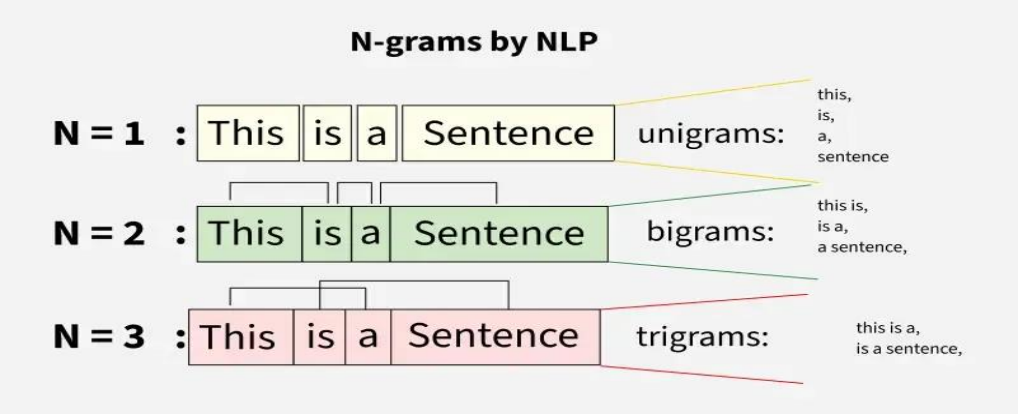
- They be used to make keywords and phrases more consistent, more easily searchable, and more usable for natural language processing tasks such as text classification, information retrieval, and natural language understanding

3.6 N-gram in NLP

N-gram is a contiguous sequence of 'N' items like words or characters from text or speech. The items can be letters, words or base pairs according to the application. The value of 'N' determines the order of the N-gram. They are fundamental concept used in various NLP tasks such as language modeling, text classification, machine translation and more.

N-grams can be of various types based on the value of 'n':

- Unigrams (1-grams) are single words
- Bigrams (2-grams) are pairs of consecutive words
- Trigrams (3-grams) are triplets of consecutive words



N-grams in NLP are used for:

- **Capturing Context and Semantics**: N-grams help us understand how words work together in a sentence. By analyzing small word combinations they provide insight into the meaning and flow of language making text interpretation more accurate.
- **Improving Language Models**: In tools like translation systems or voice assistants N-grams help create smarter models that can better guess what comes next in a sentence, leading to more natural and accurate responses.
- **Enhancing Text Prediction**: They are widely used in predictive typing. By analyzing the words you've already typed they help suggest what you're likely to type next making writing faster and more intuitive.
- **Information Retrieval**: When searching for information they helps to find and rank documents by recognizing important word patterns. This makes search engines more effective at delivering relevant results.

Implementation of N-grams

- **text.split():** Splits the text into a list of words (tokens).

- **[tuple(tokens[i:i + n]) for i in range(len(tokens) - n + 1)]:** Generates n-grams by creating tuples of consecutive words.
- **return ngrams:** Returns the list of generated n-grams.

Laplace Smoothing for N-grams

- When working with N-grams, one of the major challenges is data sparsity especially with higher-order N-grams like 4-grams or 5-grams.
- As the value of N increases the number of possible N-grams grows exponentially and many of them may not appear in the training data resulting in zero probabilities for unseen sequences.
- To resolve this we use Laplace Smoothing also known as Additive Smoothing.
- The formula for Laplace smoothing is follows:

Smoothed Count=count+1/total N-grams+vocab

Where

- count is the frequency of a particular N-gram in dataset.
- total N-grams is the number of N-grams in the dataset.
- vocab size is the total number of unique words.

This formula ensures that even N-grams that never appeared in the training data will have a non-zero probability.

Comparison Table: N-grams vs. Other NLP models:

Here we would be comparing N-gram models with various other NLP models like HMM, RNN or Transformer-based Models.

Feature / Aspect	N-gram Models	HMM (Hidden Markov Model)	RNN (Recurrent Neural Network)	Transformer-based Models
Context Window	Fixed-size (N words)	Limited, depends on state transitions	Flexible (remembers previous states)	Very large (Global attention)
Semantic Understanding	Very limited	Weak	Moderate	Good
Data Efficiency	Good with small data	Good with small data	Needs more data	Needs large data
Speed and Simplicity	Fast and simple	Moderate	Slower than N-grams	Slow

Feature / Aspect	N-gram Models	HMM (Hidden Markov Model)	RNN (Recurrent Neural Network)	Transformer-based Models
Interpretability	Easy to understand	Moderate	Hard to interpret	Black-box
Use Cases	Basic NLP tasks	POS tagging, sequence labeling	Language modeling, sequence labeling	Translation, summarization, QA

Applications of N-grams

- **Language Modelling**: They predict the next word in a sentence based on the previous words helping generate relevant text in tasks like text generation, chatbots and autocomplete systems.
- **Text Prediction**: In predictive typing they suggest the next word based on recent input, improving typing speed and user experience in apps like mobile keyboards and messaging tools.
- **Sentiment and Text Classification**: N-grams capture word sequences to classify text into categories or sentiments making it easier to identify tone and topics like sports or politics.
- **Plagiarism Detection**: By comparing N-grams in documents systems can spot similar patterns helping detect copied or reworded content.
- **Speech Recognition**: In speech-to-text systems they predict the next word hence enhancing transcription accuracy with contextually correct sequences.

Advantages of N-grams in NLP

- **Simple and Easy to Implement:** They are simple to understand and implement and they require minimal computational resources. They are suitable for baseline modeling and quick prototyping.
- **Low Computational Overhead:** They are computationally lightweight and easy to scale when compared to neural approaches which makes them suitable for systems with limited processing power or for tasks which require rapid prototyping.
- **Preservation of Local Word Order:** They capture short-range dependencies between words by preserving their immediate sequence which is beneficial in modeling syntactic and patterns such as negation ("not good") or phrasal constructs ("New York City").
- **Strong Baseline Performance:** They are simple yet they often provide competitive baselines for a range of tasks including text classification, sentiment analysis, information retrieval and topic detection.

Challenges and Limitations

Despite their benefits N-grams also has some challenges like:

- **Data sparsity**: With larger N-grams it becomes less likely to find repeated instances of the same sequence leading to sparse data.
- **Lack of semantic understanding:** While N-grams are good at recognizing patterns they lack the understanding of context beyond the sequences they were trained on.
- **Lack of long-range context:** They only consider nearby words and ignore broader sentence meaning.

3.7 POS (Parts-Of-Speech) Tagging in NLP

- One of the core tasks in **Natural Language Processing (NLP)** is **Parts of Speech (PoS) tagging**, which is giving each word in a text a grammatical category, such as nouns, verbs, adjectives, and adverbs.
- Through improved comprehension of phrase structure and semantics, this technique makes it possible for machines to study and comprehend human language more accurately.
- In many NLP applications, including machine translation, sentiment analysis, and information retrieval, PoS tagging is essential.
- PoS tagging serves as a link between language and machine understanding, enabling the creation of complex language processing systems and serving as the foundation for advanced linguistic analysis.

Definition of POS Tagging:

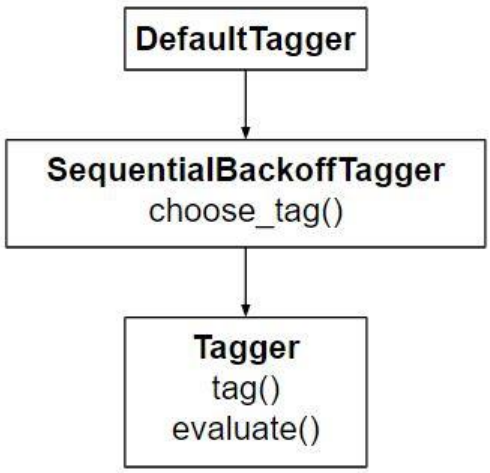
Parts of Speech tagging is a linguistic activity in Natural Language Processing (NLP) wherein each word in a document is given a particular part of speech (adverb, adjective, verb, etc.) or grammatical category.This procedure makes it easier to comprehend the sentence's structure and meaning.

In NLP applications, POS tagging is useful for machine translation, named entity recognition, and information extraction, among other things. It also works well for clearing out ambiguity in terms with numerous meanings and revealing a sentence's grammatical structure.

Part of Speech	Tag
Noun	n
Verb	v
Adjective	a
Adverb	r

Default tagging:

Default tagging is a basic step for the part-of-speech tagging. It is performed using the DefaultTagger class. The DefaultTagger class takes 'tag' as a single argument. **NN** is the tag for a singular noun. DefaultTagger is most useful when it gets to work with most common part-of-speech tag. that's why a noun tag is recommended.



.

Example of POS Tagging

Consider the sentence: "The quick brown fox jumps over the lazy dog."

After performing POS Tagging:

- "The" is tagged as determiner (DT)
- "quick" is tagged as adjective (JJ)
- "brown" is tagged as adjective (JJ)
- "fox" is tagged as noun (NN)
- "jumps" is tagged as verb (VBZ)
- "over" is tagged as preposition (IN)
- "the" is tagged as determiner (DT)
- "lazy" is tagged as adjective (JJ)
- "dog" is tagged as noun (NN)

Workflow of POS Tagging in NLP

The following are the processes in a typical natural language processing (NLP) example of part-of-speech (POS) tagging:

- **Tokenization:** Divide the input text into discrete tokens, which are usually units of words or sub words. The first stage in NLP tasks is tokenization.
- **Loading Language Models:** To utilize a library such as NLTK or SpaCy, be sure to load the relevant language model. These models offer a foundation for comprehending a language's grammatical structure since they have been trained on a vast amount of linguistic data.
- **Text Processing:** If required, preprocess the text to handle special characters, convert it to lowercase, or eliminate superfluous information. Correct PoS labeling is aided by clear text.
- **Linguistic Analysis:** To determine the text's grammatical structure, use linguistic analysis. This entails understanding each word's purpose inside the sentence, including whether it is an adjective, verb, noun, or other.
- **Part-of-Speech Tagging:** To determine the text's grammatical structure, use linguistic analysis. This entails understanding each word's purpose inside the sentence, including whether it is an adjective, verb, noun, or other.

- **Results Analysis:** Verify the accuracy and consistency of the PoS tagging findings with the source text. Determine and correct any possible problems or mistagging.

Types of POS Tagging in NLP

Assigning grammatical categories to words in a text is known as Part-of-Speech (PoS) tagging, and it is an essential aspect of Natural Language Processing (NLP). Different PoS tagging approaches exist, each with a unique methodology. Here are a few typical kinds:

1. Rule-Based Tagging

- Rule-based part-of-speech (POS) tagging involves assigning words their respective parts of speech using predetermined rules.
- In a rule-based system, POS tags are assigned based on specific word characteristics and contextual cues.
- For instance, a rule-based POS tagger could designate the "noun" tag to words ending in "-tion" or "-ment," recognizing common noun-forming suffixes. This approach offers transparency and interpretability, as it doesn't rely on training data.

Example:

Rule: Assign the POS tag "noun" to words ending in "-tion" or "-ment."

Text: "The presentation highlighted the key achievements of the project's development."

Rule based Tags:

- "The" - Determiner (DET)
- "presentation" - Noun (N)
- "highlighted" - Verb (V)
- "the" - Determiner (DET)
- "key" - Adjective (ADJ)
- "achievements" - Noun (N)
- "of" - Preposition (PREP)
- "the" - Determiner (DET)
- "project's" - Noun (N)
- "development" - Noun (N)

In this instance, "Noun" tags are applied to words like "presentation," "achievements," and "development" because of the aforementioned restriction.

2. Transformation Based tagging

- Transformation-based tagging (TBT) is a part-of-speech (POS) tagging method that uses a set of rules to change the tags that are applied to words inside a text.
- To change word tags in TBT, a set of rules is created depending on contextual information.
- When compared to rule-based tagging, TBT can provide higher accuracy, especially when dealing with complex grammatical structures.
- To attain ideal performance, it might require a large rule set and additional computer power.

Example:
Consider the transformation rule: Change the tag of a verb to a noun if it follows a determiner like "the."
Text: "The cat chased the mouse".
Initial Tags:

- "The" - Determiner (DET)
- "cat" - Noun (N)
- "chased" - Verb (V)
- "the" - Determiner (DET)
- "mouse" - Noun (N)

Transformation rule applied:
Change the tag of "chased" from Verb (V) to Noun (N) because it follows the determiner "the."
Updated tags:

- "The" - Determiner (DET)
- "cat" - Noun (N)
- "chased" - Noun (N)
- "the" - Determiner (DET)
- "mouse" - Noun (N)

3. Statistical POS Tagging

- Utilizing probabilistic models, statistical part-of-speech (POS) tagging is a computer linguistics technique that places grammatical categories on words inside a text.
- In order to capture the statistical linkages present in language, the algorithms learn the probability distribution of word-tag sequences.
- CRFs (conditional random fields) and Hidden Markov Models (HMMs) are popular models for statistical point-of-sale classification.
- The algorithm estimates the chance of observing a specific tag given the current word and its context by learning from labeled samples during training.
- The most likely tags for text that hasn't been seen are then predicted using the trained model.
- Statistical POS tagging works especially well for languages with complicated grammatical structures because it is exceptionally good at handling linguistic ambiguity and catching subtle language trends.

Hidden Markov Model POS tagging:

- Hidden Markov Models (HMMs) serve as a statistical framework for part-of-speech (POS) tagging in natural language processing (NLP). In HMM-based POS tagging, the model undergoes training on a sizable annotated text corpus to discern patterns in various parts of speech. Leveraging this training, the model predicts the POS tag for a given word based on the probabilities associated with different tags within its context.
- Comprising states for potential POS tags and transitions between them, the HMM-based POS tagger learns transition probabilities and word-emission probabilities during training.
- To tag new text, the model, employing the Viterbi algorithm, calculates the most probable sequence of POS tags based on the learned

probabilities.
Widely applied in NLP, HMMs excel at modeling intricate sequential data, yet their performance may hinge on the quality and quantity of annotated training data.

Advantages of POS Tagging

There are several advantages of Parts-Of-Speech (POS) Tagging including:

- Text Simplification
- Information Retrieval:
- Named Entity Recognition:
- Syntactic Parsing:

- **Text Simplification:** Breaking complex sentences down into their constituent parts makes the material easier to understand and easier to simplify.
- **Information Retrieval:** Information retrieval systems are enhanced by point-of-sale (POS) tagging, which allows for more precise indexing and search based on grammatical categories.
- **Named Entity Recognition:** POS tagging helps to identify entities such as names, locations, and organizations inside text and is a precondition for named entity identification.
- **Syntactic Parsing:** It facilitates syntactic parsing, which helps with phrase structure analysis and word link identification.

Disadvantages of POS Tagging

Some common disadvantages in part-of-speech (POS) tagging include:

- Ambiguity:
- Idiomatic Expressions:
- Out-of-Vocabulary Words:
- Domain Dependence:

- **Ambiguity:** The inherent ambiguity of language makes POS tagging difficult since words can signify different things depending on the context, which can result in misunderstandings.
- **Idiomatic Expressions:** Slang, colloquialisms, and idiomatic phrases can be problematic for POS tagging systems since they don't always follow formal grammar standards.
- **Out-of-Vocabulary Words:** Out-of-vocabulary words (words not included in the training corpus) can be difficult to handle since the model might have trouble assigning the correct POS tags.
- **Domain Dependence:** For best results, POS tagging models trained on a single domain should have a lot of domain-specific training data because they might not generalize well to other domains.

3.8 Named Entity Recognition

- **Named Entity Recognition (NER)** in NLP focuses on identifying and categorizing important information known as **entities** in text. These entities can be names of people, places, organizations, dates, etc.
- It helps in transforming unstructured text into structured information which helps in tasks like text summarization, knowledge graph creation and question answering.
- NER helps in detecting specific information and sort it into predefined categories. It plays an important role in enhancing other NLP tasks like part-of-speech tagging and parsing.
- Examples of Common Entity Types:

- **Person Names**: Albert Einstein
- **Organizations**: GeeksforGeeks
- **Locations**: Paris
- **Dates and Times**: 5th May 2025
- **Quantities and Percentages**: 50%, $100

It helps in handling ambiguity by analyzing surrounding words, structure of sentence and the overall context to make the correct classification. It means context can change based on entity's meaning.

Example 1:

- Amazon is expanding rapidly (Organization)
- The Amazon is the largest rainforest (Location)

Example 2:

- Jordan won the MVP award (Person)
- Jordan is a country in the Middle East (Location)

Working of Named Entity Recognition (NER) (OR) steps involves in NER

Various steps involves in NER and are as follows:

1. Analyzing the Text
2. Finding Sentence Boundaries
3. Tokenizing and Part-of-Speech Tagging
4. Entity Detection and Classification
5. Model Training and Refinement
6. Adapting to New Contexts

1. **Analyzing the Text**: It processes entire text to locate words or phrases that could represent entities.
2. **Finding Sentence Boundaries**: It identifies starting and ending of sentences using punctuation and capitalization which helps in maintaining meaning and context of entities.
3. **Tokenizing and Part-of-Speech Tagging**: Text is broken into tokens (words) and each token is tagged with its grammatical role which provides important clues for identifying entities.

4. **Entity Detection and Classification**: Tokens or groups of tokens that match patterns of known entities are recognized and classified into predefined categories like Person, Organization, Location etc.
5. **Model Training and Refinemen**t: Machine learning models are trained using labeled datasets and they improve over time by learning patterns and relationships between words.
6. **Adapting to New Contexts**: A well-trained model can generalize to different languages, styles and unseen types of entities by learning from context.

Methods of Named Entity Recognition

There are different methods present in NER which are:

1. Lexicon Based Method
2. Rule Based Method
3. Machine Learning-Based Method
4. Deep Learning Based Method

1. Lexicon Based Method

- This method uses a dictionary of known entity names. This process involves checking if any of these words are present in a given text.
- However, this approach isn't commonly used because it requires constant updating and careful maintenance of the dictionary to stay accurate and effective.

2. Rule Based Method

- It uses a set of predefined rules which helps in extraction of information. These rules are based on patterns and context.
- Pattern-based rules focus on the structure and form of words helps in looking at their morphological patterns.
- On the other hand context-based rules focus on the surrounding words or the context in which a word appears within the text document.
- This combination of pattern-based and context-based rules increases the accuracy of information extraction in NER.

3. Machine Learning-Based Method

There are two main types of category in this:

- **Multi-Class Classification**: Trains model on labeled examples where each entity is categorized. In addition to labelling model also requires a deep understanding of context which makes it a challenging task for a simple machine learning algorithm.
- **Conditional Random Field (CRF)**: It is implemented by both NLP Speech Tagger and NLTK. It is a probabilistic model that understands the sequence and context of words which helps in making entity prediction more accurate.

4. Deep Learning Based Method

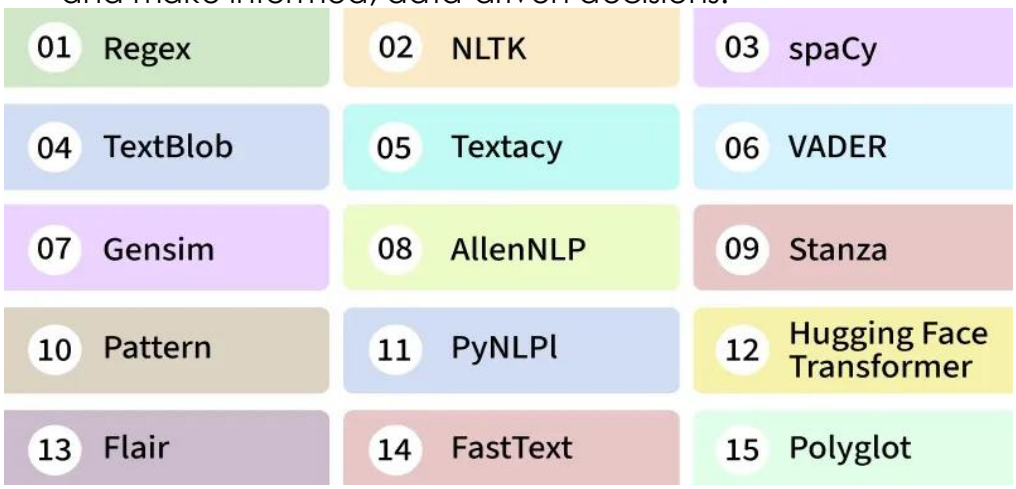
- **Word Embeddings**: Captures the meaning of words in context.
- **Automatic Learning**: Deep models learn complex patterns without manual feature engineering.
- **Higher Accuracy**: Performs well on large varied datasets.

Implementation of NER in Python

Step 1: Installing Libraries
Step 2: Importing and Loading data
Step 3: Applying NER to a Sample Text
Step 4: Visualizing Entities
Step 5: Creating a DataFrame for Entities

4 NLP Libraries in Python

- NLP (Natural Language Processing) helps in the extraction of valuable insights from large amounts of text data.
- Python has a wide range of libraries specifically designed for text analysis helps in making it easier for data scientists and analysts to process, analyze and derive meaningful insights from text.
- These libraries handle various NLP tasks such as text preprocessing, tokenization, sentiment analysis, named entity recognition and topic modeling.
- By using these libraries we can automate text analysis, uncover patterns and make informed, data-driven decisions.



NLTK (Natural Language Toolkit)
NLTK provides various tools for text analysis. It is used for educational and research purposes which offers features for tokenization, stemming and part-of-speech tagging.

- **Tokenization:** Break down text into smaller, meaningful units like words or sentences.
- **Stemming and Lemmatization:** Simplify words to their root form for more consistent analysis.

Real-life applications

1. **Customer Feedback Analysis:** Split reviews into words or sentences for sentiment analysis.
2. **Text Classification:** Automatically categorize content like news articles or social media posts.

Key Features of NLTK:

1. Text Tokenization
2. Stop Words Removal
3. Stemming and Lemmatization
4. Part-of-Speech Tagging (POS)
5. Named Entity Recognition (NER)

1. **Text Tokenization**

Tokenization is the process of breaking down text into smaller units such as words or sentences. NLTK provides several built-in tokenizers to split text into manageable chunks:

```
import nltk
from nltk.tokenize import word_tokenize

sentence = "NLTK is a powerful toolkit for natural language processing."
tokens = word_tokenize(sentence)
print(tokens)
```

2. **Stop Words Removal**

Stop words are common words such as "is", "the", and "and" that don't carry much information and can be filtered out during analysis. NLTK has a pre-built list of stop words that can be removed:

```
from nltk.corpus import stopwords

stop_words = set(stopwords.words('english'))
filtered_sentence = [word for word in tokens if not word in stop_words]
print(filtered_sentence)
```

3. **Stemming and Lemmatization**

Stemming and lemmatization reduce words to their root forms. Stemming uses a crude heuristic, while lemmatization ensures the root word is valid in the language. NLTK provides functions for both:

```
from nltk.stem import PorterStemmer

stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(word) for word in filtered_sentence]
print(stemmed_words)
```

4. **Part-of-Speech Tagging (POS)**

Part-of-speech tagging identifies the grammatical role of each word in a sentence, such as noun, verb, adjective, etc.:

```
tagged_words = nltk.pos_tag(tokens)
print(tagged_words)
```

5. **Named Entity Recognition (NER)**

NER classifies named entities in text like people, organizations, and locations. This is particularly useful for tasks like information extraction:

```
from nltk import ne_chunk
ner_tree = ne_chunk(tagged_words)
print(ner_tree)
```

Pros and Cons of using NLTK for NLP:

- **Pros:**
  - Most well-known NLP library
  - Third-party extensions
- **Cons:**
  - Learning curve
  - Slow at times
  - No neural network models
  - Only splits text by sentences

spaCy

- spaCy does not provide a software as a service, or a web application. It's an open-source library designed to help you build NLP applications, not a consumable service.
- In the documentation, you'll come across mentions of spaCy's features and capabilities. Some of them refer to linguistic concepts, while others are related to more general machine learning functionality.

NAME	DESCRIPTION
Tokenization	Segmenting text into words, punctuations marks etc.
Part-of-speech (POS) **Tagging**	Assigning word types to tokens, like verb or noun.
Dependency Parsing	Assigning syntactic dependency labels, describing the relations between individual tokens, like subject or object.
Lemmatization	Assigning the base forms of words. For example, the lemma of "was" is "be", and the lemma of "rats" is "rat".
Sentence Boundary Detection (SBD)	Finding and segmenting individual sentences.
Named Entity Recognition (NER)	Labelling named "real-world" objects, like persons, companies or locations.
Entity Linking (EL)	Disambiguating textual entities to unique identifiers in a knowledge base.
Similarity	Comparing words, text spans and documents and how similar they are to each other.

NAME	DESCRIPTION
Text Classification	Assigning categories or labels to a whole document, or parts of a document.
Rule-based Matching	Finding sequences of tokens based on their texts and linguistic annotations, similar to regular expressions.
Training	Updating and improving a statistical model's predictions.
Serialization	Saving objects to files or byte strings.

spaCy is designed for high-performance text processing. It is good at tasks such as named entity recognition (NER) and dependency parsing which helps in making it ideal for real-time applications.

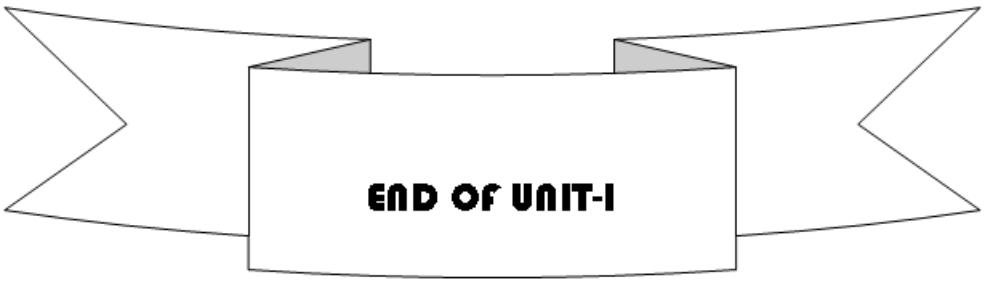
- **Named Entity Recognition (NER):** Identify and classify entities like names, locations or organizations in text.
- **Dependency Parsing:** Understand the grammatical relationships between words in a sentence.

Real-life applications

1. **Legal Document Analysis:** Identify and extract key entities like company names or legal terms from contracts.
2. **Customer Service Automation:** Extract relevant details like product names or addresses from customer queries for faster responses.

Pros and Cons of using spaCy for NLP:

- **Pros:**
  - Fast
  - Easy to use
  - Great for beginner developers
  - Relies on neural networks for training models
- **Cons:**
  - Not as flexible as other libraries like NLTK



UNIT II
Syllabus

Text Representation and Statistical NLP

Bag of Words and TF-IDF, Language Modeling: Unigrams, Bigrams, N-gram Models, Word Embeddings: Word2Vec, GloVe, FastText, Cosine Similarity and Distance Measures, Text Classification using Naive Bayes and SVM, Evaluation Metrics: Accuracy, Precision, Recall, F1.

STUDY MATERIAL

Prepared by **K.ANJANEYULU, M. Tech (CSE), MBA, PGDCA, M. Sc, B. Ed.,**
Assistant Professor,
Department of Computer Science & Engineering (AI&ML),
Sri Venkateswara College of Engineering & Technology,
RVS Nagar. Chittoor.
Phno: **(+91) – 9640985702, (+91) – 9398526101**
E-mail: **anji.karapakula@gmail.com**

UNIT II: Text Representation and Statistical NLP

1. Bag of words and Inverse Document Frequency (TF-IDF)

Bag of words

- In bag of words, we take all unique words from the corpus, note the frequency of occurrence and sort them in descending order.
- All the word — vector mapping will be used to represent each sentence.
- Let us take it in steps and examples to have a clear picture.

Paragraph: **"The news mentioned here is fake. Audience do not encourage fake news. Fake news is false or misleading".**

Step 1: Tokenize the data, remove stop words and perform stemming or lemmatization.

```
import nltk
from nltk.stem import WordNetLemmatizer
from nltk.corpus import stopwords
import re

paragraph = """The news mentioned here is fake. Audience do not
encourage fake news. Fake news is false or misleading"""


sentences = nltk.sent_tokenize(paragraph)
lemmatizer = WordNetLemmatizer()

corpus = []

for i in range(len(sentences)):
    sent = re.sub('[^a-zA-Z]', ' ', sentences[i])
    sent = sent.lower()
    sent= sent.split()
    sent = [lemmatizer.lemmatize(word) for word in sent if not word in
set(stopwords.words('english'))]
    sent = ' '.join(sent)
    corpus.append(sent)print(corpus)
```

Output:

```
['news mentioned fake', 'audience encourage fake news', 'fake news false
misleading']
```

Step 2: List all unique words

Unique words: ['news', 'mentioned', 'fake', 'audience', 'encourage', 'false', 'misleading']

Step 3: Create a dictionary with mapping of words to a number. This should now be sorted on frequency of occurrence in descending order.

Vector (Number)	Words	Frequency
0	fake	3
1	news	3
2	audience	1
3	encourage	1
4	false	1
5	mentioned	1
6	misleading	1

Step 4: Now, create a table in each sentence, for the presence of each word in the dictionary, assign '1' else assign '0'

Sentence	0 (fake)	1 (news)	2 (audience)	3 (encourage)	4 (false)	5 (mentioned)	6 (misleading)
news mentioned fake	1	1	0	0	0	1	0
audience encourage fake news	1	1	1	1	0	0	0
fake news false misleading	1	1	0	0	1	0	1

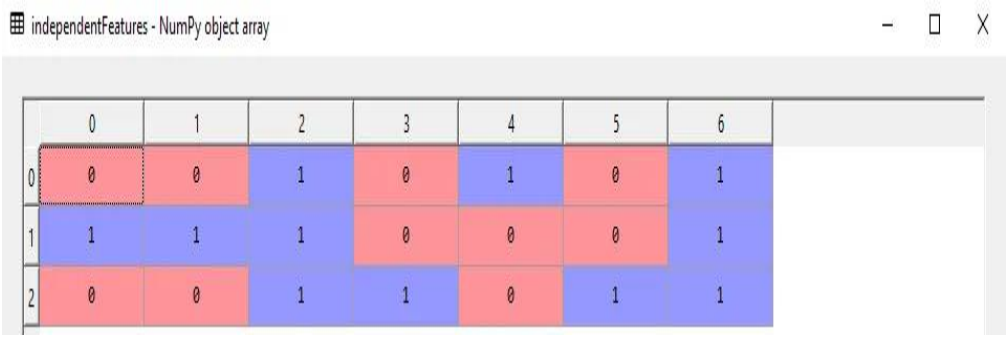
The data we had is now transformed as:

- news mentioned fake — [1 1 0 0 0 1 0]
- audience encourage fake news — [1 1 1 1 0 0 0]
- fake news false misleading — [1 1 0 0 1 0 1]

And these vectors are sent as input to the model as independent features. We can use 'CountVectorizer' from sci-kit learn to perform steps 2, 3 and 4

```
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer()
independentFeatures = cv.fit_transform(corpus).toarray()
```

Output:



Bag of words will really be helpful in prediction problems like language modeling and documentation classification. Bag of words do have few shortcomings. Mentioning a few of them below:

1. **Vocabulary**: The vocabulary requires careful design to manage the size, which in-turn impacts the sparsity of the document representations.
2. **Meaning:** The values here are represented either as 1's or 0's. Consider the words 'news' and 'fake' — both the words are having same representation, and semantics of the words are same. This causes the difficulty to identify the importance of words.

To overcome these shortcomings, TF-IDF can be used.

Term Frequency — Inverse Document Frequency (TF-IDF)

We calculate the term frequency and Inverse document frequency for every word in the corpus, and multiplication of TF and IDF gives the document vectors. So, how do we calculate TF-IDF

Term Frequency (TF) — (No. of repeated words in sentence) / (No. of words in sentence)

Inverse Document Frequency (IDF) — log[(No. of sentences) / (No. of sentences containing word)]

Example: — "The news mentioned here is fake. Audience do not encourage fake news. Fake news is false or misleading"

Step 1: Passing the data through stemming or lemmatization. Take all the unique words, and sort based on frequency of occurrence. These are steps 1,2,3 we have observed in Bag of words (BOW)

# Vector (Number)	Aa Words	# Frequency
0	fake	3
1	news	3
2	audience	1
3	encourage	1
4	false	1
5	mentioned	1
6	misleading	1

Step 2: Calculate Term Frequency

Let us calculate TF for sentence — 1

- news — 1 / 3 = 0.33 {News is repeated once in the sentence, and total words are 3 — giving 1/3}
- mentioned — 1 / 3 = 0.33
- fake — 1 / 3 = 0.33

- audience , encourage, false, mentioned, misleading — 0 / 3 = 0 {These words did not occur in the sentence — there is no repetition, hence zero}
    - Let us calculate TF for sentence — 2
- audience — 1 / 4 = 0.25 (Audience word is repeated once in the sentence, and total words in the sentence are 4 — giving 1/4)
- encourage — 1 / 4 = 0.25
- fake — 1 / 4 = 0.25
- news — 1 / 4 = 0.25
- false, mentioned, misleading — 0 / 4 = 0
  Similarly — we calculate Term Frequency for rest of sentences.

Sentence	0 (fake)	1 (news)	2 (audie...	3 (enco...	4 (false)	5 (menti...	6 (misle...
news mentioned fake	0.33	0.33	0	0	0	0.33	0
audience encourage fake news	0.25	0.25	0.25	0.25	0	0	0

Step 3: Calculate IDF

Let us calculate IDF for all the words:

- news — log_e(3/3) = 0 {we have 3 sentences, and news word is present in all three sentences, hence log(3/3)}
- mentioned — log_e(3/1) = 1.0986
- fake — log_e(3/3) = 0
- audience — log_e(3/1) = 1.0986
- encourage — log_e(3/1) = 1.0986
- false — log_e(3/1) = 1.0986
- misleading — log_e(3/1) = 1.0986

Step 4: Calculate document vectors multiplying TF and IDF values. Steps 2, 3, 4 can be achieved through TF-IDF vectorizer from sci-kit learn.

```
# Creating the TF-IDF model
from sklearn.feature_extraction.text import TfidfVectorizer
tfidf = TfidfVectorizer()
independentFeatures_tfIDF = tfidf.fit_transform(corpus).toarray()
```

shortcomings even here, such as

- TF-IDF does not capture position in text, semantics, co-occurrences
- TF-IDF computes document similarity directly in the word-count space, making it slow for large documents

Bag of words or TF-IDF features can be used as inputs for Naive bayes model to classify spam and ham.

Bag-of-words vs TF-IDF

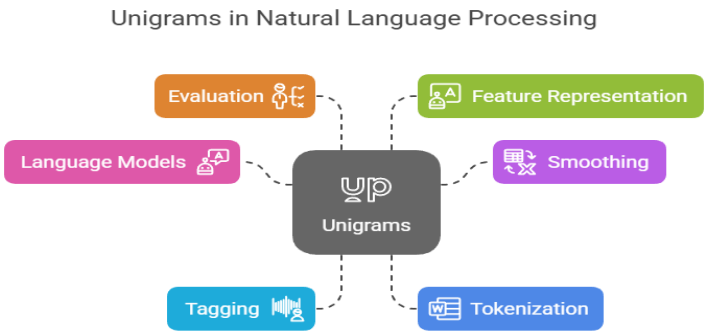
Aspect	Bag-of-Words (BoW)	TF-IDF
Word Frequency vs. Importance	Treats all words equally, counts occurrences without considering their rarity.	Adjusts word importance by considering both term frequency and rarity across the corpus.
Handling Common Words	Common words (stop words) can dilute the model's focus on meaningful terms.	Reduces the weight of common words, focusing on distinctive terms.
Sensitivity to Document Length	Document length affects word frequency, potentially skewing results.	Normalizes the effect of document length, less sensitive to total word count.
Complexity	Simple to implement, computationally inexpensive, but may result in high-dimensional, sparse vectors.	More complex and computationally expensive, but provides a more informative representation.

2. Language Modeling: Unigrams

2.1 Unigrams in NLP

unigram, often referred to as a 1-gram, is a string of one word. "I", "am", "learning", and "Natural Language Processing" are the unigram that are contained in a statement such as "I am learning NLP." The most basic type of n-grams are unigram, which are consecutive word sequences of a specific length.

The sources describe and use unigrams as follows:



Language Models: Choosing words based on their frequency which is represented as words covering a probability space proportionate to their occurrence is known as sampling from a unigrams language model.
Let's explore how to predict the next word in a sentence. We need to calculate p(w | h), where w is the candidate for the next word.
Consider the sentence **'This example is on...'**.
If we want to calculate the probability of the next word being "NLP", the probability can be expressed as:

p("NLP"|"This","example","is","on")

- To generalize, the conditional probability of the fifth word given the first four can be written as:
  p(w5|w1,w2,w3,w4) or p(W)=p(wn|w1,w2,…,wn−1)
  This is calculated using the chain rule of probability:
  P(A|B)=P(A∩B)/P(B) and P(A∩B)=P(A|B)/P(B)
  Now generalize this to sequence probability:
  P(X1,X2,…,Xn)=P(X1)P(X2|X1)P(X3|X1,X2)…P(Xn|X1,X2,…,Xn−1)
  This yields:
  P(w1,w2,w3,…,wn)=∏iP(wi|w1,w2,…,wi−1)

- By applying Markov assumptions, which propose that the future state depends only on the current state and not on the sequence of events that preceded it, we simplify the formula:
  P(wi|w1,w2,…,wi−1)≈P(wi|wi−k,…,wi−1)

- For a unigram model (k=0), this simplifies further to:
  P(w1,w2,…,wn)≈∏iP(wi)

Smoothing:

- Smoothing methods for higher-order n-gram models (such as bigrams or trigrams) that cope with sparse data (events not encountered in the training data) include unigram probabilities.

- Backoff models, for instance, can "back off" to a lower-order model, such the unigram probability, if a higher-order n-gram probability cannot be accurately calculated.
- Unigrams probabilities can be subjected to Laplace smoothing. More complex smoothing techniques, such as Kneser-Ney discounting, employ unigrams probabilities in subtle ways, occasionally emphasizing a word's "versatility" the number of situations in which it appears rather than merely its frequency.

Tagging:

- A straightforward statistical method for Part-of-Speech (POS) labelling is Unigram tagging.
- Without taking into account the surrounding words, a unigram tagger determines a token's most probable tag based solely on the token itself (its a priori most likely tag).
- Words that were not included in the training data can also be given potential tags with the use of unigrams probabilities.

Tokenization:

- One technique for dividing text into subword units or tokens is called unigrams tokenization.
- Unigram tokenization, in contrast to unit-merging approaches, begins with a big initial vocabulary and iteratively eliminates tokens according to how likely they are to attain a specified size in typical tokenizations.
- Compared to previous techniques like Byte Pair Encoding (BPE), this method is proposed to generate tokens that are more semantically relevant. Systems like T5 and ALBERT use it.

Evaluation:

- A statistic for assessing tasks such as machine translation is unigram precision.
- Unigram precision calculates the proportion of a candidate translation's unigrams (single words) that are also found in a reference translation.
- By calculating their geometric mean, metrics such as BLEU combine unigram precision with bigram, trigram, and 4-gram precision.

Feature Representation:

- NLP models can employ unigrams, or single words, as features. Counting word occurrences basically, unigram counts is the foundation of the "bag-of-words" representation notion.
- In skipgram word embeddings, a unigram language model built from empirical word probabilities is sampled to produce negative examples used in training.
- A unigram is a basic unit in natural language processing (NLP) that may be utilized as a basic model in more sophisticated approaches for a variety of tasks, including language modelling, tagging, tokenization, and evaluation.

2.2 Bigram Model in NLP

- Two words in a row are called a bigram. When 'n' is the number of consecutive words in the sequence, this is the most typical example of an n-gram.
- Bigrams capture word pairs and offer some information about word order and local context, whereas unigrams represent individual words (n=1).

- Bigram in NLP is produced from neighbouring phrase words. The bigrams for "I am learning NLP" are "I am," "am learning," and "learning Natural Language Processing." The bigrams() method may construct a list of consecutive word pairs from the input list ['more', 'is','said', 'than', 'done']. TextBlob's ngrams() function extracts bigrams when n=2.
- A key element of n-gram language models are bigrams. A bigram language model uses just the identity of the word that comes right before it to estimate the likelihood of a word.

Bigrams model:

Let's explore how to predict the next word in a sentence. We need to calculate p(w | h), where w is the candidate for the next word. Consider the sentence 'This example is on...'.If we want to calculate the probability of the next word being "NLP", the probability can be expressed as:

p("NLP"|"This","example","is","on")

- To generalize, the conditional probability of the fifth word given the first four can be written as:

  p(w5|w1,w2,w3,w4) or p(W)=p(wn|w1,w2,…,wn−1)

  This is calculated using the chain rule of probability:

  P(A|B)=P(A∩B)/P(B) and P(A∩B)=P(A|B)/P(B)

  Now generalize this to sequence probability:

  P(X1,X2,…,Xn)=P(X1)P(X2|X1)P(X3|X1,X2)…P(Xn|X1,X2,…,Xn−1)

  This yields:

  P(w1,w2,w3,…,wn)=∏iP(wi|w1,w2,…,wi−1)

- By applying Markov assumptions, which propose that the future state depends only on the current state and not on the sequence of events that preceded it, we simplify the formula:

  $P(w_i|w_1,w_2,\ldots,w_{i-1}) \approx P(w_i|w_{i-k},\ldots,w_{i-1})$

- for a bigram model (k=1):
  $P(w_i|w_1,w_2,\ldots,w_{i-1}) \approx P(w_i|w_{i-1})$

  - Smoothing techniques are employed to solve the sparsity and zero count issues. A certain amount of probability mass is redistributed from seen to unobserved n-grams by smoothing.
  - Laplace smoothing, also known as add-one smoothing, is a straightforward technique that modifies the denominator by adding the total vocabulary size, V, to the unigram count and adding one to each Bigram in NLP count.
  - Discounting techniques redistribute probability mass, usually among unseen n-grams, by taking it from observed n-grams. The count of observed n-grams is subtracted from a preset value using absolute discounting.
  - More complex techniques, such as Kneser-Ney smoothing and Good-Turing estimation, estimate probability for unseen bigrams using lower-order n-grams or take into account the "versatility" of words.
  - Using weighted sums, interpolated n-gram language models aggregate probability from many n-gram models.
  - More reliable probability estimations may result from this. Backoff models are an additional method in which the model "backs off" to a lower-order model, such a bigram or unigram model, if a higher-order n-gram probability is zero or unreliable.
  - Bigram in NLP models are essential for comprehending basic language modelling principles.
  - However, because to their sparsity and memory requirements, really high-order n-grams are not feasible since the number of parameters increases exponentially with 'n'.

Bigrams are employed in additional NLP tasks outside language modelling:

Collocations
Part-of-Speech Tagging
Feature Representation
Tokenization
Evaluation

Collocations:

- Collocations are word pairs that occur together exceptionally frequently and are frequently resistant to replacement, whereas bigrams simply represent any two-word sequence.
- Collocations can be found by starting with frequent bigrams, however methods to normalise for word frequency are required.

Part-of-Speech Tagging:

- In addition to the word itself, Bigram in NLP taggers also issue tags based on the likelihood that a tag will be awarded the tag that comes just before it.
- For tag transitions, this is frequently presented as a Hidden Markov Model (HMM) with a bigram assumption.
- Errors can be caused by minor problems, such as unseen words or tag sequences.

Feature Representation:

- Bigrams can be used into NLP models as features. A "bag-of-bigrams" representation, which can be more effective than a bag-of-words (unigram) representation, counts or shows the presence of bigrams in a text without taking into account their order.
- Because letter-bigrams (character pairings) are more resilient to words that are not in the lexicon than word-bigrams, they may also be utilized as features, especially for language classification.

Tokenization: Character bigrams, or pairs of characters, are repeatedly combined to create a vocabulary of subword units in Byte Pair Encoding (BPE), a subword tokenization technique.

Evaluation:

- The overlap of bigrams between a candidate translation and a reference translation is measured by a metric called **"bigram precision,"** which is used to assess the quality of machine translation.
- It should be noted that sources that discuss evaluation appear to be more interested in bigrams than unigram precision, even though the BLEU metric incorporates both.
- To be more precise: BLEU combined unigram, bigram, trigram, and 4-gram accuracy, according to my earlier comment on unigrams.

2.3 N-grams model

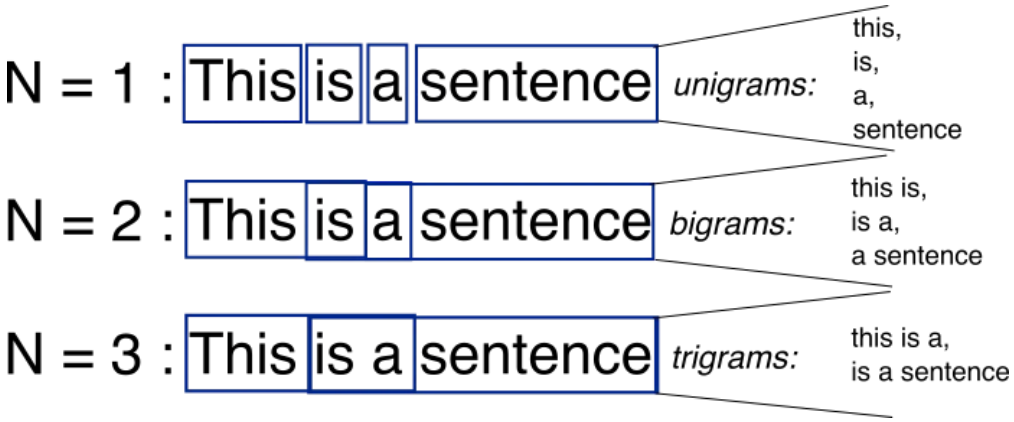
Definition:

N-grams are contiguous sequences of 'n' items, typically words in the context of NLP. These items can be characters, words, or even syllables, depending on the granularity desired. The value of 'n' determines the order of the N-gram.

Examples:

- **Unigrams (1-grams):** Single words, e.g., "cat," "dog."

- **Bigrams (2-grams):** Pairs of consecutive words, e.g., "natural language," "deep learning."
- **Trigrams (3-grams):** Triplets of consecutive words, e.g., "machine learning model," "data science approach."
- 4-grams, 5-grams, etc.: Sequences of four, five, or more consecutive words.



N-gram Language Model

- An N-gram language model predicts the probability of a given N-gram within any sequence of words in a language. A well-crafted N-gram model can effectively predict the next word in a sentence.
- We need to calculate p(w|h), where w is the candidate for the next word.
- Consider the sentence 'This example is on...'.If we want to calculate the probability of the next word being "NLP", the probability can be expressed as:
  p("NLP"|"This","example","is","on")
  - To generalize, the conditional probability of the fifth word given the first four can be written as:
    p(w5|w1,w2,w3,w4) or p(W)=p(wn|w1,w2,…,wn−1)
  - This is calculated using the chain rule of probability:
    P(A|B)=P(A∩B)/P(B) and P(A∩B)=P(A|B)/P(B)
  - Now generalize this to sequence probability:
    P(X1,X2,…,Xn)=P(X1)P(X2|X1)P(X3|X1,X2)…P(Xn|X1,X2,…,Xn−1)
    This yields:
    P(w1,w2,w3,…,wn)=∏iP(wi|w1,w2,…,wi−1)

- By applying Markov assumptions, which propose that the future state depends only on the current state and not on the sequence of events that preceded it, we simplify the formula:
  P(wi|w1,w2,…,wi−1)≈P(wi|wi−k,…,wi−1)

- For a unigram model (k=0), this simplifies further to:
  P(w1,w2,…,wn)≈∏iP(wi)

And for a bigram model (k=1):
$P(w_i|w_1,w_2,\ldots,w_{i-1}) \approx P(w_i|w_{i-1})$

Metrics for Language Modelling

- **Entropy**: Entropy, as a measure of the amount of information conveyed by Claude Shannon. Below is the formula for representing entropy

$H(p)=\sum x p(x)\cdot(-\log(p(x)))$

H(p) is always greater than equal to 0.

- **Cross-Entropy**: It measures the ability of the trained model to represent test data.

$H(p)=\sum i=1 x 1 n(-\log 2(p(w_i|w_{1i-1})))$

The cross-entropy is always greater than or equal to Entropy i.e the model uncertainty can be no less than the true uncertainty.

- **Perplexity**: Perplexity is a measure of how good a probability distribution predicts a sample. It can be understood as a measure of uncertainty. The perplexity can be calculated by cross-entropy to the exponent of 2.

Perplexity = $2^{\text{Cross-Entropy}}$

- Following is the formula for the calculation of Probability of the test set assigned by the language model, normalized by the number of words:

$$PP(W) = \sqrt[n]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_{i-1})}}$$

Example:

- Let's take an example of the sentence: ***'Natural Language Processing'.*** For predicting the first word, let's say the word has the following probabilities:

word	**P(word | <start>)**
The	0.4
Processing	0.3
Natural	0.12
Language	0.18

- Now, we know the probability of getting the first word as natural. But, what's the probability of getting the next word after getting the word '*Language*' after the word '*Natural*'.

word	**P(word | 'Natural')**
The	0.05
Processing	0.3
Natural	0.15
Language	0.5

- After getting the probability of generating words 'Natural Language', what's the probability of getting '*Processing*'.

word	**P(word | 'Language')**
The	0.1
Processing	0.7
Natural	0.1
Language	0.1

- Now, the perplexity can be calculated as:

$$PP(W) = \sqrt[n]{\prod_{i=1}^{N} \frac{1}{P(w_i|w_{i-1})}} = \sqrt[3]{\frac{1}{0.12*0.5*0.7}} \approx 2.876$$

- From that we can also calculate entropy:

Entropy=log_2(2.876)=1.524

3. Word Embeddings

3.1 Word2Vec

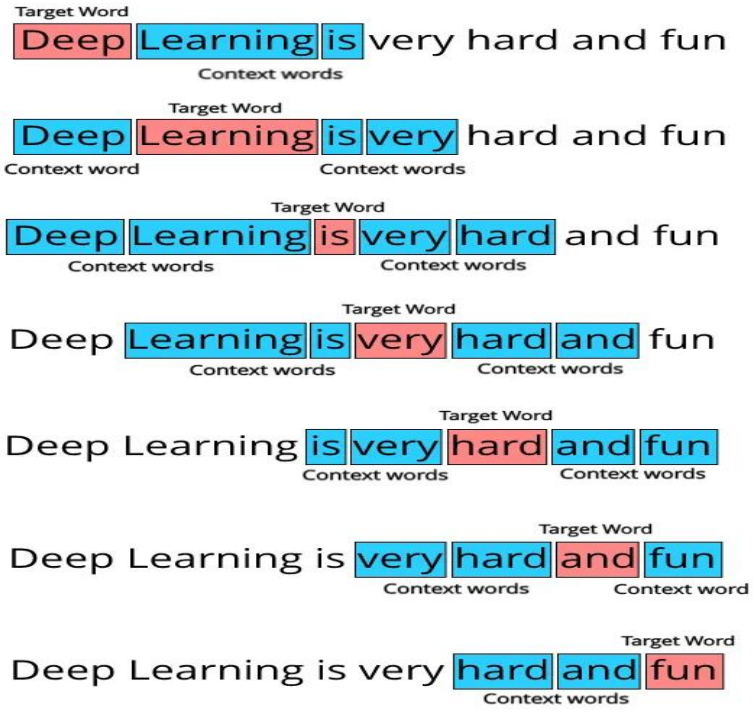
- Word2Vec is a widely used method in natural language processing (NLP) that allows words to be represented as vectors in a continuous vector space.
- Researchers at Google developed word2Vec that maps words to high-dimensional vectors to capture the semantic relationships between words.
- Words with similar meanings should have similar vector representations.
- Word vectors are much better ways to represent words than **one hot encoded vectors.**
- The index which is assigned to each word does not hold any semantic meaning.
- In one hot encoded vectors, the vectors for "dog" and "cat" are just as close to each other as "dog" and "computer", hence neural network has to try really hard to understand each word since they are being treated as completely isolated entities.
- The usage of word vectors aim to resolve both these issues.

Architecture of Word2Vec:

Word2Vec utilizes two architectures:

1. CBOW (Continuous Bag of Words):

- The CBOW model predicts the current word given context words within a specific window.
- The input layer contains the context words and the output layer contains the current word.
- The hidden layer contains the dimensions we want to represent the current word present at the output layer.
- Consider, "Deep Learning is very hard and fun".
- We need to set something known as ***window size***. Let's say 2 in this case. We have to iterate over all the words in the given data, which in this case is just one sentence, and **then consider a window of word which surrounds it**.
- Here since our window size is 2 we will consider 2 words behind the word and 2 words after the word, hence each word will get 4 words associated with it. We will do this for each and every word in the data and collect the word pairs.



1st Window pairs: (Deep, Learning), (Deep, is)
2nd Window pairs: (Learning, Deep), (Learning, is), (Learning, very)
3rd Window pairs: (is, Deep), (is, Learning), (is, very), (is, hard)

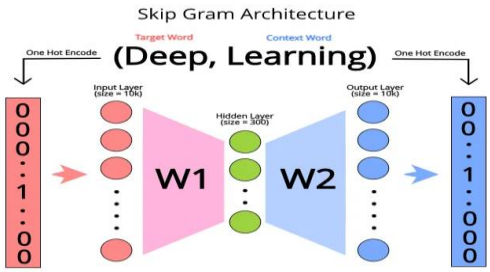
And so on. At the end our **target word vs context word data set** is going to look like this:

(Deep, Learning), (Deep, is), (Learning, Deep), (Learning, is), (Learning, very), (is, Deep), (is, Learning), (is, very), (is, hard), (very, learning), (very, is), (very, hard), (very, and), (hard, is), (hard, very), (hard, and), (hard, fun), (and, very), (and, hard), (and, fun), (fun, hard), (fun, and)

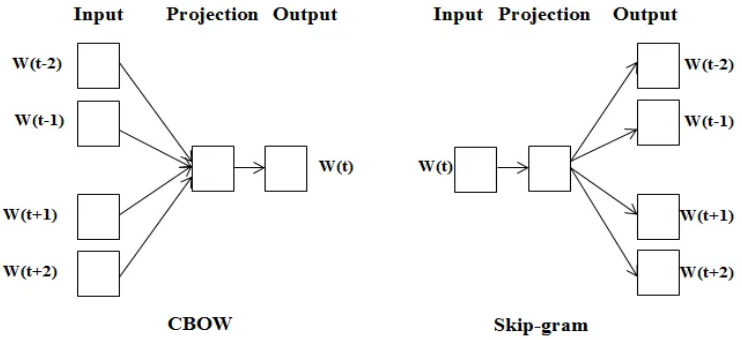
This can be considered as our **"training data"** for word2vec

2. Skip Gram :

- Skip gram predicts the surrounding context words within specific window given current word.
- The input layer contains the current word and the output layer contains the context words.
- The hidden layer contains the number of dimensions in which we want to represent current word present at the input layer.



- Since this neural network has a total of 3 layers, there will be only 2 weight matrices for the network, **W1** and **W2**.
- **W1** will have dimensions of 10000*300 and **W2** will have dimensions of 300*10000. These two weight matrices will play an important role in calculating word vectors.
- For the entire dataset which we have collected from the original textual data, we will pass each pair into the neural network and train it. **Neural network here is trying to guess which context words can appear given a target word.**
- After training the neural network, if we input any target word into the neural network, it will give a vector output which represents the words which have a high probability of appearing near the given word.
- For CBOW the only difference is that we try to predict the target word given the context words, essentially we just invert the skip gram model to get the CBOW model. It looks like this:



Word2Vec Code Implementation

Download the text file used for generating word vectors from here. Below is the implementation:

1. Importing Required Libraries

- We import Word2Vec from Gensim to build the model.
- nltk.tokenize helps split the text into sentences and words.
- Warnings are disabled for cleaner output.

```
from gensim.models import Word2Vec
import gensim
from nltk.tokenize import sent_tokenize, word_tokenize
nltk.download('punkt_tab')
import warnings
```

```
warnings.filterwarnings(action='ignore')
```

2. Loading and Cleaning the Dataset

- The Gutenburg.zip file is read from the local directory.
- Newline characters (\n) are replaced with spaces for consistent sentence structure.

```
import zipfile
with zipfile.ZipFile("/content/Gutenburg.zip", 'r') as zip_ref:
    file_name = zip_ref.namelist()[0]  # First file in the ZIP
    with zip_ref.open(file_name) as file:
        content = file.read().decode('utf-8', errors='ignore')
        cleaned_text = content.replace("\n", " ")
        print("File loaded")
```

Output:

File loaded

3. Text Tokenization

- The text is first split into sentences using sent_tokenize().
- Each sentence is then split into lowercase words using word_tokenize().
- In result, each sublist contains tokenized words from one sentence.

```
data = []

for i in sent_tokenize(cleaned_text):
    temp = []

    # tokenize the sentence into words
    for j in word_tokenize(i):
        temp.append(j.lower())

    data.append(temp)
```

4. Building Word2Vec Models

1. CBOW Model

- **min_count=1:** Includes all words (even those appearing once).
- **vector_size=100**: Generates 100-dimensional embeddings.
- **window=5:** Considers 5 words before and after the target word.
- **sg=0 (default):** Uses CBOW (Continuous Bag of Words) architecture.

```
# Create CBOW model
model1 = gensim.models.Word2Vec(data, min_count=1,
                                vector_size=100, window=5)
```

2. Skip-Gram Model

- **sg=1:** Enables Skip-Gram architecture, which predicts context words from a target word.

```
# Create Skip Gram model
model2 = gensim.models.Word2Vec(data, min_count=1, vector_size=100,
                                window=5, sg=1)
```

6. Evaluating Word Similarities

- Calculates cosine similarity between 'alice' and two other words using the CBOW model.
- Cosine similarity shows how semantically related two words are, ranging from -1 (opposite) to 1 (very similar).

```
print("Cosine similarity between 'alice' " +
      "and 'wonderland' - CBOW : ",
      model1.wv.similarity('alice', 'wonderland'))

print("Cosine similarity between 'alice' " +
      "and 'machines' - CBOW : ",
      model1.wv.similarity('alice', 'machines'))
```

Output :

```
Cosine similarity between 'alice' and 'wonderland' - CBOW :  0.98820096
Cosine similarity between 'alice' and 'machines' - CBOW :  0.9544018
```

Applications of Word Embedding:

- **Text classification:** Using word embeddings to increase the precision of tasks such as topic categorization and sentiment analysis.
- **Named Entity Recognition (NER):** Using word embeddings semantic context to improve the identification of entities such as names and locations.
- **Information Retrieval:** To provide more precise search results, embeddings are used to index and retrieve documents based on semantic similarity.
- **Machine Translation:** The process of comprehending and translating the semantic relationships between words in various languages by using word embeddings.
- **Question Answering**: Increasing response accuracy and understanding of semantic context in Q&A systems.

3.2 GloVe

- GloVe (Global Vectors for Word Representation) is an unsupervised learning algorithm designed to generate dense vector representations also known as embeddings.
- Its primary objective is to capture semantic relationships between words by analyzing their **co-occurrence patterns** in a large text corpus.
- GloVe approach is unique as it effectively combines the strengths of two major approaches:
  - Latent Semantic Analysis (LSA) which uses global statistical information.
  - Context-based models like Word2Vec which focuses on local word context.
- At the core of GloVe lies the idea of mapping each word into a continuous vector space where both the magnitude and direction of the vectors reflect meaningful semantic relationships.

- For instance, the relationship captured in a vector equation can be like:
  king - man + woman = queen.
- To achieve this, GloVe constructs a word co-occurrence matrix where each element reflects how often a pair of words appears together within a given context window.
- It then optimizes the word vectors such that the dot product between any two word vectors approximates the **pointwise mutual information (PMI)** of the corresponding word pair.
- This optimization allows GloVe to produce embeddings that effectively encode both syntactic and semantic relationships across the vocabulary.

Glove Data

- Glove has pre-defined dense vectors for around every 6 billion words of English literature along with many other general-use characters like commas, braces and semicolons.
- The algorithm's developers frequently make the pre-trained GloVe embeddings available.
- Users can select a pre-trained GloVe embedding in a dimension like 50-d, 100-d, 200-d or 300-d vectors that best fits their needs in terms of computational resources and task specificity.
- Glove files are simple text files in the form of a dictionary.
- Words are key and dense vectors are values of key.

GloVe working:

The creation of a word co-occurrence matrix is the fundamental component of GloVe.

This matrix provides a quantitative measure of the semantic affinity between words by capturing the frequency with which they appear together in a given context.

Further, by minimising the difference between the dot product of vectors and the pointwise mutual information of corresponding words, GloVe optimises word vectors.
It is able to produce dense vector representations that capture syntactic and semantic relationships.

Algorithm for word embedding

- Preprocess the text data.
- Create the dictionary.
- Traverse the glove file of a specific dimension and compare each word with all words in the dictionary,
- if a match occurs, copy the equivalent vector from the glove and paste into embedding_matrix at the corresponding index.

Code Implementation

1. Importing Libraries

```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np
```

2. Creatong Vocabulary

Here we will create a list of words to build a vocabulary

```
# Define a sample corpus
texts = ['text', 'the', 'leader', 'prime', 'natural', 'language']
```

3. Initialize and Fit Tokenizer

- Initializes a tokenizer and fits it on the text corpus.
- Prints the number of unique words and their assigned indices.

```
# Create and fit the tokenizer
tokenizer = Tokenizer()
tokenizer.fit_on_texts(texts)

# Output the word-index dictionary
print("Number of unique words in dictionary =", len(tokenizer.word_index))
print("Dictionary is =", tokenizer.word_index)
```

4. Define function to Create embedding matrix

- Defines function that loads GloVe word vectors from file.
- Creates an embedding matrix matching tokenizer word indices with GloVe vectors.

```
def embedding_for_vocab(filepath, word_index, embedding_dim):
    vocab_size = len(word_index) + 1  # +1 for padding token (index 0)
    embedding_matrix_vocab = np.zeros((vocab_size, embedding_dim))

    with open(filepath, encoding="utf8") as f:
        for line in f:
```

```
        word, *vector = line.split()
        if word in word_index:
            idx = word_index[word]
            embedding_matrix_vocab[idx] = np.array(vector,
dtype=np.float32)[:embedding_dim]

    return embedding_matrix_vocab
```

5. Download GloVe File

Here we will Download and Unzip the GloVe file

```
# Download the GloVe dataset
!wget http://nlp.stanford.edu/data/glove.6B.zip

# Unzip the file
!unzip -q glove.6B.zip
```

6. Load GloVe Embeddings and Create Matrix

- Sets embedding dimension and file path.
- Generates the embedding matrix using the vocabulary.

```
# Set embedding dimension (match this with glove file)

embedding_dim = 50

# Path to GloVe file
glove_path = './glove.6B.50d.txt'

# Generate embedding matrix
embedding_matrix_vocab = embedding_for_vocab(glove_path,
tokenizer.word_index, embedding_dim)
```

7. Access Embedding Vector for a Word

Prints the GloVe vector for the word with index 1 in the tokenizer.

```
# Print the dense vector for the first word in the tokenizer index
first_word_index = 1  # Tokenizer indexes start from 1
print("Dense vector for word with index 1 =>",
embedding_matrix_vocab[first_word_index])
```

```
Dense vector for word with index 1 => [ 0.32615     0.36686    -0.0074905  -0.37553     0.66715002  0.21646
 -0.19801    -1.10010004 -0.42221001  0.10574    -0.31292     0.50953001
  0.55774999  0.12019     0.31441    -0.25042999 -1.06369996 -1.32130003
  0.87797999 -0.24627     0.27379    -0.51091999  0.49324     0.52243
  1.16359997 -0.75322998 -0.48052999 -0.11259    -0.54595    -0.83920997
  2.98250008 -1.19159997 -0.51958001 -0.39365    -0.1419     -0.026977
  0.66295999  0.16574    -1.1681      0.14443     1.63049996 -0.17216
 -0.17436001 -0.01049    -0.17794     0.93076003  1.0381      0.94265997
 -0.14805    -0.61109   ]
```

GloVe Embeddings Applications

GloVe embeddings are a popular option for representing words in text data and have found applications in various natural language processing (NLP) tasks. The following are some typical uses for GloVe embeddings:

- **Text Classification**
- **Named Entity Recognition (NER)**
- **Machine Translation**
- **Question Answering Systems**
- **Document Similarity and Clustering**
- **Word Analogy Tasks**
- **Semantic Search**

3.3 FastText

- FastText embeddings are a type of word embedding developed by Facebook's AI Research (FAIR) lab.
- They are based on the idea of subword embeddings, FastText breaks them down into smaller components called **character n-grams**.
- By doing so, FastText can capture the semantic meaning of morphologically related words, making it particularly useful for handling languages with rich morphology or for tasks where out-of-vocabulary words are common.

Advantages of FastText Embeddings:

- FastText offers a significant advantage over traditional word embedding techniques like Word2Vec and GloVe, especially for morphologically rich languages.
- Here's a breakdown of how FastText addresses the limitations of traditional word embeddings and its implications:
  - **Utilization of Character-Level Information**:
    - FastText takes advantage of character-level information by representing words as the average of embeddings their character n-grams.
    - This approach allows FastText to capture the internal structure of words.
    - It includes prefixes, suffixes, and roots, which is particularly beneficial for morphologically rich languages where word formations follow specific rules.
  - **Extension of Word2Vec Model:**
    FastText is an extension of the Word2Vec model, which means inherits the advantages of Word2Vec, such as capturing semantic relationships between words and producing dense vector representations.
  - **Handling Out-of-Vocabulary Words**:
    - One significant limitation of traditional word embeddings is their inability to handle out-of-vocabulary (OOV) words—words that are not present in the training data or vocabulary.

- Since Word2Vec and GloVe provide embeddings only for words seen during training, encountering an OOV word during inference can pose a challenge.

➢ **FastText's Solution for OOV Words**:

- FastText overcomes the limitation of OOV words by providing embeddings for character n-grams.
- If an OOV word occurs during inference, FastText can still generate an embedding for it based on its constituent character n-grams.
- This ability makes FastText more robust and suitable for handling scenarios where encountering new or rare words are common, such as social media data or specialized domains.

➢ **Improved Vector Representations for Morphologically Rich Languages:**

- By leveraging character-level information and providing embeddings for OOV words, FastText significantly improves vector representations for morphologically rich languages.
- It captures only the semantic meaning but also the internal structure and syntactic relations of words, leading to more accurate and contextually rich embeddings.

Working of FastText Embeddings

FastText embeddings revolutionize natural language processing by leveraging character-level information to generate robust word representations. For instance, consider the word **"basketball"** with the character n-grams:

"<ba, bas, ask, sket, ket, et, etb, tb, tb, bal, all, ll>" and "<basketball>".

❖ FastText computes the embedding for "basketball" by averaging the embeddings of these character n-grams along with the word itself.
❖ This approach captures both the semantic meaning and the internal structure of the word, making FastText particularly effective for morphologically rich languages.
❖ During training, FastText utilizes models like Continuous Bag of Words (CBOW) or Skip-gram, which are neural networks trained to predict context given a target word or vice versa.
❖ These models optimize neural network parameters to minimize loss, enabling FastText to learn meaningful word representations from large text corpora.

Furthermore, FastText's ability to handle out-of-vocabulary words helps in real-world applications where encountering new or rare words is common. Trained FastText embeddings serve as powerful features for various NLP tasks, facilitating tasks like text classification, sentiment analysis, and machine translation with improved accuracy and efficiency.

Skip-gram Vs CBOW

In the context of FastText embeddings, both Skip-gram and Continuous Bag of Words (CBOW) serve as training methodologies to generate word representations.
Considering the example of the word "basketball," let's compare how Skip-gram and CBOW operate:

Skip-gram:

- Input: Given the target word "basketball," Skip-gram aims to predict its context words, such as "play," "court," "team," etc.
- Training Objective: The model learns to predict the surrounding context words based on the target word.
- Example Usage: Given "basketball" as input, Skip-gram predicts its surrounding context words from a given text corpus.

Continuous Bag of Words (CBOW):

- Input: CBOW takes a window of context words, such as "play," "on," "the," "basketball," "court," as input.
- Training Objective: The model learns to predict the target word "basketball" given its surrounding context.
- Example Usage: With the context words "play," "on," "the," "court" provided as input, CBOW predicts the target word "basketball."
- In essence, Skip-gram and CBOW differ in their input and output configurations:

"Skip-gram predicts context words given the target word. CBOW predicts the target word given its context."

- Both methodologies contribute to training FastText embeddings, enabling the model to capture semantic relationships and syntactic structures effectively.
- The choice between Skip-gram and CBOW depends on the specific requirements of the NLP task and the characteristics of the text corpus being used.

Code Implementation of FastText Embeddings

- This code demonstrates training a FastText model using Gensim and using it to find word embeddings and similar words
- .It begins with importing the necessary libraries and defining a corpus, followed by the training of the FastText model with specified parameters.
- Word embeddings for a specific word ("computer" in this case) are then obtained from the trained model, and the most similar words to "computer" are found based on their embeddings.
- Finally, the word embedding for "computer" and the list of most similar words are printed.

```
from gensim.models import FastText
from gensim.test.utils import common_texts

# Example corpus (replace with your own corpus)
corpus = common_texts

# Training FastText model
model = FastText(sentences=corpus, vector_size=100, window=5,
min_count=1, workers=4, sg=1)

# Example usage: getting embeddings for a word
word_embedding = model.wv['computer']

# Most similar words to a given word
similar_words = model.wv.most_similar('computer')

print("Most similar words to 'computer':", similar_words)
```

Output:

Most similar words to 'computer': [('user', 0.15659411251544952), ('response', 0.12383826076984406),
('eps', 0.030704911798238754), ('system', 0.025573883205652237), ('interface', 0.0058587524108588696),
('survey', -0.03156976401805878), ('minors', -0.0545564740896225), ('human', -0.0668589174747467),
('time', -0.06855931878089905), ('trees', -0.10636083036661148)]

The output lists words along with their corresponding similarity scores to the word "computer." These scores indicate how semantically close each word is to "computer" within the model's learned vector space.

4. Cosine Similarity and Distance Measures

Distance Measures

- Distance measures play an important role in machine learning.
- They provide the foundation for many popular and effective machine learning algorithms like k-nearest neighbors for supervised learning and k-means clustering for unsupervised learning.
- Different distance measures must be chosen and used depending on the types of the data.

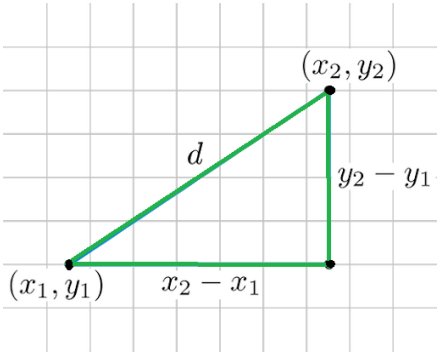
Types of Distance Measures in Machine Learning:

1. Euclidean Distance
2. Manhattan Distance

3. Minkowski Distance
4. Hamming Distance

Euclidean Distance

- Euclidean Distance represents the shortest distance between two vectors. It is the square root of the sum of squares of differences between corresponding elements.
- The Euclidean distance metric corresponds to the L2-norm of a difference between vectors and vector spaces.
- The cosine similarity is proportional to the dot product of two vectors and inversely proportional to the product of their magnitudes.
- Most machine learning algorithms, including K-Means use this distance metric to measure the similarity between observations. Let's say we have two points, as shown below:



Euclidean Distance Formula

Consider two points (x1, y1) and (x2, y2) in a 2-dimensional space; the Euclidean Distance between them is given by using the formula:

d = √[(x2 - x1)2 + (y2 - y1)2]

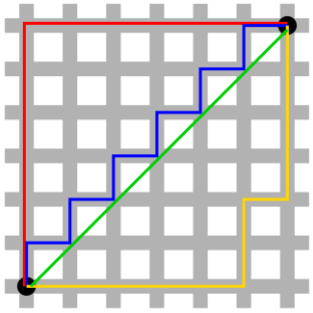
Where,

- d is Euclidean Distance
- (x1, y1) is Coordinate of the first point
- (x2, y2) is Coordinate of the second point

Manhattan Distance

- Manhattan Distance is the sum of absolute differences between points across all the dimensions.
- This determines the absolute difference among the pair of the coordinates. Suppose we have two points P and Q to determine the distance between these points we simply have to calculate the perpendicular distance of the points from X-Axis and Y-Axis.
- In a plane with P at coordinate (x1, y1) and Q at (x2, y2). Manhattan distance between P and Q = |x1 – x2| + |y1 – y2|

We can represent Manhattan Distance as:



Formula for Manhattan Distance

Minkowski Distance

Minkowski Distance is the generalized form of Euclidean and Manhattan Distance.

N-dimensional space, a point is represented as, (x1, x2, ..., xN)

Consider two points P1 and P2:

P1: (X1, X2, ..., XN)

P2: (Y1, Y2, ..., YN)

Then, the Minkowski distance between P1 and P2 is given as:

$$D = \left(\sum_{i=1}^{n} |p_i - q_i|^p \right)^{1/p}$$

Here, p represents the order of the norm. The p parameter of the Minkowski Distance metric of SciPy represents the order of the norm. When the order(p) is 1, it will represent Manhattan Distance and when the order in the above formula is 2, it will represent Euclidean Distance.

Hamming Distance

In machine learning, the hamming distance measures the similarity between two strings of the same length. It is the number of positions at which the corresponding characters are different.

Let's understand the concept using an example. Let's say we have two strings:

"euclidean" and **"manhattan"**

Since the length of these strings is equal, we can calculate the Hamming Distance. We will match the strings character by character. The first

character of both strings (e and m, respectively) differs. Similarly, the second character of both strings (u and a) is different, and so on.
Look carefully – seven characters are different, whereas two characters (the last two characters) are similar:
Hence, the hamming distance here will be **7**.

Hamming Distance Calculator

To compute the Hamming distance between two strings, and, we conduct their XOR operation, (a⊕ b), and then count the total number of 1s in the resultant string.

Example

Assume you have two strings 11011001 and 10011101.
11011001 ⊕ 10011101 = 01000100.
Because this has two 1s, the Hamming distance is d (11011001, 10011101) = 2.

Non-Metric Similarity Functions

Non-metric similarity functions are a category of functions used to measure similarity between two objects in machine learning and data analysis. Unlike traditional **metric** functions, non-metric functions don't necessarily satisfy the typical properties of a metric, such as:

- **Non-negativity**: The similarity is always non-negative.
- **Identity of indiscernibles**: Similarity is zero only if the objects are identical.
- **Symmetry**: Similarity between two objects is the same in both directions.
- **Triangle inequality**: The similarity between two objects should be less than or equal to the similarity between any intermediate object.

In the case of **non-metric** similarity functions, they might fail one or more of these properties, but they can still provide useful ways to compare objects in certain contexts, especially in cases where the metric properties are not important or do not capture the true relationships between objects. Some examples of non-metric similarity functions include:

1. Cosine Similarity

Cosine similarity measures the cosine of the angle between two non-zero vectors in an inner product space. It is often used in text analysis, where documents are represented as vectors in a high-dimensional space (like TF-IDF vectors). The cosine similarity is defined as:

$$\text{cosine similarity}(A,B) = A \cdot B \,/\, \|A\| \, \|B\|$$

Non-metric aspects: It doesn't satisfy the triangle inequality, and it can be symmetric but may not satisfy other properties.

2. Jaccard Similarity

Jaccard similarity is used for comparing the similarity and diversity of sample sets. It is defined as the size of the intersection divided by the size of the union of two sets:
Jaccard similarity(A,B)=|A∩B|/|A∪B|

Non-metric aspects: It does not satisfy the triangle inequality. For example, if A⊂BA then the Jaccard similarity A,C might not be directly comparable to B,C, violating the triangle inequality.

3. Hamming Distance

Hamming distance is used for comparing two strings or sequences of equal length. It counts the number of positions at which the corresponding elements are different. In binary strings, this would be the number of bit positions that differ between two binary strings.

- **Non-metric aspects**: It fails to satisfy the triangle inequality. For example, if A is one bit different from B and B is one bit different from C, it does not imply that A is two bits different from C.

4. Kernel Functions (in Support Vector Machines)

In some machine learning models, like Support Vector Machines (SVMs), **kernel functions** are used to calculate the similarity between two points in a high-dimensional space. Kernels such as the **Radial Basis Function (RBF)** or **Gaussian Kernel** don't always satisfy metric properties (especially the triangle inequality) but are used because they can map input data into a space where classes are more easily separable. The RBF kernel is defined as:

$K(x,y)=\exp(-\|x-y\|^2) / 2\sigma^2$

Non-metric aspects: It is not a true distance function and does not satisfy all metric properties, especially triangle inequality.

5. Mahalanobis Distance

Mahalanobis distance is a measure of the distance between a point and a distribution. Unlike Euclidean distance, it accounts for correlations of the data set and the covariance of the data.

$D_M(x,y) = \sqrt{(x-y)^T S^{-1}(x-y)}$

Non-metric aspects: In some cases, especially when dealing with data that is not distributed according to the assumed distribution, the Mahalanobis distance might fail some of the metric properties.

Uses of Non-Metric Functions

Non-metric similarity functions can still be highly useful in machine learning because they are often better suited to the inherent structure or relationships of the data. For instance:

- In text or natural language processing, cosine similarity might be more meaningful than Euclidean distance, as it captures the orientation of vectors (e.g., documents) in high-dimensional space, not just their magnitude.
- Jaccard similarity is used in cases of binary or set-based data, such as measuring overlap between sets of items.
- Kernel methods provide a powerful way of handling complex data structures that may not fit well into traditional metric spaces.

While non-metric similarity functions may not always have the formal properties of a metric, they can offer more flexibility and often lead to better performance in real-world machine learning tasks.

5. Naive Bayes Classifiers

Naive Bayes is a classification algorithm that uses probability to predict which category a data point belongs to, assuming that all features are unrelated.

Illustration behind the Naive Bayes algorithm.

We estimate $P(x_a|y)$ independently in each dimension and then obtain an estimate of the full data distribution by assuming conditional independence $P(x|y)=\prod_a P(x_a|y)$.

Key Features of Naive Bayes Classifiers

The main idea behind the Naive Bayes classifier is to use Bayes' Theorem to classify data based on the probabilities of different classes given the features of the data. It is used mostly in high-dimensional **text classification.**

- The Naive Bayes Classifier is a simple probabilistic classifier.
- It is a probabilistic classifier because it assumes that one feature in the model is independent of existence of another feature.
- Naïve Bayes Algorithm is used in spam filtration, Sentimental analysis, classifying articles and many more.

Assumption of Naive Bayes

The fundamental Naive Bayes assumption is that each feature makes an:

- **Feature independence:** This means that when we are trying to classify something, we assume that each feature (or piece of information) in the data does not affect any other feature.
- **Continuous features are normally distributed:** If a feature is continuous, then it is assumed to be normally distributed within each class.

- **Discrete features have multinomial distributions:** If a feature is discrete, then it is assumed to have a multinomial distribution within each class.
- **Features are equally important:** All features are assumed to contribute equally to the prediction of the class label.
- **No missing data:** The data should not contain any missing values.

Introduction to Bayes' Theorem

Bayes' Theorem provides a principled way to reverse conditional probabilities. It is defined as:

$P(y|X)=P(X|y)\cdot P(y)/P(X)$

Where:

- P(y|X): Posterior probability, probability of class y*y* given features X*X*
- P(X|y): Likelihood, probability of features X*X* given class y*y*
- P(y): Prior probability of class y*y*
- P(X): Marginal likelihood or evidence

Naive Bayes Working

1. Terminology

Consider a classification problem (like predicting if someone plays golf based on weather). Then:

- y is the class label (e.g. "Yes" or "No" for playing golf)
- X=(x1,x2,...,xn) is the feature vector (e.g. Outlook, Temperature, Humidity, Wind)

A sample row from the dataset:

X=(Rainy, Hot, High, False),y=No

This represents:

What is the probability that someone will not play golf given that the weather is Rainy, Hot, High humidity, and No wind?

2. The Naive Assumption

The "naive" in Naive Bayes comes from the assumption that all features are independent given the class. That is:

$P(x1,x2,...,xn|y)=P(x1|y)\cdot P(x2|y)\cdots P(xn|y)$

Thus, Bayes' theorem becomes:

$$P(y|x_1,...,x_n) = \frac{P(y)\cdot\prod_{i=1}^{n} P(x_i|y)}{P(x_1)P(x_2)...P(x_n)}$$

Since the denominator is constant for a given input, we can write:

$$P(y|x_1,...,x_n) \propto P(y)\cdot\prod_{i=1}^{n} P(x_i|y)$$

3. Constructing the Naive Bayes Classifier

We compute the posterior for each class y*y* and choose the class with the highest probability:

$$\hat{y} = \arg\max_y P(y)\cdot\prod_{i=1}^{n} P(x_i|y)$$

This becomes our Naive Bayes classifier.

4. Example: Weather Dataset

Consider a fictional dataset that describes the weather conditions for playing a game of golf. Given the weather conditions, each tuple classifies the conditions as fit("Yes") or unfit("No") for playing golf. Here is a tabular representation of our dataset.

	Outlook	Temperature	Humidity	Windy	Play Golf
0	Rainy	Hot	High	False	No
1	Rainy	Hot	High	True	No
2	Overcast	Hot	High	False	Yes
3	Sunny	Mild	High	False	Yes
4	Sunny	Cool	Normal	False	Yes
5	Sunny	Cool	Normal	True	No
6	Overcast	Cool	Normal	True	Yes
7	Rainy	Mild	High	False	No
8	Rainy	Cool	Normal	False	Yes
9	Sunny	Mild	Normal	False	Yes
10	Rainy	Mild	Normal	True	Yes
11	Overcast	Mild	High	True	Yes
12	Overcast	Hot	Normal	False	Yes

	Outlook	Temperature	Humidity	Windy	Play Golf
13	Sunny	Mild	High	True	No

The dataset is divided into two parts, namely, feature matrix and the response vector.

- Feature matrix contains all the vectors(rows) of dataset in which each vector consists of the value of dependent features. In above dataset, features are 'Outlook', 'Temperature', 'Humidity' and 'Windy'.
- Response vector contains the value of class variable(prediction or output) for each row of feature matrix. In above dataset, the class variable name is 'Play golf'.

Let's take a dataset used for predicting if golf is played based on:

- Outlook: Sunny, Rainy, Overcast
- Temperature: Hot, Mild, Cool
- Humidity: High, Normal
- Wind: True, False

Outlook

	Yes	No	P(yes)	P(no)
Sunny	3	2	3/10	2/4
Overcast	4	0	4/10	0/4
Rainy	3	2	3/10	2/4
Total	10	4	100%	100%

Temperature

	Yes	No	P(yes)	P(no)
Hot	2	2	2/9	2/5
Mild	4	2	4/9	2/5
Cool	3	1	3/9	1/5
Total	9	5	100%	100%

Humidity

	Yes	No	P(yes)	P(no)
High	3	4	3/9	4/5
Normal	6	1	6/9	1/5
Total	9	5	100%	100%

Wind

	Yes	No	P(yes)	P(no)
False	6	2	6/9	2/5
True	3	3	3/9	3/5
Total	9	5	100%	100%

Play		P(Yes)/P(No)
Yes	9	9/14
No	5	5/14
Total	14	100%

Example Tables for Naive Bayes:
Example Input:
X=(Sunny,Hot,Normal,False)
Goal: Predict if golf will be played (Yes or No).

5. Pre-computation from Dataset
Class Probabilities:
From dataset of 14 rows:

- P(Yes)=9/14
- P(No)=5/14

Conditional Probabilities (Tables 1–4):

Feature	Value	P (Value | Yes)	P (Value | No)
Outlook	Sunny	2/9	3/5
Temperature	Hot	2/9	2/5
Humidity	Normal	6/9	1/5
Wind	False	6/9	2/5

6. Calculate Posterior Probabilities
For Class = Yes:

$$P(\text{Yes} \mid \text{today}) \propto \frac{2}{9} \cdot \frac{2}{9} \cdot \frac{6}{9} \cdot \frac{6}{9} \cdot \frac{9}{14}$$

$$P(\text{Yes} \mid \text{today}) \approx 0.02116$$

For Class = No:

$$P(\text{No} \mid \text{today}) \propto \frac{3}{5} \cdot \frac{2}{5} \cdot \frac{1}{5} \cdot \frac{2}{5} \cdot \frac{5}{14}$$

$$P(\text{No} \mid \text{today}) \approx 0.0068$$

7. Normalize Probabilities
To compare:

$$P(\text{Yes} \mid \text{today}) = \frac{0.02116}{0.02116+0.0068} \approx 0.756$$

$$P(\text{No} \mid \text{today}) = \frac{0.0068}{0.02116+0.0068} \approx 0.244$$

8. Final Prediction

Since:

P(Yes | today)>P(No | today)

The model predicts: Yes (Play Golf)

Disadvantages of Naive Bayes Classifier

- Assumes that features are independent, which may not always hold in real-world data.
- Can be influenced by irrelevant attributes.
- May assign zero probability to unseen events, leading to poor generalization.

Applications of Naive Bayes Classifier

- **Spam Email Filtering**: Classifies emails as spam or non-spam based on features.
- **Text Classification**: Used in sentiment analysis, document categorization, and topic classification.
- **Medical Diagnosis**: Helps in predicting the likelihood of a disease based on symptoms.
- **Credit Scoring**: Evaluates creditworthiness of individuals for loan approval.
- **Weather Prediction**: Classifies weather conditions based on various factors.

6. Text Classification using SVM

Text Classification:

- Text Classification is an automated process of classification of text into predefined categories.
- We can classify Emails into spam or non-spam, news articles into different categories like Politics, Stock Market, Sports, etc.
- This can be done with the help of Natural Language Processing and different Classification Algorithms like Naive Bayes, SVM and even Neural Networks in Python.

SVM for classification:

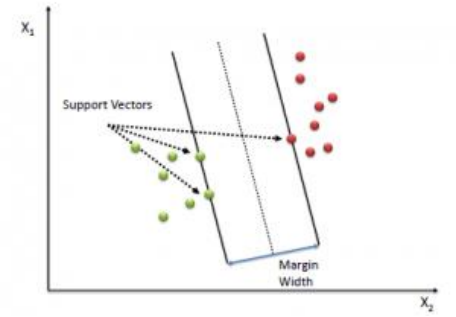
- Support vector machines are a type of supervised machine algorithm for learning which is used for classification and regression tasks.
- Though they are used for both classification and regression, they are mainly used for classification challenges.
- It tries to find the best boundary known as hyperplane that separates different classes in the data.
- The main goal of SVM is to maximize the margin between the two classes. The larger the margin the better the model performs on new and unseen data.

Key Concepts of Support Vector Machine

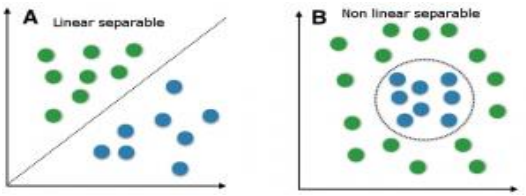
Hyperplane: A **decision boundary** separating different classes in feature space and is represented by the equation wx + b = 0 in linear classification.

- **Support Vectors**: The **closest data points** to the hyperplane, crucial for determining the hyperplane and margin in SVM.
- **Margin**: The distance between the **hyperplane and the support vectors**. SVM aims to **maximize** this margin for better classification performance.
- **Kernel**: A function that maps data to a higher-dimensional space enabling SVM to handle non-linearly separable data.
- **Hard Margin**: A maximum-margin hyperplane that perfectly separates the data without misclassifications.
- **Soft Margin**: Allows some misclassifications by introducing slack variables, balancing margin maximization and misclassification penalties when data is not perfectly separable.
- **C**: A **regularization term** balancing margin maximization and misclassification penalties. A higher C value forces stricter penalty for misclassifications.
- **Hinge Loss**: A loss function penalizing misclassified points or margin violations and is combined with regularization in SVM.
- **Dual Problem**: Involves solving for Lagrange multipliers associated with support vectors, facilitating the kernel trick and efficient computation.

Support Vector Algorithm is performed by plotting each acquired value of data as a point on an n-dimensional space or graph. Where "n" represents the total number of a features of data that is present.
The value of each data is represented as a particular coordinate on the graph.
After distribution of coordinate data, we can perform classification by finding the line or hyper-plane that distinctly divides and differentiates between the two classes of data.



- Support vectors are the data points nearest to the hyperplane.
- Because of this, they can be considered the critical elements of a data set.
- As a simple example, consider the image given above, for a classification task with only two features, we can think of a hyperplane as a line that linearly separates and classifies a set of data. This is shown below:



- ❖ Furthermore, from the hyperplane our data points lie, the more confident we are that they have been correctly classified.
- ❖ We therefore want our data points to be as far away from the hyperplane as possible, while still being on the correct side of it.

The SVM optimizes the following equation to balance margin maximization and penalty minimization:

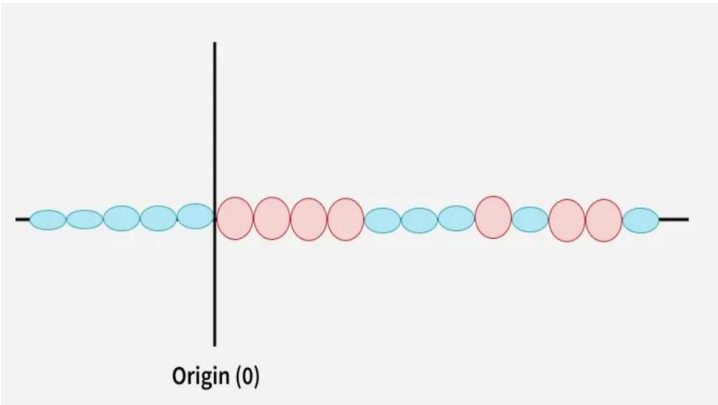
$$\text{Objective Function} = \left(\frac{1}{\text{margin}}\right) + \lambda \sum \text{penalty}$$

The penalty used for violations is often hinge loss which has the following behaviour:

- ➢ If a data point is correctly classified and within the margin there is no penalty (loss = 0).
- ➢ If a point is incorrectly classified or violates the margin the hinge loss increases proportionally to the distance of the violation.

Classification of non-linearly separable data:

- ❖ When data is not linearly separable i.e it can't be divided by a straight line, SVM uses a technique called **kernels** to map the data into a higher-dimensional space where it becomes separable.
- ❖ This transformation helps SVM to find a decision boundary even for non-linear data.

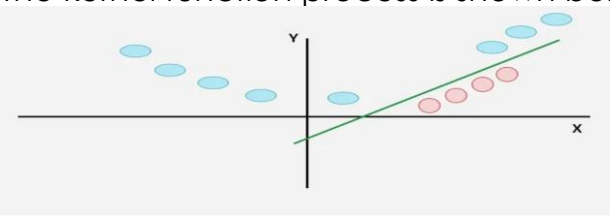


By applying a kernel function SVM transforms the data points into a higher-dimensional space where they become linearly separable. Different types of kernels are:

- ➢ **Linear Kernel**: For linear separability.
- ➢ **Polynomial Kernel**: Maps data into a polynomial space.

- **Radial Basis Function (RBF) Kernel**: Transforms data into a space based on distances between data points.

The kernel function process is shown below:



In this case the new variable y is created as a function of distance from the origin.

Mathematical Computation of SVM:

- Consider a binary classification problem with two classes, labelled as +1 and -1.
- We have a training dataset consisting of input feature vectors X and their corresponding class labels Y.
- The equation for the linear hyperplane can be written as:

$$w^T x + b = 0$$

Where:

- w is the normal vector to the hyperplane (the direction perpendicular to it).
- b is the offset or bias term representing the distance of the hyperplane from the origin along the normal vector w.

Distance from a Data Point to the Hyperplane

The distance between a data point xi and the decision boundary can be calculated as:

$$d_i = \frac{w^T x_i + b}{||w||}$$

where | |w| | represents the Euclidean norm of the weight vector w.

Linear SVM Classifier

Distance from a Data Point to the Hyperplane:

$$\hat{y} = \begin{cases} 1 & : w^T x + b \geq 0 \\ 0 & : w^T x + b < 0 \end{cases}$$

Where y^ is the predicted label of a data point.

SVM Decision Boundary
The decision boundary is given by:

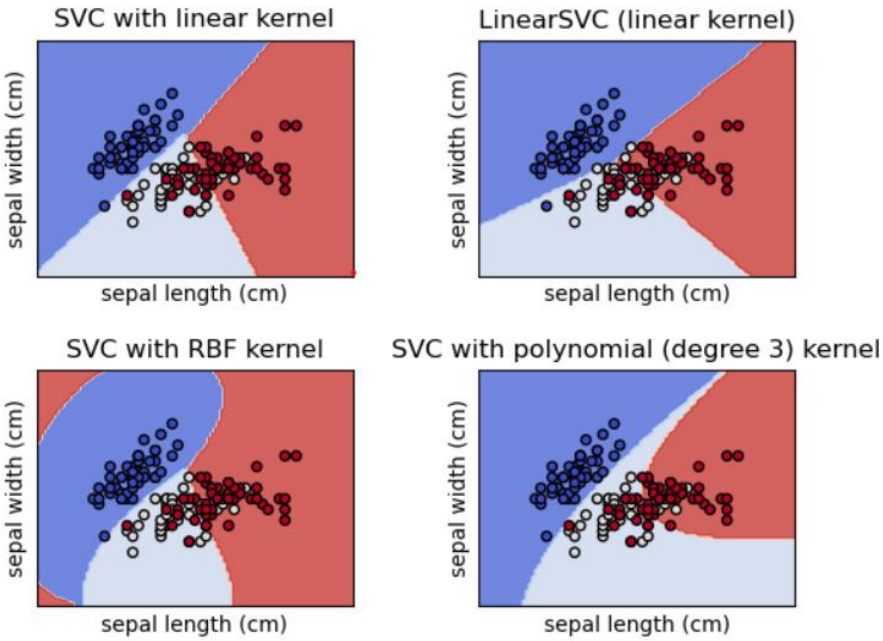
$$w = \sum_{i=1}^{m} \alpha_i t_i K(x_i, x) + b$$

Where w is the weight vector, x is the test data point and b is the bias term.
Finally the bias term *b* is determined by the support vectors, which satisfy:

$$t_i(w^T x_i - b) = 1 \quad \Rightarrow \quad b = w^T x_i - t_i$$

Where xi is any support vector.

Classification by SVM in Python:
SVC, NuSVC and LinearSVC are classes capable of performing binary and multi-class classification on a dataset in Python.



Example text classification with SVM:
movie reviews labeled as positive/ negative

program:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.svm import LinearSVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
```

```
# Sample text data and labels
documents = [
    "This movie is fantastic and I loved it.",
    "The plot was boring and the acting was terrible.",
    "A truly great film with superb direction.",
    "I hated every minute of this dreadful movie."
]
labels = ["positive", "negative", "positive", "negative"]

# Initialize TF-IDF Vectorizer
vectorizer = TfidfVectorizer()

# Fit and transform the documents
X = vectorizer.fit_transform(documents)

#model training
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, labels, test_size=0.25,
random_state=42)

# Initialize and train LinearSVC
svm_model = LinearSVC()
svm_model.fit(X_train, y_train)
#Prediction and Evaluation

# Predict on the test set
y_pred = svm_model.predict(X_test)
print(y_pred)
```

output:

```
['positive']
```

#Model evaluation:

```
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")
```

output:

```
Accuracy: 0.0
```

Applications of Support Vector Machine:

- Text Classification
- Spam Detection
- Sentiment Analytics

- Image Recognition
- Postal Automation Services

Advantages of Support Vector Machine (SVM)

1. **High-Dimensional Performance**: SVM excels in high-dimensional spaces, making it suitable for image classification and gene expression analysis.
2. **Nonlinear Capability**: Utilizing kernel functions like RBF and polynomial SVM effectively handles nonlinear relationships.
3. **Outlier Resilience**: The soft margin feature allows SVM to ignore outliers, enhancing robustness in spam detection and anomaly detection.
4. **Binary and Multiclass Support**: SVM is effective for both binary classification and multiclass classification suitable for applications in text classification.
5. **Memory Efficiency**: It focuses on support vectors making it memory efficient compared to other algorithms.

Disadvantages of Support Vector Machine (SVM)

1. **Slow Training**: SVM can be slow for large datasets, affecting performance in SVM in data mining tasks.
2. **Parameter Tuning Difficulty**: Selecting the right kernel and adjusting parameters like C requires careful tuning, impacting SVM algorithms.
3. **Noise Sensitivity**: SVM struggles with noisy datasets and overlapping classes, limiting effectiveness in real-world scenarios.
4. **Limited Interpretability**: The complexity of the hyperplane in higher dimensions makes SVM less interpretable than other models.
5. **Feature Scaling Sensitivity**: Proper feature scaling is essential, otherwise SVM models may perform poorly.

Differences between Naive Bayes and SVM for Text Classification

Criteria	Naive Bayes	Support Vector Machine
Advantages	➢ Simple and easy to implement. ➢ Computationally efficient. ➢ Works well with small datasets.	➢ Effective in high-dimensional spaces. ➢ Robust to overfitting. ➢ Flexibility in choosing kernel functions. ➢ Can capture complex relationships.

Criteria	Naive Bayes	Support Vector Machine
Efficiency	➢ Fast training and prediction.	➢ Training can be computationally expensive. ➢ Slower training but faster prediction.
Performance	➢ Good for simple classification tasks. ➢ Can handle noisy data well.	➢ Better performance in complex tasks. ➢ Sensitive to noisy data, especially if it affects the positioning of the decision boundary.
Scalability	➢ Scales well with large datasets and features.	➢ Less scalable with large datasets. ➢ Memory-intensive for large feature spaces.
Interpretability	➢ Provides straightforward interpretability. ➢ Directly calculates class probabilities.	➢ Provides less interpretability. ➢ Decision boundaries are harder to interpret. ➢ Provides little insight into feature importance.
Robustness	➢ Sensitive to feature distribution. ➢ Can be sensitive to violations of independence assumption.	➢ More robust to outliers and noise.
Limitations	➢ Feature dependence challenges validity, impacting Naive Bayes' performance. ➢ Naive Bayes' simplicity compromise accuracy on intricate	➢ SVM training demands significant computational resources for large datasets. ➢ SVM success relies on precise tuning of kernel

Criteria	Naive Bayes	Support Vector Machine
	relationships. ➢ Feature distribution deviations impair Naive Bayes' performance assumptions.	and regularization. ➢ SVMs lack interpretability, especially in text classification with numerous features.

7.Evaluation Metrics of classification

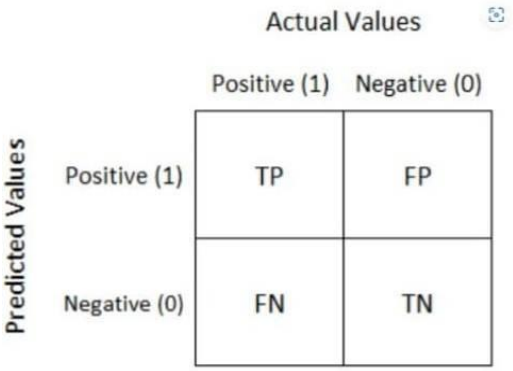
Confusion Matrix:

Confusion Matrix is a performance evaluation measurement for the machine learning classification problems where the output can be two or more classes. It is a table with combinations of predicted and actual values.

Definition:

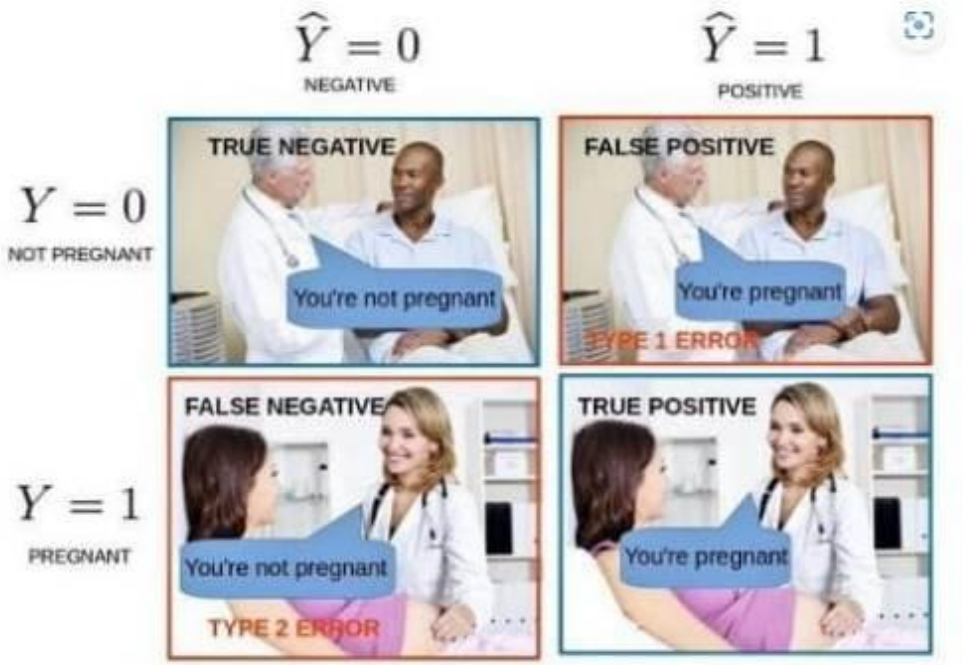
A confusion matrix is defined as the table that is often used to describe the performance of a classification model on a set of the test data for which the true values are known.

Is looks like this:



It is extremely useful for measuring the Recall, Precision, Accuracy, and AUC-ROC curves.

Let's try to understand TP, FP, FN, TN with an example of pregnancy analogy.

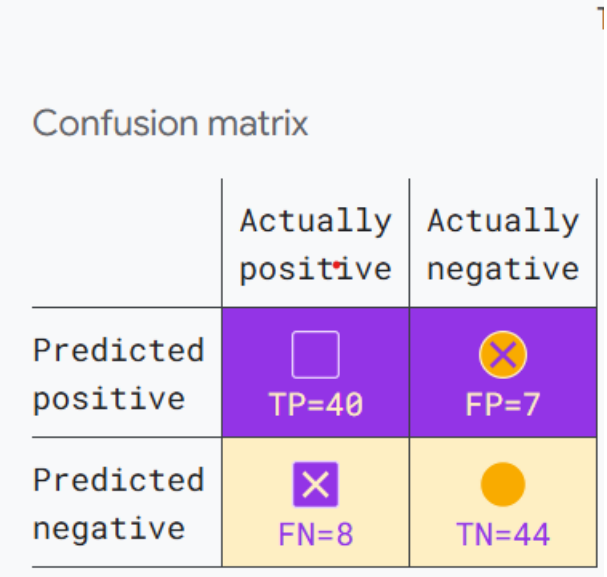


- **True Positive:** We predicted positive and it's true. In the image, we predicted that a woman is pregnant and she actually is.
- **True Negative:** We predicted negative and it's true. In the image, we predicted that a man is not pregnant and he actually is not.
- **False Positive (Type 1 Error)**: We predicted positive and it's false. In the image, we predicted that a man is pregnant but he actually is not.
- **False Negative (Type 2 Error)**: We predicted negative and it's false. In the image, we predicted that a woman is not pregnant but she actually is.

Evaluation metrics (Measures):

- Accuracy
- Precision
- Recall (Sensitivity)
- F1 Score

Example confusion matrix: let us consider some textual data observations which are distributed in the confusion matrix as shown below:



Accuracy

Accuracy simply measures how often the classifier correctly predicts. We can define accuracy as the ratio of the number of correct predictions and the total number of predictions.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy = 40+44/99= 0.85

Precision

It explains how many of the correctly predicted cases actually turned out to be positive.

Precision for a label is defined as the number of true positives divided by the number of predicted positives.

$$Precision = \frac{True\,Positive}{True\,Positive + False\,Positive}$$

Precision = 40/40+7 = 0.85

Recall (Sensitivity)

It explains how many of the actual positive cases we were able to predict correctly with our model.

Recall is a useful metric in cases where False Negative is of higher concern than False Positive.

It is important in medical cases where it doesn't matter whether we raise a false alarm but the actual positive cases should not go undetected!

Recall for a label is defined as the number of true positives divided by the total number of actual positives.

$$Recall = \frac{TruePositive}{TruePositive + FalseNegative}$$

Recall = 40/40+8=0.83

F1 Score

It gives a combined idea about Precision and Recall metrics. It is maximum when Precision is equal to Recall.

F1 Score is the harmonic mean of precision and recall.

$$F1 = 2.\frac{Precision \times Recall}{Precision + Recall}$$

F1 Score = 2*(0.85*0.83)/(0.85+0.83)
=2*(0.71/1.68)
=2*0.42
=0.85

The F1 score punishes extreme values more. F1 Score could be an effective evaluation metric in the following cases:

- When FP and FN are equally costly.
- Adding more data doesn't effectively change the outcome
- True Negative is high

END OF UNIT - II

UNIT III
Syllabus

Deep Learning for NLP

Neural Network Basics for NLP, Recurrent Neural Networks (RNNs) and Limitations, LSTM and GRU Networks,
Sequence Labeling: POS Tagging, NER using Bi-LSTM, Text Classification using CNNs and RNNs, Model Evaluation and Hyperparameter Tuning.

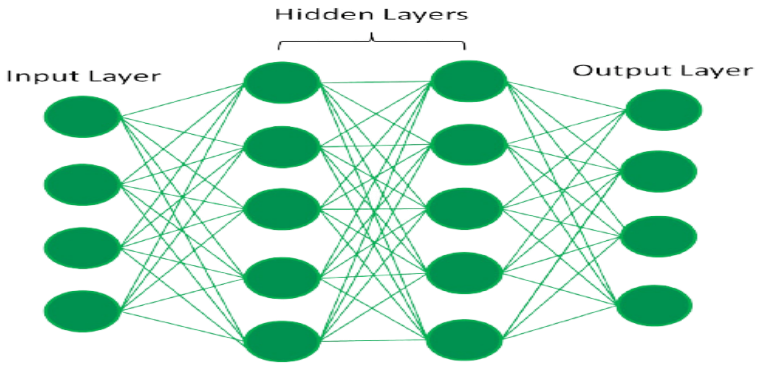
STUDY MATERIAL

Prepared by **K.ANJANEYULU, M. Tech (CSE), MBA, PGDCA, M. Sc, B. Ed.,**
Assistant Professor,
Department of Computer Science & Engineering (AI&ML),
Sri Venkateswara College of Engineering & Technology,
RVS Nagar. Chittoor.
Phno: **(+91) – 9640985702, (+91) – 9398526101**
E-mail: **anji.karapakula@gmail.com**

1. Neural Network Basics for NLP

Neural Network Basics for NLP

- Neural networks process and understand human language by learning complex patterns from text data, powering Natural Language Processing (NLP) applications like machine translation, sentiment analysis, and chatbots. The evolution of neural network architectures, from simple feedforward networks to advanced Transformers, has transformed the field by enabling models to handle context and long-range dependencies more effectively.
- Neural network consists of layers of interconnected nodes or neurons that collaborate to process input data.
- In a fully connected deep neural network data flows through multiple layers where each neuron performs nonlinear transformations, allowing the model to learn complicated representations of the data.
- In a deep neural network the input layer receives data which passes through hidden layers that transform the data using nonlinear functions. The final output layer generates the model's prediction. A general neural network is shown below:



Preprocessing text for neural networks

- To be processed by a neural network, raw text must first be converted into a numerical format through a crucial preprocessing pipeline.
- Tokenization: The text is split into smaller units called "tokens," which are typically words or sub-words.
- Text cleaning: Unwanted elements like punctuation, special characters, and HTML tags are removed.
- Stopword removal: Common words that carry little meaning (e.g., "a," "the," "is") are filtered out.
- Vector representation: Words are converted into numerical vectors. Modern NLP uses advanced methods that capture semantic meaning, unlike older methods like one-hot encoding.

Word embeddings

- Word embeddings are dense, low-dimensional vector representations that capture a word's meaning and context.

- In this vector space, semantically similar words are mapped to proximate points.
- Traditional one-hot encoding: Represents each word as an orthogonal vector with a size equal to the vocabulary. This creates a sparse, high-dimensional vector that fails to capture any relationships between words.
- Word2Vec and GloVe: These models learn to generate dense word vectors by considering the context in which words appear. For example, the vector relationship in Word2Vec can represent analogies like "king - man + woman = queen".
- Contextual embeddings (ELMo and BERT): Advanced models like BERT produce dynamic word embeddings where the vector for a word can change based on the specific context of the sentence it appears in.

Key neural network architectures for NLP

Recurrent Neural Networks (RNNs)

- RNNs are designed to handle sequential data by using recurrent connections that allow them to remember information from previous time steps.
- How they work: At each step, an RNN processes the current input and an internal "hidden state" that contains information from the preceding steps. This makes them suitable for tasks like next-word prediction.
- Limitations: Simple RNNs suffer from the "vanishing gradient problem," which makes it difficult to retain information over long sequences.

Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU)

- These are advanced variants of RNNs developed to solve the vanishing gradient problem and capture long-term dependencies in text.
- LSTMs: Use a complex gating mechanism (input, forget, and output gates) to regulate the flow of information into a memory cell, allowing them to remember or forget context selectively.
- GRUs: A simplified version of LSTMs that combine the forget and input gates into a single "update gate." They offer comparable performance to LSTMs with fewer parameters and are computationally more efficient.

Sequence-to-Sequence (Seq2Seq) models

- This architecture, often built with RNNs, takes an input sequence and generates an output sequence.
- Encoder-decoder structure: A Seq2Seq model has an encoder that processes the input sequence into a context vector and a decoder that uses this vector to generate the output sequence one token at a time.
- Attention mechanism: A key innovation that allows the decoder to focus on the most relevant parts of the input sequence at each step, significantly improving performance, especially for longer sequences.

Transformers

- Introduced in 2017, the Transformer architecture entirely replaces recurrent layers with a more powerful "self-attention" mechanism.
- How they work: Unlike RNNs that process sequentially, Transformers process all words simultaneously, allowing for much faster training on larger datasets. The self-attention mechanism determines the importance

of different words relative to each other, regardless of their position in the sequence, which allows them to capture long-range dependencies effectively.

- Pre-trained models: This architecture has led to the development of powerful pre-trained models like BERT and GPT. These are initially trained on massive amounts of text data and can then be "fine-tuned" for specific downstream tasks with smaller datasets

2. Recurrent Neural Networks (RNNs) and Limitations

Recurrent Neural Networks (RNNs) are a class of neural networks designed to process **sequential** data, such as text, where the order of information is crucial for its meaning.
They have a unique architecture that allows them to maintain a **"memory"** of previous inputs, influencing the processing of subsequent data points

How RNNs work in NLP

An RNN processes a sequence of text one item at a time, such as word-by-word or character-by-character

Example:

RNNs work by "remembering" past information and passing the output from one step as input to the next i.e it considers all the earlier words to choose the most likely next word.
This memory of previous steps helps the network understand context and make better predictions.

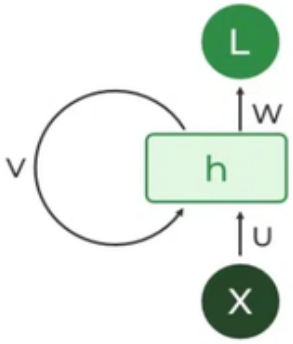
Key Components of RNNs

There are mainly two components of RNNs

1. Recurrent Neurons
2. RNN Unfolding

Recurrent Neurons

- The fundamental processing unit in RNN is a Recurrent Unit. They hold a hidden state that maintains information about previous inputs in a sequence.
- Recurrent units can "remember" information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.



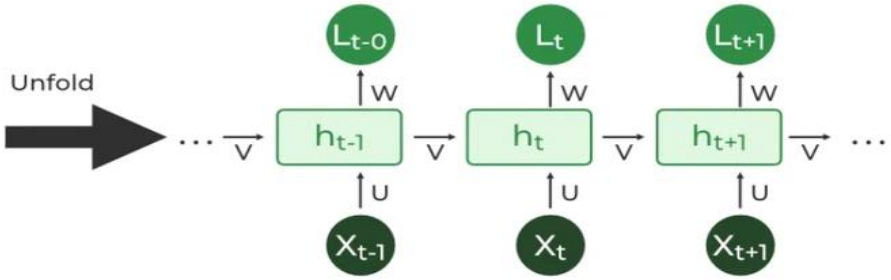
RNN Unfolding

RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps.

During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.

This unrolling enables backpropagation through time (BPTT) a learning process.

In this approach, errors are propagated across time steps to adjust the network's weights enhancing the RNN's ability to learn dependencies within sequential data.



Recurrent Neural Network Architecture

RNNs use shared weights across time steps, allowing them to remember information over sequences.

In RNNs the hidden state Hi is calculated for every input *Xi* to retain sequential dependencies. The computations follow these core formulas:

Hidden State Calculation:

$$h=\sigma(U\cdot X+W\cdot h_{t-1}+B)$$

Here:

h represents the current hidden state.

U and W are weight matrices.

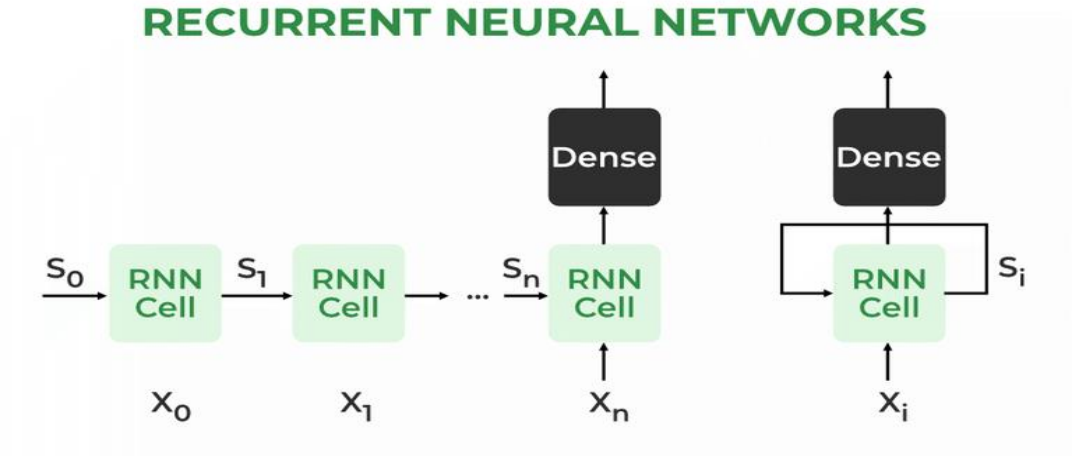
B is the bias.

Output Calculation:

Y=O(V·h+C)
The output Y is calculated by applying O an activation function to the weighted hidden state where V and C represent weights and bias.

Overall Function:
Y=f(X,h,W,U,V,B,C)

This function defines the entire RNN operation where the state matrix S holds each element si representing the network's state at each time step i.



RNN working:
At each time step RNNs process units with a fixed activation function. These units have an internal hidden state that acts as memory that retains information from previous time steps.
This memory allows the network to store past knowledge and adapt based on new inputs.

Updating the Hidden State in RNNs
The current hidden state ht depends on the previous state ht−1 and the current input xt and is calculated using the following relations:

$h_t=f(h_{t-1},x_t)$

where:

- ht is the current state
- ht−11 is the previous state
- xt is the input at the current time step

2. Activation Function Application:

ht=tanh() is used as activation function in RNN.

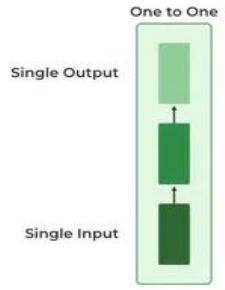
3. Output Calculation:
$y_t = W_{hy} \cdot h_t$

where yt is the output and *Why* is the weight at the output layer.

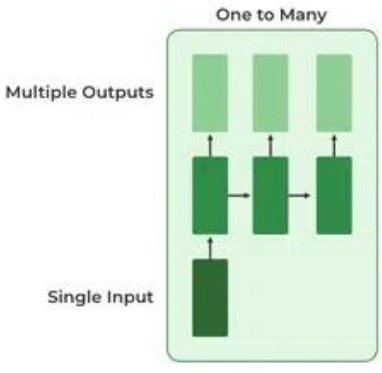
These parameters are updated using backpropagation. However, since RNN works on sequential data here we use an updated backpropagation which is known as **backpropagation through time**.

Types Of Recurrent Neural Networks
There are four types of RNNs based on the number of inputs and outputs in the network:

1. One-to-One RNN
This is the simplest type of neural network architecture where there is a single input and a single output. It is used for straightforward classification tasks such as binary classification where no sequential data is involved.



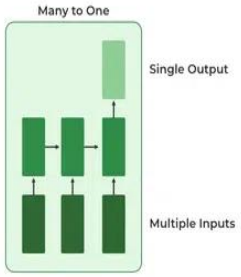
2.One-to-Many RNN
In a One-to-Many RNN the network processes a single input to produce multiple outputs over time. This is useful in tasks where one input triggers a sequence of predictions (outputs).
Ex: in image captioning a single image can be used as input to generate a sequence of words as a caption.



3. Many-to-One RNN

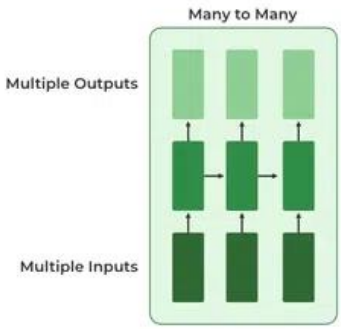
The Many-to-One RNN receives a sequence of inputs and generates a single output. This type is useful when the overall context of the input sequence is needed to make one prediction.

In sentiment analysis the model receives a sequence of words and produces a single output like positive, negative or neutral.



4. Many-to-Many RNN

The Many-to-Many RNN type processes a sequence of inputs and generates a sequence of outputs. In language translation task a sequence of words in one language is given as input and a corresponding sequence in another language is generated as output.



Limitations of Recurrent Neural Networks (RNNs)

While RNNs excel at handling sequential data they face two main training challenges i.e **vanishing gradient** and **exploding gradient** problem:

Vanishing Gradient: During backpropagation gradients diminish as they pass through each time step leading to minimal weight updates. This limits the RNN's ability to learn **long-term dependencies** which is crucial for tasks like language translation.

Exploding Gradient: Sometimes gradients grow uncontrollably causing excessively large weight updates that de-stabilize training.

These challenges reduce the performance of standard RNNs on complex, long-sequence tasks.

3. LSTM - Long Short Term Memory

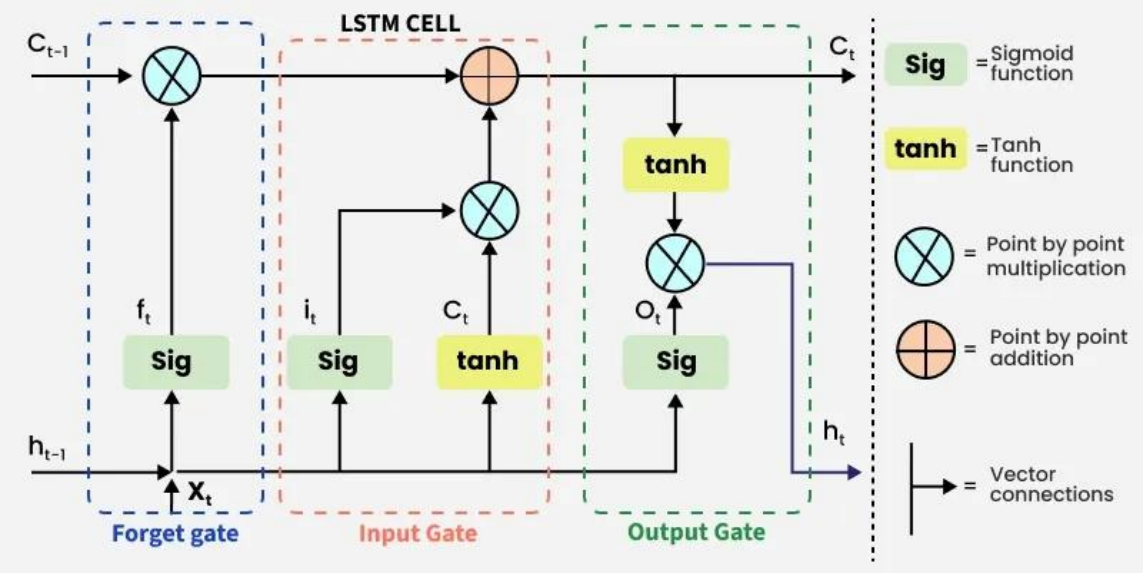
- Long Short-Term Memory (LSTM) is an enhanced version of the Recurrent Neural Network (RNN) designed by Hochreiter and Schmid Huber.
- LSTMs can capture long-term dependencies in sequential data making them ideal for tasks like language translation, speech recognition and time series forecasting.
- Unlike traditional RNNs which use a single hidden state passed through time LSTMs introduce a memory cell that holds information over extended periods addressing the challenge of learning long-term dependencies.

LSTM Architecture

LSTM architectures involve the memory cell which is controlled by three gates:

1. **Input gate**: Controls what information is added to the memory cell.
2. **Forget gate**: Determines what information is removed from the memory cell.
3. **Output gate**: Controls what information is output from the memory cell.

- This allows LSTM networks to selectively retain or discard information as it flows through the network which allows them to learn long-term dependencies.
- The network has a hidden state which is like its short-term memory.
- This memory is updated using the current input, the previous hidden state and the current state of the memory cell.



Working of LSTM

LSTM architecture has a chain structure that contains four neural networks and different memory blocks called cells.

Information is retained by the cells and the memory manipulations are done by the gates**.** There are three gates -

1. Forget Gate

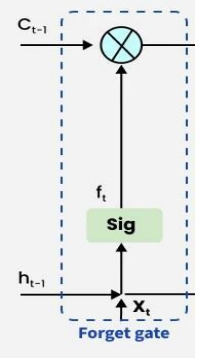
- The information that is no longer useful in the cell state is removed with the forget gate.
- Two inputs xt (input at the particular time) and ht−1 (previous cell output) are fed to the gate and multiplied with weight matrices followed by the addition of bias.
- The resultant is passed through sigmoid activation function which gives output in range of [0,1].

The equation for the forget gate is:

$$f_t=\sigma(W_f\cdot[h_{t-1},x_t]+b_f)$$

Where:

- Wf represents the weight matrix associated with the forget gate.
- [ht−1,xt] denotes the concatenation of the current input and the previous hidden state.
- Bf is the bias with the forget gate.
- σ is the sigmoid activation function.



2. Input gate

- The addition of useful information to the cell state is done by the input gate.
- First the information is regulated using the sigmoid function and filter the values to be remembered similar to the forget gate using inputs ht−1 and xt. Then, a vector is created using **tanh** function that gives an output from -1 to +1 which contains all the possible values from ht−1and xt.
- At last the values of the vector and the regulated values are multiplied to obtain the useful information. The equation for the input gate is:

$$i_t=\sigma(W_i\cdot[h_{t-1},x_t]+b_i)$$

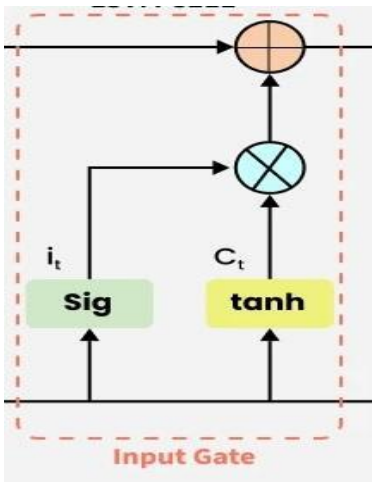
$$C\hat{}_t=\tanh(W_c\cdot[h_{t-1},x_t]+b_c)$$

We multiply the previous state by ft effectively filtering out the information we had decided to ignore earlier. Then we add it ⊙Ct which represents the new candidate values scaled by **how much we decided to update** each state value.

$$C_t=f_t\odot C_{t-1}+i_t\odot C\hat{}_t$$

where

- ⊙ denotes element-wise multiplication
- tanh is activation function



3. Output gate

- The output gate is responsible for deciding what part of the current cell state should be sent as the hidden state (output) for this time step.
- First, the gate uses a sigmoid function to determine which information from the current cell state will be output. This is done using the previous hidden state ht−1 and the current input xt:

$$o_t=\sigma(W_o\cdot[h_{t-1},x_t]+b_o)$$

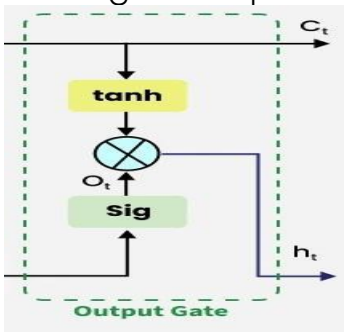
Next, the current cell state Ct is passed through a tanh activation to scale its values between −1and +1.
Finally, this transformed cell state is multiplied element-wise with ot to produce the hidden state ht:

$$h_t=o_t\odot\tanh(C_t)$$

Here:

- ot is the output gate activation.
- Ct is the current cell state.
- ⊙ represents element-wise multiplication.
- σ is the sigmoid activation function.

This hidden state ht is then passed to the next time step and can also be used for generating the output of the network.



Applications of LSTM

Some of the famous applications of LSTM includes:

Language Modeling: Used in tasks like language modeling, machine translation and text summarization. These networks learn the dependencies between words in a sentence to generate coherent and grammatically correct sentences.

Speech Recognition: Used in transcribing speech to text and recognizing spoken commands. By learning speech patterns they can match spoken words to corresponding text.

Time Series Forecasting: Used for predicting stock prices, weather and energy consumption. They learn patterns in time series data to predict future events.

Anomaly Detection: Used for detecting fraud or network intrusions. These networks can identify patterns in data that deviate drastically and flag them as potential anomalies.

Recommender Systems: In recommendation tasks like suggesting movies, music and books. They learn user behavior patterns to provide personalized suggestions.

Video Analysis: Applied in tasks such as object detection, activity recognition and action classification. When combined with Convolutional Neural Networks (CNNs) they help analyze video data and extract useful information.

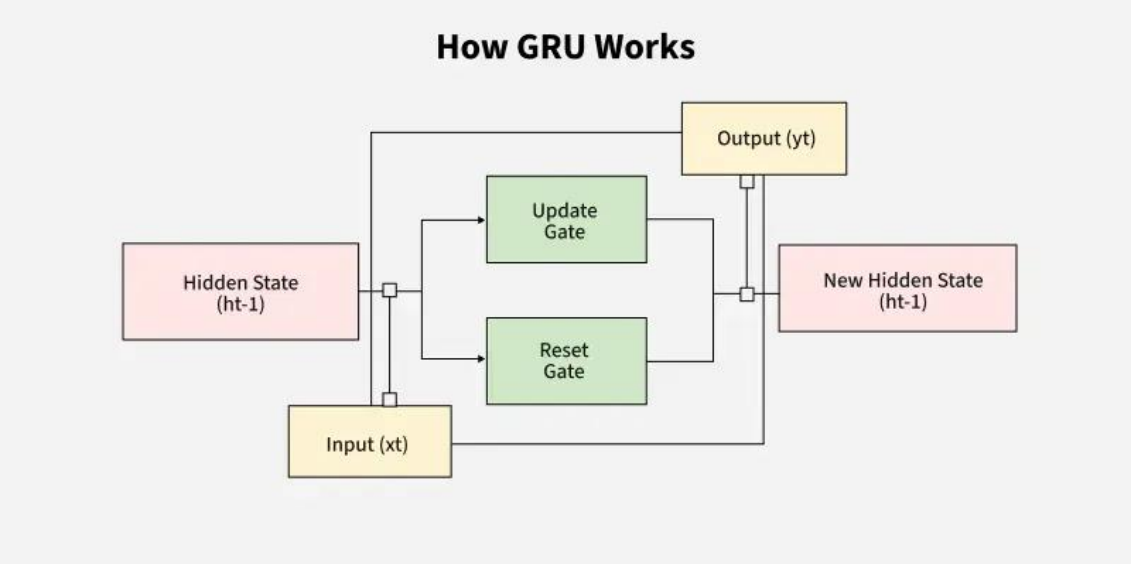
4. Gated Recurrent Unit Networks

- In machine learning, Recurrent Neural Networks (RNNs) are essential for tasks involving sequential data such as text, speech and time-series analysis.
- While traditional RNNs struggle with capturing long-term dependencies due to the **vanishing gradient problem** architectures like Long Short-Term Memory (LSTM) networks were developed to overcome this limitation.
- However LSTMs are very complex structure with higher computational cost. To overcome this **Gated Recurrent Unit (GRU)** where introduced which uses LSTM architecture by merging its gating mechanisms offering a more efficient solution for many sequential tasks without sacrificing performance.

Gated Recurrent Units (GRU):

Gated Recurrent Units (GRUs) are a type of RNN introduced by Cho et al. in 2014. The core idea behind GRUs is to use **gating mechanisms** to selectively update the hidden state at each time step allowing them to remember important information while discarding irrelevant details.

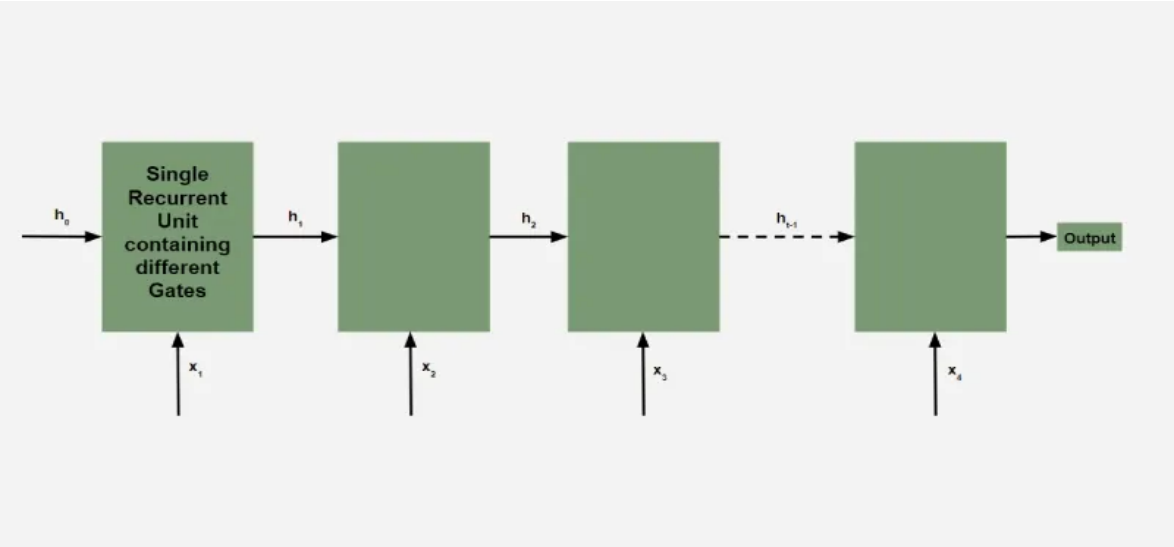
GRUs aim to simplify the LSTM architecture by merging some of its components and focusing on just two main gates: the **update gate** and the **reset gate**.



The GRU consists of **two main gates**:

1. **Update Gate** (zt): This gate decides how much information from previous hidden state should be retained for the next time step.
2. **Reset Gate** (rt): This gate determines how much of the past hidden state should be forgotten.

These gates allow GRU to control the flow of information in a more efficient manner compared to traditional RNNs which solely rely on hidden state.



Equations for GRU Operations

The internal workings of a GRU can be described using following equations:

1. Reset gate:

$rt=\sigma(Wr\cdot[ht-1,xt])$

The reset gate determines how much of the previous hidden state $ht-1$ should be forgotten.

2. Update gate:

$zt=\sigma(Wz\cdot[ht-1,xt])$

The update gate controls how much of the new information xt should be used to update the hidden state.

3. Candidate hidden state:

$$h_t' = \tanh(W_h \cdot [r_t \cdot h_{t-1}, x_t])$$

This is the potential new hidden state calculated based on the current input and the previous hidden state.

4. Hidden state:

$h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot h_t'$

The final hidden state is a weighted average of the previous hidden state h_{t-1} and the candidate hidden state h_t' based on the update gate z_t.

How GRUs Solve the Vanishing Gradient Problem

Like LSTMs, GRUs were designed to address the **vanishing gradient problem** which is common in traditional RNNs.

GRUs help mitigate this issue by using gates that regulate the flow of gradients during training ensuring that important information is preserved and that gradients do not shrink excessively over time.

By using these gates, GRUs maintain a balance between remembering important past information and learning new, relevant data.

Differences between LSTM and GRU

Feature	**LSTM (Long Short-Term Memory)**	**GRU (Gated Recurrent Unit)**
Gates	3 (Input, Forget, Output)	2 (Update, Reset)
Cell State	Yes it has cell state	No (Hidden state only)
Training Speed	Slower due to complexity	Faster due to simpler architecture
Computational Load	Higher due to more gates and parameters	Lower due to fewer gates and parameters
Performance	Often better in tasks requiring long-term memory	Performs similarly in many tasks with less complexity

5.POS Tagging using Bi-LSTM

- A Bi-directional Long Short-Term Memory (Bi-LSTM) network is a type of recurrent neural network (RNN) that is highly effective for Part-of-Speech (POS) tagging.
- Unlike a traditional LSTM that processes data in one direction, a Bi-LSTM processes a sequence in both a forward and a backward direction simultaneously. This allows the model to capture context from both the past and future, which is crucial for determining a word's part of speech.
- For instance, to tag the word "saw" in the sentence "I saw a movie," a Bi-LSTM processes the sequence from "I" to "movie" (forward) and from "movie" to "I" (backward).
- The forward pass provides context from the beginning of the sentence, while the backward pass gives context from the end, allowing the network to understand that "saw" is a verb.

Architecture for POS tagging

A Bi-LSTM-based POS tagging model typically has the following layered architecture:

1. **Input layer**
2. **Embedding layer**
3. **Bi-LSTM layer**
4. **Dense/Fully connected layer**
5. **Output layer**

1. **Input layer:** A sequence of words (tokens) from a sentence is fed into the network.
2. **Embedding layer:** Each word is converted into a dense vector representation. These word embeddings capture the semantic and syntactic properties of words. Pre-trained embeddings like GloVe can be used to improve performance.
3. **Bi-LSTM layer: The** word embeddings are passed through the Bi-LSTM layer, which consists of two separate LSTM networks:
    - **Forward LSTM:** Processes the input sequence from left to right (e.g., *the, fast, car, is, red*).
    - **Backward LSTM:** Processes the input sequence from right to left (e.g., *red, is, car, fast, the*).
4. **Dense/Fully connected layer:** The final hidden state from the Bi-LSTM is passed to a dense layer, which maps the high-dimensional representation to the number of possible POS tags.
5. **Output layer:** A softmax function is applied to the output of the dense layer to produce a probability distribution over the possible POS tags for each word.

Enhancing performance with a CRF layer

While a standard Bi-LSTM model can achieve high accuracy, a more advanced and powerful approach is to add a Conditional Random Field (CRF) layer on top of the Bi-LSTM.

Example:

Problem: The softmax function predicts each word's tag independently, without considering the tags of surrounding words. For example, a softmax-only model might predict a sequence like "noun-verb-adjective," which is often grammatically incorrect.

Solution: A CRF layer models the dependencies between consecutive output tags. It learns constraints from the training data that ensure the sequence of predicted tags is grammatically valid.

Common constraints learned by a CRF layer include:

- A sentence should not begin with a verb.
- A determiner (e.g., "the") should be followed by a noun or adjective.

By using a Bi-LSTM-CRF model, the network leverages the Bi-LSTM's ability to capture long-range dependencies and the CRF's ability to enforce valid tag sequences, resulting in state-of-the-art performance for sequence labelling tasks like POS tagging.

Advantages of Bi-LSTMs for POS tagging

- **Captures bidirectional context:** Considers both preceding and succeeding words to resolve ambiguity and make more accurate tag predictions.
- **Handles long-term dependencies:** LSTMs overcome the vanishing gradient problem of traditional RNNs, allowing them to remember information over long sequences.
- **No handcrafted feature engineering:** Deep learning models automatically learn effective features from the data, eliminating the need for manual feature extraction.
- **Performance with unknown words:** Can utilize character-level embeddings to predict tags for out-of-vocabulary (OOV) words, common in languages with complex morphology

6.NER using Bi-LSTM

- For Named Entity Recognition (NER), a Bi-directional Long Short-Term Memory (Bi-LSTM) network is an exceptionally powerful architecture.
- NER is a sequence-labelling task that identifies and classifies proper nouns in text, such as people, organizations, and locations.
- A Bi-LSTM is ideal because it can capture the complete context of a word within a sentence by processing the text in both forward and backward directions.

Bi-LSTM working for NER

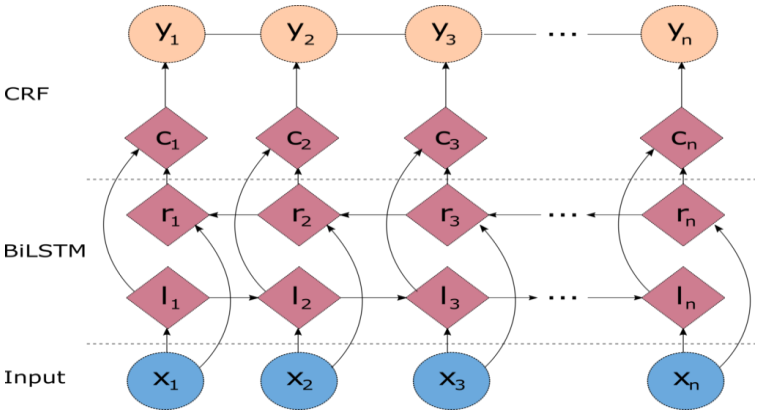
The architecture of a Bi-LSTM model for NER is built to process a sequence of words and produce a corresponding sequence of entity tags.

Architecture overview
It contains the following layers:

1. **Embedding layer**
2. **Bi-LSTM layer:**
3. **Time Distributed Dense layer**
4. **Conditional Random Field**

1. **Embedding layer:** Converts each word in the input sentence into a dense vector representation. These vectors can be learned during training or initialized with pre-trained word embeddings like GloVe or FastText to capture semantic information.
2. **Bi-LSTM layer:** Processes the word embeddings. This layer is composed of two separate LSTMs:
   - **Forward LSTM:** Reads the input sequence from left to right.
   - **Backward LSTM:** Reads the input sequence from right to left. The outputs of the two LSTMs are concatenated at each time step, giving the model a rich representation that incorporates both past and future context.
3. **Time Distributed Dense layer:** Applies a fully connected (Dense) layer to every time step of the Bi-LSTM's output. This prepares the output for the final classification.
4. **Conditional Random Field (CRF) layer (optional, but highly recommended):** For complex sequence labelling tasks like NER, a CRF layer is typically added on top of the Bi-LSTM.
   A CRF layer considers the dependencies between output tags, enforcing constraints to produce a globally optimal sequence of tags.

The combination of CRF and BiLSTM is often referred to as a BiLSTM-CRF model and its architecture is shown in Figure:



Building a statistical named entity Recognition (NER) model
The IOB format

- Inside–outside–beginning (IOB) is a common format for tagging entities in computer linguistics, especially in a NER context.
- This scheme was initially proposed by Ramshaw and Marcus (1995), and the meaning of the IOB tags is as follows:
  - ✓The I-prefix indicates that the tag is inside a chunk (i.e. a noun group, a verb group etc.)
  - ✓The O-prefix indicates that the token belongs to no chunk
  - ✓The B-prefix indicates that the tag is at the beginning of a chunk that follows another chunk without O tags between the two chunks.

The entity tags used in the sample dataset are as follows:

Tag	Meaning	Example
geo	Geography	Indian
org	Organisation	GOOGLE
per	Person	MK Gandhi
gpe	Geopolitical Entity	India
tim	Time	Wednesday
art	Artifact	Pentastar
eve	Event	Selesta
nat	Natural Phenomenon	H5N1

the application of IOB on the class labels:

Example workflow
Imagine a Bi-LSTM-CRF model processing the sentence: "Elon Musk is the CEO of Tesla."

- **Tokenization:** The sentence is broken down into words: [Elon, Musk, is, the, CEO, of, Tesla].
- **Embedding:** Each word is converted into a vector. [Vector(Elon), Vector(Musk), ...].
- **Bi-LSTM Processing:**
  - The forward LSTM reads the sentence from "Elon" to "Tesla" and creates a sequence of hidden states.
  - The backward LSTM reads from "Tesla" to "Elon" and creates another sequence of hidden states.
  - For the word "Musk," the model uses the combined context from "Elon" (previous) and "is" (future) to better understand its role.
- **CRF Output:** The CRF layer refines the Bi-LSTM's output by applying learned constraints. For example, it knows that a person's first name ("Elon") should be followed by another person's name ("Musk"), ensuring a valid tag sequence like [B-PER, I-PER] instead of an error like [B-PER, O]

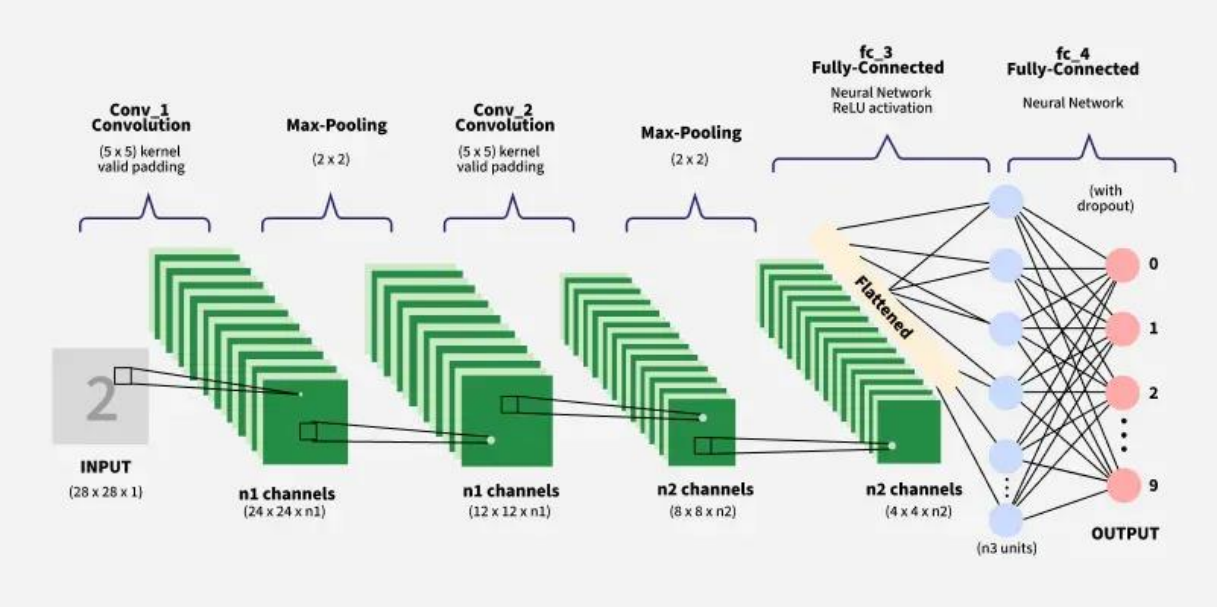
7.Text Classification using CNN

- In text classification, a Convolutional Neural Network (CNN) is adapted from its original use in computer vision to analyze text by treating sentences or documents as one-dimensional data.
- Instead of finding spatial features in images, a CNN for text classification learns to recognize local patterns like n-grams (word sequences) that are important for classification, such as identifying phrases that indicate sentiment.
- It transforms human-readable text into numerical representations that machine learning algorithms can process. There are several preprocessing steps that significantly impact model performance.

Architecture of CNNs for text classification

The architecture for text classification with a CNN typically involves several key layers.

1. **Embedding layer/Input Layer (Converts words to dense vector representations):** The raw text input, tokenized into a sequence of word IDs, is fed into an embedding layer. This layer converts each word ID into a dense vector (a word embedding), which can capture the semantic and syntactic properties of words. Pre-trained embeddings like Word2Vec or GloVe can be used, or the embeddings can be learned from scratch during training.
2. **Convolutional layer (Apply filters to detect local text patterns):** The embedding vectors form an input matrix, and one-dimensional (1D) filters are applied to this matrix. Each filter slides over a "window" of words (e.g., 3, 4, or 5 words at a time) to perform a convolution operation. The filters are designed to capture local features, such as specific phrases or keyword combinations, that are relevant to the classification task.
3. **Pooling layer (Reduce dimensionality while preserving important features):** After convolution, a pooling layer is used to downsample the feature maps. Max pooling is a common choice, which extracts the maximum value from each filter's output. This process keeps the most important features detected by the filters while reducing the dimensionality of the data and creating a fixed-size output.
4. **Fully connected layer (Combine features for final classification):** The output from the pooling layer is flattened into a single vector and passed to one or more fully connected (dense) layers. These layers use the extracted high-level features to make a classification decision.
5. **Output layer (Produces probability distributions over target classes):** The final layer uses an activation function, such as sigmoid for binary classification or softmax for multi-class classification, to output a probability distribution over the possible classes.



- The embedding layer serves as the foundation, transforming discrete word tokens into continuous vector space where semantic relationships can be captured.
- These embeddings can be randomly initialized or pre-trained using methods like Word2Vec or GloVe.
- Convolutional layers then apply multiple filters of varying sizes (typically 3, 4 and 5 words) to capture different n-gram patterns.
- Each filter learns to detect specific linguistic patterns that are relevant for the classification task.

Filter size considerations:

- **Size 3**: Captures trigrams and short phrases
- **Size 4**: Detects longer phrase patterns
- **Size 5**: Identifies extended expressions and longer dependencies
- **Multiple sizes**: Provides comprehensive pattern coverage

Example application

- Sentiment analysis is a classic text classification task where CNNs are often used.
- A CNN model can learn to recognize how the local arrangement of words and phrases signals a positive, negative, or neutral sentiment.
- For instance, a filter might detect the phrase "terrible movie" as a negative indicator, regardless of where it appears in a review.

Types of CNN in NLP

1. 1D CNNs

1D CNNs, are a subtype of CNN created specifically to process 1D data sequences, like text.

2. 2D CNNs

Convolutional Neural Networks, or 2D CNNs, are a subtype of CNN that work with 2D data like images.

3. Temporal CNNs

Time-series data or natural language are examples of sequential data with temporal dependencies that can be processed using Temporal CNNs (Convolutional Neural Networks), a variant of 1D CNNs.

4. Dynamic CNNs

Convolutional neural networks (CNNs) that are dynamically trained can handle input sequences with variable lengths, like text.

Performance Analysis

Understanding CNN performance requires monitoring key metrics:

- **Accuracy**: Overall correctness across all classes
- **Precision**: Proportion of positive predictions that are actually positive
- **Recall**: Proportion of actual positive cases correctly identified
- **F1-Score**: Harmonic mean of precision and recall

Typical CNN performance on text classification tasks achieves 85-95% accuracy on well-defined problems like sentiment analysis, depending on dataset quality and model architecture complexity.

Real-World Applications

CNN-based text classification has found success across numerous industries:

- **E-commerce**: Product categorization, review sentiment analysis
- **Healthcare**: Medical document classification, symptom analysis
- **Finance**: Fraud detection, risk assessment, compliance monitoring
- **Media**: Content moderation, news categorization
- **Customer Service**: Ticket classification, automated routing

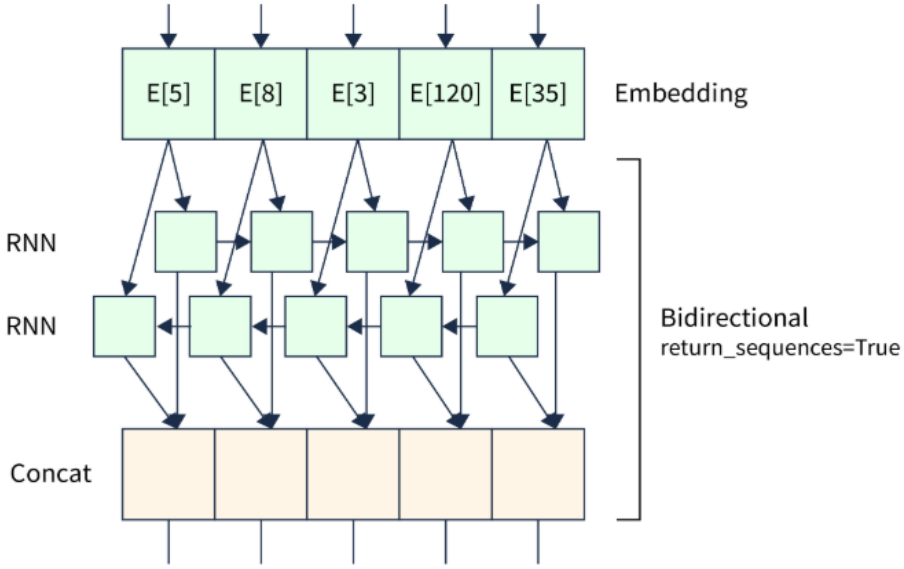
Best Practices:

- **Use dropout layers (0.2–0.5)**: Randomly dropping connections during training reduces overfitting and helps the network to learn more robust features.
- **Apply L2 regularization**: Adds a penalty to the loss function for large weights in dense layers, promoting simpler models that generalize better.
- **Implement early stopping**: Stops training when validation loss stops improving, preventing unnecessary epochs and reducing overfitting risk.
- **Employ multiple filter sizes**: Using different kernel sizes in convolutional layers captures patterns of varying lengths (e.g., bi-grams, tri-grams), improving feature extraction.

8. Text Classification with RNN

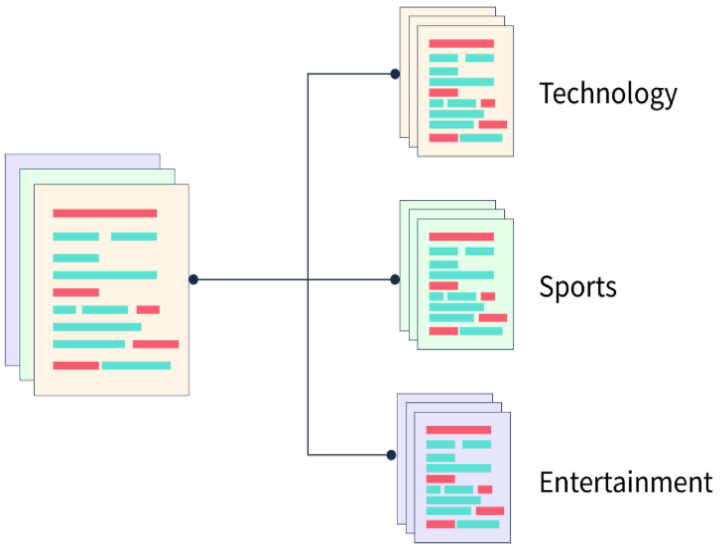
- RNN for Text classification in natural language processing (NLP) tasks involves categorizing text content according to predetermined categories.
- Recurrent neural networks (RNNs) are a powerful method for tackling text classification issues.
- RNNs for text classification are particularly effective in modeling sequential data, considering word relationships and context.

- They are suited for text analysis due to their loop-based loops, allowing information to remain over time.
- The technique of automatically classifying or categorizing text documents is referred to as text classification or text categorization.
- RNN for text classification has several applications and is a critical issue with natural language processing (NLP).
- By effectively categorizing text, we may extract insights, comprehend sentiment, identify themes, detect spam, and facilitate effective information retrieval.



To organize, retrieve, and analyze textual material effectively, text classification aims to automatically analyze and categorize text documents based on their content.

By giving the text the right labels or categories, we may extract useful insights, make it easier to find information, automate decision-making, and allow a variety of downstream applications.



Architecture for text classification using RNNs

A typical RNN-based model for text classification follows this architecture:

1. **Input Layer**
2. **Embedding Layer**
3. **Recurrent Layer**
4. **Pooling/Output Layer**
5. **Activation Layer**

1. **Input Layer:** Takes a sequence of pre-processed text (words).
2. **Embedding Layer:** Converts each word into a dense vector representation (word embedding). These embeddings can be learned during training or initialized with pre-trained vectors like GloVe.
3. **Recurrent Layer:** An LSTM or GRU layer processes the sequence of word embeddings. This layer retains information across time steps to model the dependencies within the text.
4. **Pooling/Output Layer**: The final output of the RNN is passed to a dense layer for classification. In a "many-to-one" architecture, the output of the final recurrent step is used as the input to the classification layer.
5. **Activation Layer**: An activation function like sigmoid for binary classification or softmax for multi-class classification is applied to produce the final classification probability

Example:

Text Classification with RNN using TensorFlow

Natural language processing (NLP) is frequently used for text classification, which is categorizing or labelling text content according to predetermined standards.

Recurrent Neural Networks (RNN for text classification) have been demonstrated to be effective models for encapsulating the sequential nature and contextual relationships contained in text data.

The stages we'll go through for utilizing RNNs and TensorFlow to classify text are as follows:

a. Getting the Text Data Ready:

- The text data and associated labels should be loaded.
- Remove punctuation from data and perform preprocessing operations like tokenization and lowercasing.
- Create training and test sets from the data.

b. Creating an RNN Text Classification Model:

- Pick a suitable RNN architecture, such as LSTM or GRU.
- Utilise Keras, the high-level API for TensorFlow, to specify the network design.
- Indicate the model's input and output dimensions.
- Set the model's activation, loss, and optimizer functions appropriately.

c. RNN Model Training and Evaluation:

- Feed the RNN model with the preprocessed data.

- Set the training settings, such as the batch size and epoch count.
- Utilise validation data to keep track of the model's performance throughout training.
- Using gradient descent and backpropagation, modify the model's weights.
- Utilise the testing dataset to evaluate the trained model's performance.
- Calculate evaluation measures such as F1 score, recall, accuracy, and precision.
- Examine the predictions made by the model and note any shortcomings.

Python program to implement above process for text classification:

```
import tensorflow as tf
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout

# --- 1. Data Preparation ---
# Load the IMDB dataset, which comes pre-tokenized.
# `num_words` keeps only the top N most frequent words.
vocab_size = 10000
(X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=vocab_size)

# Pad sequences to a consistent length for the model input.
# Reviews shorter than `max_review_length` will be padded with zeros.
max_review_length = 500
X_train = pad_sequences(X_train, maxlen=max_review_length)
X_test = pad_sequences(X_test, maxlen=max_review_length)

print("Shape of X_train:", X_train.shape)
print("Shape of y_train:", y_train.shape)

# --- 2. Build the LSTM Model ---
embedding_vector_length = 128

model = Sequential()

# Embedding layer: Converts word indices to dense vectors.
model.add(Embedding(vocab_size, embedding_vector_length,
input_length=max_review_length))

# LSTM layer: Processes the sequence of embeddings and captures long-
term dependencies.
model.add(LSTM(128))
```

```
# Dense layers for classification.
model.add(Dense(64, activation='relu'))
model.add(Dropout(0.5)) # Dropout for regularization to prevent overfitting
model.add(Dense(1, activation='sigmoid')) # Output layer for binary classification

# Compile the model
model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

# Print a summary of the model's architecture
print(model.summary())

# --- 3. Train the Model ---
# Train the model on the training data.
history = model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.2, # Use a portion of the training data for validation
    verbose=1
)

# --- 4. Evaluate the Model ---
# Evaluate the model's performance on the unseen test data.
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print(f"\nTest Loss: {loss:.4f}")
print(f"Test Accuracy: {accuracy:.4f}")

# --- 5. Make Predictions on New Data ---
# Define a function to preprocess new, unseen text for prediction.
def predict_sentiment(text):
    # Tokenize and pad the new text
    tokenizer = tf.keras.preprocessing.text.Tokenizer(num_words=vocab_size)
    tokenizer.fit_on_texts([text])
    sequence = tokenizer.texts_to_sequences([text])
    padded_sequence = pad_sequences(sequence, maxlen=max_review_length)

    # Make the prediction
    prediction = model.predict(padded_sequence)
    sentiment = "Positive" if prediction[0][0] > 0.5 else "Negative"

    print(f"\nReview: '{text}'")
```

print(f"Predicted sentiment: {sentiment} (Confidence: {prediction[0][0]:.4f})")

# Example predictions
predict_sentiment("This movie was absolutely fantastic and a thrilling ride from start to finish.")
predict_sentiment("The plot was dull and the acting was terrible, I would not recommend this movie.")

output:

```
D:\SVCET\2025-26\ODD SEMS\NLP\LAB PROGRAMS>py textclassificationrnn.py
2025-09-05 20:10:20.685583: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-09-05 20:10:34.962470: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/imdb.npz
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 13s 1us/step
Shape of X_train: (25000, 500)
Shape of y_train: (25000,)
C:\Users\ssits\AppData\Local\Programs\Python\Python312\Lib\site-packages\keras\src\layers\core\embedding.py:97: UserWarning: Argument `input_length` is deprecated. Just remove it.
  warnings.warn(
2025-09-05 20:11:12.279999: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Model: "sequential"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
lstm (LSTM)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)
dropout (Dropout)	?	0
dense_1 (Dense)	?	0 (unbuilt)

```
 Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)
None
```

```
Epoch 1/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 810s 3s/step - accuracy: 0.7725 - loss: 0.4687 - val_accuracy: 0.8616 - val_loss: 0.3362
Epoch 2/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 458s 1s/step - accuracy: 0.8881 - loss: 0.2890 - val_accuracy: 0.8552 - val_loss: 0.3578
Epoch 3/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 426s 1s/step - accuracy: 0.9129 - loss: 0.2275 - val_accuracy: 0.8470 - val_loss: 0.3511
Epoch 4/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 418s 1s/step - accuracy: 0.9450 - loss: 0.1498 - val_accuracy: 0.8500 - val_loss: 0.4227
Epoch 5/5
313/313 ━━━━━━━━━━━━━━━━━━━━ 421s 1s/step - accuracy: 0.9663 - loss: 0.0999 - val_accuracy: 0.8498 - val_loss: 0.4450

Test Loss: 0.4431
Test Accuracy: 0.8518
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 417ms/step

Review: 'This movie was absolutely fantastic and a thrilling ride from start to finish.'
Predicted sentiment: Positive (Confidence: 0.8616)
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 112ms/step

Review: 'The plot was dull and the acting was terrible, I would not recommend this movie.'
Predicted sentiment: Positive (Confidence: 0.8498)
```

Advantages of RNNs for text classification

- **Contextual Understanding:** RNNs excel at capturing the sequential nature of language, allowing them to understand how words and phrases relate to one another in a document.
- **Variable-Length Inputs:** RNNs can naturally handle texts of varying lengths, making them well-suited for a wide range of text classification tasks.
- **Automatic Feature Extraction**: RNNs automatically learn to identify the most relevant features for classification from the raw text, eliminating the need for manual feature engineering.

9.Model Evaluation and Hyperparameter Tuning for text classification using CNN and RNN

In text classification using deep learning models like Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), both model evaluation and hyperparameter tuning are critical for developing robust and accurate classifiers

.

Model evaluation

Evaluating the performance of text classification models goes beyond simple accuracy. Choosing the right metrics depends on the specific goals of the classification task.

Key metrics

- **Accuracy:** Measures the overall correctness of the model's predictions (ratio of correctly predicted instances to the total instances). However, accuracy might be misleading in cases of imbalanced datasets where one class is significantly more prevalent than others.
- **Precision:** Represents the proportion of true positive predictions among all positive predictions made by the model. High precision is crucial when the cost of false positives is high (e.g., classifying a legitimate email as spam).
- **Recall:** Measures the proportion of true positive predictions among all actual positive instances. High recall is important when the cost of false negatives is high (e.g., failing to identify a critical safety report).
- **F1-score**: The harmonic mean of precision and recall. It balances both metrics and is particularly useful when dealing with imbalanced classes, according to ApX Machine Learning. It's preferable to accuracy for imbalanced datasets.
- **Confusion matrix:** A table that summarizes the predictions made by the model versus the actual labels. It shows true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN), providing a detailed view of the model's performance on each class.

Hyperparameter tuning

Hyperparameters are settings that are not learned by the model during training but are set before training begins. Tuning these parameters is crucial for achieving optimal model performance.

Hyperparameters specific to CNNs and RNNs

- **CNNs:**
  - **Filters/kernel size:** Define the number and size of convolutional filters, impacting the local patterns the model can detect.
  - **Pooling layer parameters:** Control how features are downsampled after convolution.
  - **Learning rate and optimizer:** Affect the training process's speed and stability.
- **RNNs (LSTMs/GRUs):**

- **Number of hidden units:** Determines the capacity of the recurrent layers to learn complex patterns.
- **Number of layers:** Affects the depth of the network and its ability to capture hierarchical dependencies.
- **Dropout rate**: Controls the amount of regularization applied to prevent overfitting.
- **Learning rate, optimizer, and batch size**: Similar to CNNs, these affect training efficiency and stability.

Tuning techniques

Models can have many hyperparameters and finding the best combination of parameters can be treated as a search problem.

The two best strategies for Hyperparameter tuning are:

1. GridSearchCV : Explores a predefined set of hyperparameter values by training and evaluating a model for every possible combination. It is thorough but computationally expensive, especially with a large number of hyperparameters or values.

2. RandomizedSearchCV : Samples a fixed number of hyperparameter combinations from specified ranges or distributions. This method is often more efficient than grid search, particularly when only a few hyperparameters significantly impact performance. However, there is an element of chance, as it may not find the absolute best combination.

GridSearchCV

GridSearchCV is a brute-force technique for hyperparameter tuning. It trains the model using all possible combinations of specified hyperparameter values to find the best-performing setup. It is slow and uses a lot of computer power which makes it hard to use with big datasets or many settings. It works using below steps:

- Create a grid of potential values for each hyperparameter.
- Train the model for every combination in the grid.
- Evaluate each model using cross-validation.
- Select the combination that gives the highest score.

For example if we want to tune two hyperparameters C and Alpha for a Logistic Regression Classifier model with the following sets of values:

C = [0.1, 0.2, 0.3, 0.4, 0.5]
Alpha = [0.01, 0.1, 0.5, 1.0]

C \ Alpha	0.1	0.2	0.3	0.4
0.5	0.701	0.703	0.697	0.696
0.4	0.699	0.702	0.698	0.702
0.3	0.721	0.726	0.713	0.703
0.2	0.706	0.705	0.704	0.701
0.1	0.698	0.692	0.688	0.675

The grid search technique will construct multiple versions of the model with all possible combinations of C and Alpha, resulting in a total of 5 * 4 = 20 different models. The best-performing combination is then chosen.

2. RandomizedSearchCV

As the name suggests **RandomizedSearchCV** picks random combinations of hyperparameters from the given ranges instead of checking every single combination like GridSearchCV.

- In each iteration it **tries a new random combination** of hyperparameter values.
- It **records the model's performance** for each combination.
- After several attempts it **selects the best-performing set**.

Other common methods:

- **Bayesian optimization**: A more advanced technique that treats hyperparameter tuning as a mathematical optimization problem. It uses a probabilistic model (surrogate function) to predict performance based on hyperparameters and learns from past results to decide which values to try next. This approach is more efficient than grid search and random search, particularly for complex models with many hyperparameters.

Common surrogate models used in Bayesian optimization include:

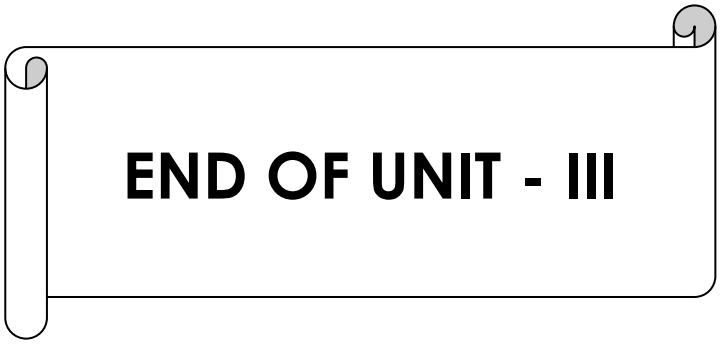
- Gaussian Processes
- Random Forest Regression
- Tree-structured Parzen Estimators (TPE)

- **Cross-validation integration**: It is essential to perform hyperparameter tuning using cross-validation to get a reliable estimate of model performance and avoid overfitting to a specific train-test split.

By carefully evaluating the model and tuning its hyperparameters, practitioners can develop accurate and robust text classification systems for various real-world applications.

Advantages of Hyperparameter tuning

- **Improved Model Performance**: Finding the optimal combination of hyperparameters can significantly boost model accuracy and robustness.
- **Reduced Overfitting and Underfitting**: Tuning helps to prevent both overfitting and underfitting resulting in a well-balanced model.
- **Enhanced Model Generalizability**: By selecting hyperparameters that optimize performance on validation data the model is more likely to generalize well to unseen data.
- **Optimized Resource Utilization**: With careful tuning resources such as computation time and memory can be used more efficiently avoiding unnecessary work.
- **Improved Model Interpretability**: Properly tuned hyperparameters can make the model simpler and easier to interpret.



UNIT IV
Syllabus

Transformers and Advanced NLP
Attention Mechanism and Self-Attention, Transformer Architecture: Encoder-Decoder Models, Pretrained Language Models: BERT, RoBERTa, GPT, Fine-tuning Transformers for Text Classification, Question Answering and Text Summarization using Transformers, Sentiment Analysis and Zero-shot Classification.
.

STUDY MATERIAL

Prepared by **K.ANJANEYULU, M. Tech (CSE), MBA, PGDCA, M. Sc, B. Ed.,**
Assistant Professor,
Department of Computer Science & Engineering (AI&ML),
Sri Venkateswara College of Engineering & Technology,
RVS Nagar. Chittoor.
Phno: **(+91) – 9640985702, (+91) – 9398526101**
E-mail: **anji.karapakula@gmail.com**

1. Encoder - Decoder Models

- In deep learning, the encoder-decoder model is a type of neural network that is mainly used for tasks where both the input and output are sequences.
- This architecture is used when the input and output sequences are not the same length.
- **Example:** translating a sentence from one language to another, summarizing a paragraph, describing an image with a caption or convert speech into text.

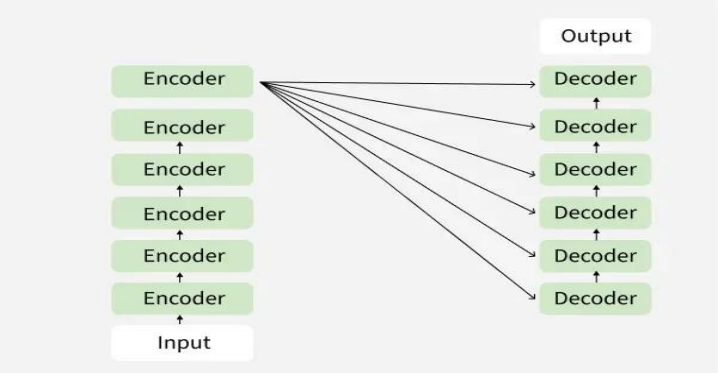
It works in two stages:

- **Encoder:** The encoder takes the input data like a sentence and processes each word one by one then creates a single, fixed-size summary of the entire input called a context vector or latent space.
- **Decoder:** The decoder takes the context vector and begins to produce the output one step at a time.

For example, in machine translation an encoder-decoder model might take an English sentence as input (like "I am learning AI") and translate it into French ("Je suis en train d'apprendre l'IA").

Encoder-Decoder Model Architecture

In an encoder-decoder model both the encoder and decoder are separate networks each one has its own specific task. These networks can be different types such as Recurrent Neural Networks (RNNs), Long Short-Term Memory networks (LSTMs), Gated Recurrent Units (GRUs), Convolutional Neural Networks (CNNs) or even more advanced models like Transformers.



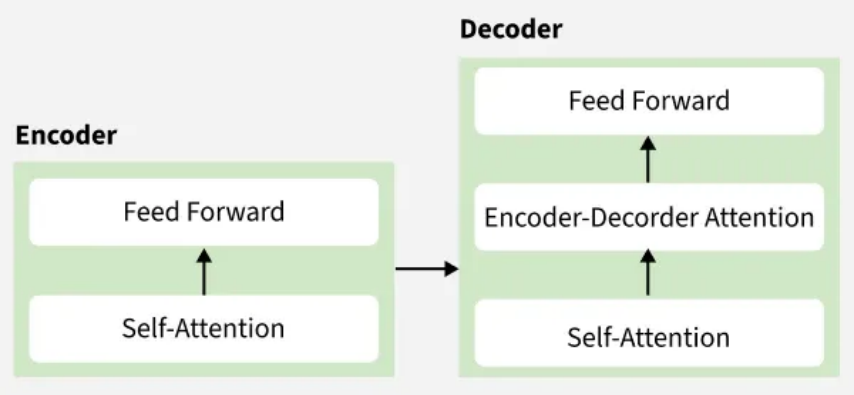
Encoder:

The encoder's job is to process the input data and convert it into a form that the model can understand. It does this using two main steps:

- **Self-Attention Layer:** This layer helps the encoder focus on different parts of the input data that are important for understanding the context.

For example in a sentence it allows the model to consider **how each word relates to the others**.

- **Feed-Forward Neural Network:** After the self-attention layer this network processes the information further to capture complex patterns and relationships in the data.



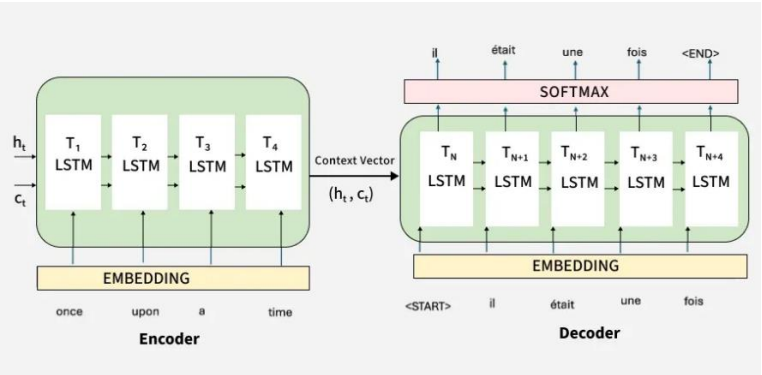
Decoder:

The decoder takes the processed information from the encoder and generates the output. It also has three main components:

- **Self-Attention Layer:** Similar to the encoder this layer allows the decoder to focus on different parts of the output it has generated.
- **Encoder-Decoder Attention Layer:** This unique layer enables the decoder to focus on relevant parts of the input data help to generate more accurate outputs.
- **Feed-Forward Neural Network:** Like the encoder the decoder uses this network to process the information and generate the final output.

Working of Encoder Decoder Model:

The actual working of the encoder decoder model is shown in below diagram. Now we will understand it stepwise:



Step 1: Tokenizing the Input Sentence

- The sentence "I am learning AI" is first broken into tokens: **["I", "am", "learning", "AI"]**.
- Each word (token) is converted into a **vector** that a machine can understand. This process is called **embedding**.

Step 2: Encoding the Input

- The **Encoder** processes these embeddings using **self-attention**.
- **Self-attention** helps the encoder to focus on **important words**.
- For example, while encoding "learning", it understands its relation with "I" and "AI."

- After processing the encoder generates a **Context Vector** which captures the meaning of the entire sentence. For example, in the image the arrows show how each word relates to the others during encoding. The final output from the encoder is the **context representation**

Step 3: Passing the Context to the Decoder

- The **Context Vector** is passed to the Decoder as shown in image.
- It acts like a summary of the full input sentence.

Step 4: Decoder Generates Output Step-by-Step

- The **Decoder** uses the context and starts creating the output one word at a time.
- First it predicts the first word then uses that to predict the second word and so on

Step 5: Decoder Attention

- While generating each word the decoder **attends** to different parts of the input sentence to make better predictions.
- For example, when translating "learning," it might pay more attention to the word "learning" in the input.

Step 6: Producing the Final Output

- The decoder continues generating until the full translated sentence is produced.
- Each output token depends on the previous ones and the input context.

2. Pretrained Language Models: BERT

- **Pretrained models** are deep learning models that have been trained on huge amounts of data before fine-tuning for a specific task.
- The pre-trained models have revolutionized the landscape of **natural language processing** as they allow the developer to transfer the learned knowledge to specific tasks, even if the tasks differ from the original training data.
- Some of the pre-trained models that are the driving force behind smart **NLP-based** the **AI** models are ChatGPT, Gemini, Bard and more.
- A pre-trained model, having been trained on extensive data, serves as a foundational model for various tasks, leveraging its learned patterns and features.
- In natural language processing (NLP), these models are commonly employed as a starting point for tasks like language translation, sentiment analysis, and text summarization.
- Utilizing pre-trained models allows NLP practitioners to economize on time and resources, bypassing the need to train a model from scratch on a large dataset.

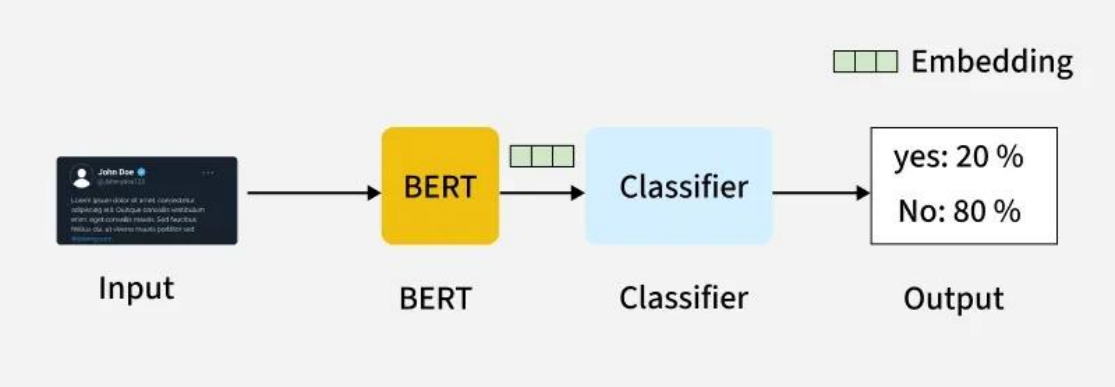
Advantages of using pre-trained models for projects are:

- Reduces the computational burden required for initial model training.
- The learned knowledge can be used for various applications.

- ❖ Models can be fine-tuned according to the task and can result in superior performance to training from the initial point.
- ❖ Less labelled data is required for fine-tuning specific tasks.

BERT Model - NLP

BERT (Bidirectional Encoder Representations from Transformers) stands as an open-source machine learning framework designed for the natural language processing (NLP).

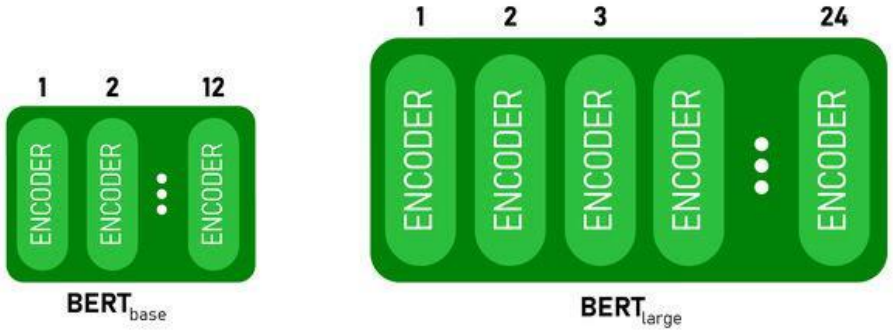


- ❖ BERT leverages a transformer-based neural network to understand and generate human-like language.
- ❖ BERT employs an encoder-only architecture. In the original **Transformer architecture,** there are both encoder and decoder modules. The decision to use an encoder-only architecture in BERT suggests a primary emphasis on understanding input sequences rather than generating output sequences.
- ❖ Traditional language models process text sequentially, either from left to right or right to left.
- ❖ This method limits the model's awareness to the immediate context preceding the target word.
- ❖ BERT uses a bi-directional approach considering both the left and right context of words in a sentence, instead of analyzing the text sequentially, BERT looks at all the words in a sentence simultaneously.

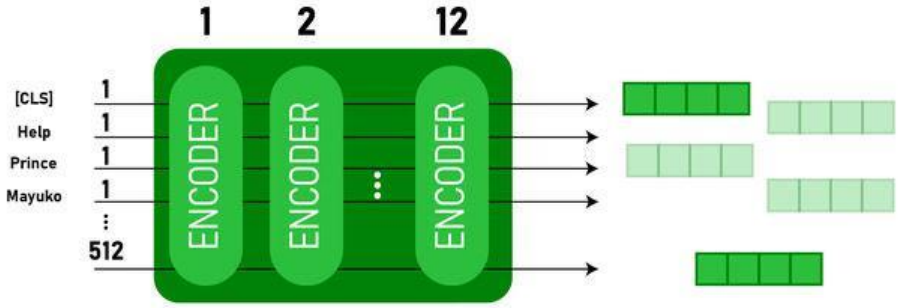
BERT Architecture

The architecture of BERT is a multilayer bidirectional **transformer encoder** which is quite similar to the transformer model. A transformer architecture is an encoder-decoder network that uses self-attention on the encoder side and attention on the decoder side.

1. BERTBASE has 12 layers in the Encoder stack while BERTLARGE has 24 layers in the Encoder stack.
2. BERT architectures (BASE and LARGE) also have larger feedforward networks (768 and 1024 hidden units respectively), and more attention heads (12 and 16 respectively) than the Transformer architecture. It contains 512 hidden units and 8 attention heads.
3. BERTBASE contains 110M parameters while BERTLARGE has 340M parameters.



- This model takes the CLS token as input first, then it is followed by a sequence of words as input where CLS is a classification token.
- It then passes the input to the above layers. Each layer applies self-attention and passes the result through a feedforward network after then it hands off to the next encoder.
- The model outputs a vector of hidden size (768 for BERT BASE). If we want to output a classifier from this model, we can take the output corresponding to the CLS token.



Now, this trained vector can be used to perform a number of tasks such as classification, translation, etc.

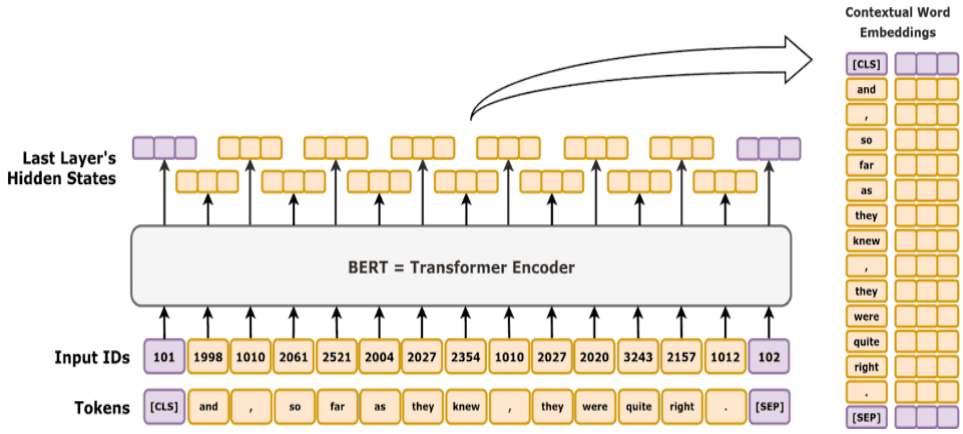
Workflow of BERT

BERT is designed to generate a language model so, only the encoder mechanism is used.

Sequence of tokens are fed to the Transformer encoder. These tokens are first embedded into vectors and then processed in the neural network.

The output is a sequence of vectors, each corresponding to an input token, providing contextualized representations.

When training language models, defining a prediction goal is a challenge. Many models predict the next word in a sequence, which is a directional approach and may limit context learning.



BERT addresses this challenge with two innovative training strategies:

1. Masked Language Model (MLM)
2. Next Sentence Prediction (NSP)

1. Masked Language Model (MLM)

In BERT's pre-training process, a portion of words in each input sequence is masked and the model is trained to predict the original values of these masked words based on the context provided by the surrounding words.

1. BERT adds a classification layer on top of the output from the encoder. This layer is important for predicting the masked words.
2. The output vectors from the classification layer are multiplied by the embedding matrix, transforming them into the vocabulary dimension. This step helps align the predicted representations with the vocabulary space.
3. The probability of each word in the vocabulary is calculated using the SoftMax activation function. This step generates a probability distribution over the entire vocabulary for each masked position.
4. The loss function used during training considers only the prediction of the masked values. The model is penalized for the deviation between its predictions and the actual values of the masked words.
5. The model converges slower than directional models because during training, BERT is only concerned with predicting the masked values, ignoring the prediction of the non-masked words. The increased context awareness achieved through this strategy compensates for the slower convergence.

2. Next Sentence Prediction (NSP)

BERT predicts if the second sentence is connected to the first. This is done by transforming the output of the [CLS] token into a 2×1 shaped vector using a classification layer, and then calculating the probability of whether the second sentence follows the first using SoftMax.

1. In the training process, BERT learns to understand the relationship between pairs of sentences, predicting if the second sentence follows the first in the original document.
2. 50% of the input pairs have the second sentence as the subsequent sentence in the original document, and the other 50% have a randomly chosen sentence.

3. To help the model distinguish between connected and disconnected sentence pairs. The input is processed before entering the model.
4. BERT predicts if the second sentence is connected to the first.

During the training of BERT model, the Masked LM and Next Sentence Prediction are trained **together**. The model aims to minimize the combined loss function of the Masked LM and Next Sentence Prediction, leading to a robust language model with enhanced capabilities in understanding context within sentences and relationships between sentences.

Uses of BERT model in NLP:

BERT can be used for various natural language processing (NLP) tasks such as:

1. Classification Task

- BERT can be used for classification task like sentiment analysis, the goal is to classify the text into different categories (positive/ negative/ neutral).
- The [CLS] token represents the aggregated information from the entire input sequence. This pooled representation can then be used as input for a classification layer to make predictions for the specific task.

2. Question Answering

- In question answering tasks, where the model is required to locate and mark the answer within a given text sequence, BERT can be trained for this purpose.
- BERT is trained for question answering by learning two additional vectors that mark the beginning and end of the answer.
- During training, the model is provided with questions and corresponding passages, and it learns to predict the start and end positions of the answer within the passage.

3. Named Entity Recognition (NER)

- BERT can be utilized for NER, where the goal is to identify and classify entities (e.g., Person, Organization, Date) in a text sequence.
- A BERT-based NER model is trained by taking the output vector of each token form the Transformer and feeding it into a classification layer. The layer predicts the named entity label for each token, indicating the type of entity it represents.

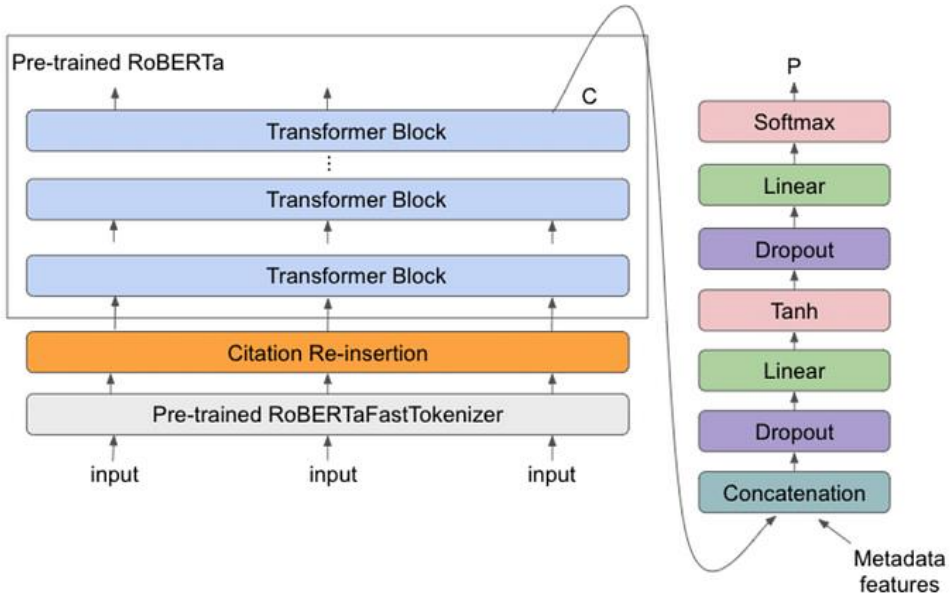
3. Pretrained Language Models: RoBERTa

- RoBERTa (Robustly Optimized BERT Approach) is a state-of-the-art language representation model developed by Facebook AI.
- It is based on the original BERT (Bidirectional Encoder Representations from Transformers) architecture but differs in several key ways.
- RoBERTa's objective is to improve the original BERT model by expanding the model, the training corpus, and the training methodology to better utilize the Transformer architecture.

- This produces a representation of language that is more expressive and robust, which has been shown to achieve state-of-the-art performance on a wide range of NLP tasks.
- This model is trained on a large amount of text data from multiple languages, which makes it capable of understanding and generating text in different languages.

Architecture

- The RoBERTa model is based on the Transformer architecture. The Transformer architecture is a type of neural network that is specifically designed for processing sequential data, such as natural language text.
- The architecture is nearly comparable to that of BERT, with a few slight modifications to the training procedure and architecture to enhance the results in comparison to BERT.



The RoBERTa model consists of a series of self-attentional and feed-forward layers.

The self-attention layers allow the model to weigh the importance of different tokens in the input sequence and compute representations that take into account the context provided by the entire sequence.

The feed-forward layers are used to transform the representations produced by the self-attention layers into a final output representation.

In the pre-training stage, the model can acquire a detailed representation of the language that can be suited for particular NLP tasks.

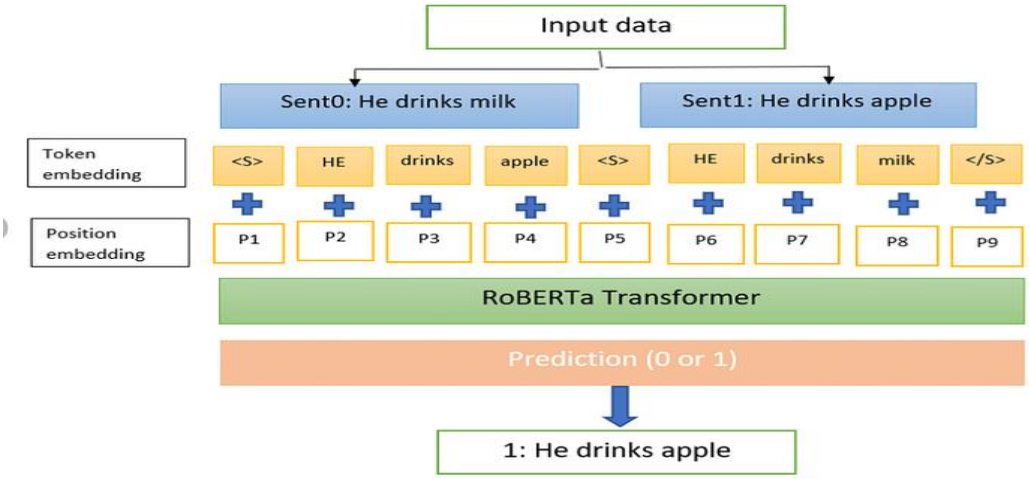
RoBERTa Working:

It works by pre-training a deep neural network on a large corpus of text. The working of **RoBERTa** is as follows**:**

1. **Pre-Training** → RoBERTa must be pre-trained first on a significant text corpus before being used.
2. **Fine-tuned** → The model can be fine-tuned for specific NLP tasks, including named entity recognition, sentiment analysis, or question answering, after pre-training.

During fine-tuning, the model is trained on a smaller dataset that is specific to the task at hand, utilizing the pre-learned weights as initialization.

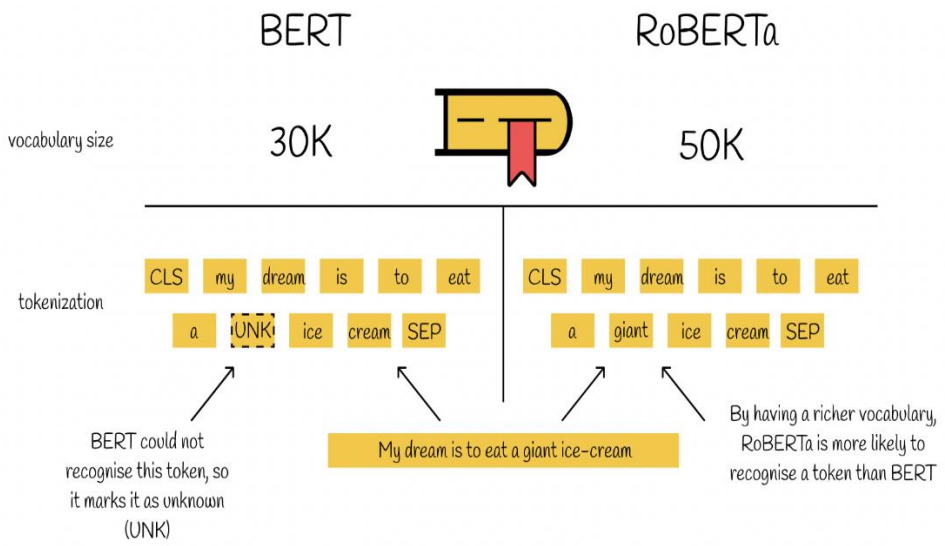
3. **Inference:** After fine-tuning, the model can be used for inference on a new text, by inputting the text into the network and using the learned representations to make predictions.



Differences between RoBERTa and BERT

Both models are pre-trained for language representation and based on the Transformer architecture, but there are several key differences between the two models:

1. **Training Corpus:** RoBERTa is trained on a larger corpus of text compared to BERT. Which makes it to learn a more robust and nuanced representation of the language.
2. **Dynamic Masking:** RoBERTa uses a dynamic masking strategy, where different tokens are masked in each training example. This allows the model to learn a more diverse set of representations, as it must predict different masks in different contexts.
3. **No Next Sentence Prediction Loss:** Unlike BERT, RoBERTa does not use a next sentence prediction (NSP) loss during pre-training. This allows RoBERTa to focus solely on the masked language modeling objective, leading to a more expressive language representation.
4. **Large Byte-Pair Encoding Vocabulary:** RoBERTa uses a larger byte-pair encoding (BPE) vocabulary size of 50k as compared to 30k size of BERT, allowing the model to learn a more fine-grained representation of the language.
5. **Fine-Tuning Strategies:** RoBERTa is trained with a more aggressive fine-tuning strategy compared to BERT, which includes longer training and learning rate schedules. This allows RoBERTa to better adapt to specific NLP tasks during fine-tuning.



4. Pretrained Language Models: GPT

Generative Pre-trained Transformer (GPT)

- The **Generative Pre-trained Transformer (GPT)** is a model, developed by Open AI to understand and generate human-like text.
- GPT has revolutionized how machines interact with human language making more meaningful communication possible between humans and computers.
- GPT is based on the transformer architecture and the core idea behind it is the use of self-attention mechanisms.
- This allows the model to weigh the importance of each word no matter its position in the sentence, leading to a more detailed understanding of language.
- As a generative model, GPT can produce new content. When provided with a prompt or a part of a sentence, GPT can generate contextually relevant continuations. This makes it extremely useful for applications like creating written content, generating creative writing or even simulating dialogue.

History and Development of GPT

The progress of GPT (Generative Pre-trained Transformer) models by OpenAI has been marked by significant advancements in natural language processing. The overview of GPT development is shown below:

1. **GPT (June 2018)**:
   - The original GPT model was introduced by OpenAI as a pre-trained transformer model that achieved results on a variety of natural language processing tasks.
   - It featured 12 layers, 768 hidden units and 12 attention heads, totalling 117 million parameters. This model was pre-trained on a diverse dataset using unsupervised learning and fine-tuned for specific tasks.
2. **GPT-2 (February 2019)**:
   - An upgrade, GPT-2 featured 48 transformer blocks, 1,600 hidden units

and 25 million parameters in its smallest version, up to 1.5 billion parameters in its largest.

- OpenAI initially delayed the release of the most powerful versions due to concerns about potential misuse. GPT-2 demonstrated an impressive ability to generate contextually relevant text over extended passages.

3. **GPT-3 (June 2020)**:
   GPT-3 marked a massive leap in the scale and capability of language models with 175 billion parameters. It improved upon GPT-2 in almost all aspects of performance and demonstrated abilities across broader tasks without task-specific tuning. GPT-3's performance showcased the potential for models to exhibit behaviours resembling understanding and reasoning.
4. **GPT-4 (March 2023)**: GPT-4 boasted more **nuanced** and accurate responses and improved performance in creative and technical domains. While the exact parameter count was been officially disclosed, it is understood to be significantly larger than GPT-3 and features architectural improvements that enhance reasoning and contextual understanding.
5. **GPT-4.5 (early 2024):** GPT-4.5 served as a bridge between GPT-4 and GPT-5. It brought **faster** response times, better **reliability** and more consistent reasoning. Though not a full architectural overhaul, it represented optimizations in performance and instruction-following capabilities, especially within the ChatGPT experience.
6. **GPT-4o (May 2024):**
   - GPT-4o ("o" for "omni") model released in May 2024 is considered the most advanced to date.
   - GPT-4o is a **multimodal model** capable of processing and generating **text, images and audio** including real-time speech input and output.
   - It offers near-instantaneous response times, reduced latency and better memory.
   - GPT-4o also represents a unification of modalities within a single neural network architecture, making it the first fully integrated model across media types.

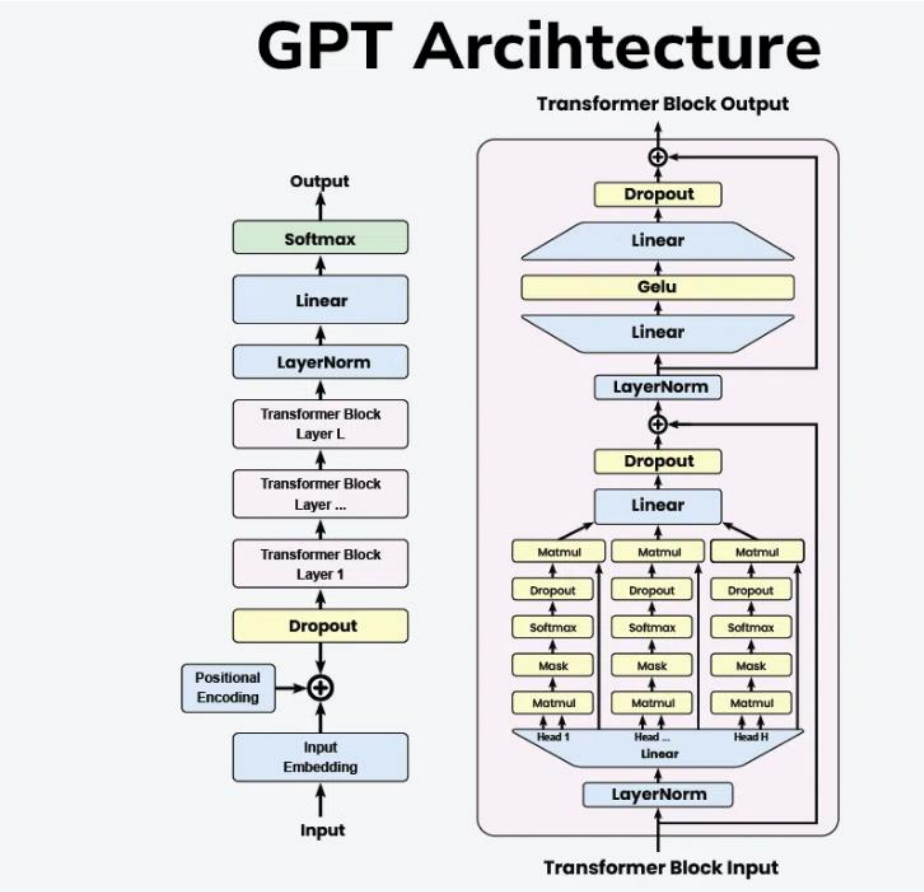
Architecture of Generative Pre-trained Transformer

The transformer architecture which is the foundation of GPT models is made up of feedforward neural networks and layers of self-attention processes. Important elements of this architecture consist of:

1. **Self-Attention System:** This enables the model to evaluate each word's significance within the context of the complete input sequence. It makes it possible for the model to comprehend word linkages and dependencies which is essential for producing content that is logical and suitable for its context.
2. **Layer normalization and residual connections:** By reducing problems such as disappearing and exploding gradients, these characteristics aid in training stabilization and enhance network convergence.

3. **Feedforward Neural Networks:** These networks process the output of the attention mechanism and add another layer of abstraction and learning capability. They are positioned between self-attention layers.

Detailed Explanation of the GPT Architecture



1. Input Embedding

- **Input**: The raw text input is tokenized into individual tokens (words or subwords).
- **Embedding**: Each token is converted into a dense vector representation using an embedding layer.

2. Positional Encoding: Since transformers do not inherently understand the order of tokens, positional encodings are added to the input embeddings to retain the sequence information.

3. Dropout Layer: A dropout layer is applied to the embeddings to prevent overfitting during training.

4. Transformer Blocks

- **LayerNorm**: Each transformer block starts with a layer normalization.
- **Multi-Head Self-Attention**: Multi-Head Self-Attention are core component, where the input passes through multiple attention heads.
- **Add & Norm**: The output of the attention mechanism is added back to the input (residual connection) and normalized again.

- **Feed-Forward Network**: A position-wise Feed-Forward Network is applied, typically consisting of two linear transformations with a GeLU activation in between.
- **Dropout**: Dropout is applied to the feed-forward network output.

5. Layer Stack: The transformer blocks are stacked to form a deeper model, allowing the network to capture more complex patterns and dependencies in the input.

6. Final Layers

- **LayerNorm**: LayerNorm is final layer normalization is applied.
- **Linear**: The output is passed through a linear layer to map it to the vocabulary size.
- **Softmax**: A Softmax layer is applied to produce the final probabilities for each token in the vocabulary.

Advantages of GPT

1. **Flexibility**: GPT's architecture allows it to perform a wide range of language-based tasks.
2. **Scalability**: As more data is fed into the model, its ability to understand and generate language improves.
3. **Contextual Understanding**: Its deep learning capabilities allow it to understand and generate text with a high degree of relevance and contextuality.

Applications of Generative Pre-trained Transformer

The versatility of GPT models allows for a wide range of applications, including but not limited to:

1. **Content Creation:** GPT can generate articles, stories and poetry, assisting writers with creative tasks.
2. **Customer Support:** Automated chatbots and virtual assistants powered by GPT provide efficient and human-like customer service interactions.
3. **Education:** GPT models can create personalized tutoring systems, generate educational content and assist with language learning.
4. **Programming:** GPT's ability to generate code from natural language descriptions aids developers in software development and debugging.
5. **Healthcare:** Applications include generating medical reports, assisting in research by summarizing scientific literature and providing conversational agents for patient support.

5.Fine-tuning Transformers for Text Classification

- Fine-tuning a transformer for text classification involves taking a pre-trained model and adapting it to a specific text classification task using our own labelled dataset.
- This process typically uses libraries like Hugging Face's `transformers` and its `Trainer` API to load the dataset and model, preprocess the text, train (fine-tune) the model on the custom data, and then evaluate its performance.

- The result is a model that leverages the general language understanding of the pre-trained transformer, but is specialized for our particular classification problem.

Key Steps for Fine-Tuning:

1. **Prepare the Data:** we need a well-balanced, labelled dataset where each text sample is paired with its correct label.
2. **Load Pre-trained Model and Tokenizer:** Choose a suitable transformer model (e.g., BERT, DistilBERT) and load it with its corresponding tokenizer, which converts text into a numerical format the model can understand.
3. **Preprocess the Dataset:** Use the tokenizer to convert text data into the numerical input format required by the pre-trained transformer.
4. **Define Training Arguments:** Set hyperparameters for the training process, such as the learning rate, batch size, number of training epochs, and where to save the model.
5. **Instantiate the Trainer:** Use the Hugging Face `Trainer` class, which simplifies the training loop, evaluation, and logging processes.
6. **Train (Fine-tune) the Model:** Run the `Trainer`'s `train()` method to fine-tune the pre-trained model on your custom dataset.
7. **Evaluate the Model:** Assess the fine-tuned model's performance on a separate test set using metrics like accuracy or F1-score to measure its effectiveness.

Uses of Fine-Tuning:

- **Domain-Specific Tasks:** It allows us to adapt general-purpose models to specific domains like legal, medical, or financial text.
- **Improved Accuracy:** Fine-tuned transformers often achieve better performance on custom classification tasks compared to traditional machine learning methods.
- **Smaller Datasets:** It enables effective training even with smaller datasets, as the model starts with strong pre-learned language understanding.
- **Custom Applications:** It's a fundamental technique for building systems like chatbots, auto-tagging tools, and personalized recommendation engines

6.Question-Answering System

- Question answering (QA) is a field of natural language processing (NLP) and artificial intelligence (AI) that aims to develop systems that can understand and answer questions posed in natural language.
- The point of a QA system is to understand the question and give an answer that is correct and helpful.
- QA systems can be based on various techniques, including information retrieval, knowledge-based, generative, and rule-based approaches. Each method has its strengths and weaknesses, and the choice of method depends on the project's specific needs.

- QA systems can be used in many places, like customer service, search engines, healthcare, education, finance, e-commerce, voice assistants, chatbots, and virtual assistants.

Working of a natural language question-answering system:

A natural language question-answering (QA) system is a computer program that automatically answers questions using NLP. The basic process of a natural language QA system includes the following steps:

1. **Text pre-processing**: The question is pre-processed to remove irrelevant information and standardise the text's format. This step includes tokenization, lemmatization, and stop-word removal, among others.
2. **Question understanding**: The pre-processed question is analysed to extract the relevant entities and concepts and to identify the type of question being asked. This step can be done using natural language processing (NLP) techniques such as named entity recognition, dependency parsing, and part-of-speech tagging.
3. **Information retrieval**: The question is used to search a database or corpus of text to retrieve the most relevant information. This can be done using information retrieval techniques such as keyword search or semantic search.
4. **Answer generation:** The retrieved information is analysed to extract the specific answer to the question. This can be done using various techniques, such as machine learning algorithms, rule-based systems, or a combination.
5. **Ranking**: The extracted answers are ranked based on relevance and confidence score.

The specific methods used in each step and the system's architecture will depend on the QA system's design and the type of questions it intends to answer.

Example: some systems are based on a knowledge base, others on information retrieval, and others on generative models.

Hybrid systems can also be designed to combine several approaches to improve overall performance.

Training a QA model requires a large dataset of questions and corresponding answers.

Types of Question Answering Systems

There are several different approaches to QA implementation in NLP. Some of them are:

1. Information retrieval-based QA
2. Knowledge-based QA
3. Generative QA
4. Hybrid QA
5. Rule-based QA

1. Information retrieval-based QA

- Information retrieval-based question answering (QA) is a method of automatically answering questions by searching for relevant documents or passages that contain the answer.
- This approach uses information retrieval techniques, such as keyword or semantic search, to identify the documents or passages most likely to hold the answer to a given question.
- Information retrieval-based QA systems are generally easy to implement and can be used to answer a wide range of questions.
- However, their performance can be limited by the quality and relevance of the indexed text and the effectiveness of the retrieval and extraction methods used.
- It's also important to note that IR-based QA systems are often used with other types of QA, like knowledge-based or generative QA, to improve the system's overall performance.

2. Knowledge-based QA

- Knowledge-based question answering (QA) automatically answers questions using a knowledge base, such as a database, to retrieve the relevant information.
- This strategy's foundation is that searching for a structured knowledge base for a question can yield the answer.
- Knowledge-based QA systems are generally more accurate and reliable than other QA approaches based on structured and well-curated knowledge.
- But their performance can be limited by how well the knowledge base is covered and how well the methods used to make queries and get information from their work.
- The knowledge-based QA systems are often used with other QA methods, like information retrieval-based or generative QA, to improve the overall performance of the QA system.

3. Generative QA

- Generative question answering (QA) automatically answers questions using a generative model, such as a neural network, to generate a natural language answer to a given question.
- This method is based on the idea that a machine can be taught to understand and create text in natural language to provide a correct answer in terms of grammar and meaning.
- Generative QA systems are powerful as they can answer a wide range of questions and generate more human-like answers.
- However, their performance can be limited by the training data's quality and diversity and the model's complexity.
- Generative QA systems are often used with other QA approaches, such as information retrieval-based or knowledge-based QA, to improve the overall performance of the QA system.
- These combinations are known as Hybrid QA systems.

4. Hybrid QA

- Hybrid question answering (QA) automatically answers questions by combining multiple QA approaches, such as information retrieval-based, knowledge-based, and generative QA.
- This approach is based on the idea that different QA approaches have their strengths and weaknesses, and by combining them, the overall performance of the QA system can be improved.
- Hybrid QA systems are considered more robust and accurate than a single QA approach, as they can leverage the strengths of multiple QA methods.
- Hybrid QA systems can also be more flexible, as they can adapt to different types of questions and different levels of complexity.

5. Rule-based QA

- Rule-based question answering (QA) automatically answers questions using a predefined set of rules based on keywords or patterns in the question.
- This approach is based on the idea that many questions can be answered by matching the question to a set of predefined rules or templates.
- Rule-based QA systems are generally simple and easy to implement. The rule-based QA systems are often combined with other QA approaches, such as information retrieval-based, knowledge-based, or generative QA, to improve the overall performance of the QA system.

Example:

The real-world implementation of transformers is carried out almost exclusively using a library called **transformers** built by a collection of people that refer to themselves as Hugging Face.

This library is open-source, and it has an incredibly active community we can begin using state-of-the-art models from Google, OpenAI, and We can install transformers for Python easily with:

pip install transformers

or we can install TensorFlow with:

pip install tensorflow

OR

conda install tensorflow

After that, we can find the two models— **deepset/bert-base-cased-squad2** and **deepset/electra-base-squad2**.

Both of these models have been built by Deepset.AI hence the deepset/. They have pre-trained for Q&A on the SQuAD 2.0 dataset as denoted by squad2 at the end.

Our question-answering process evolves:

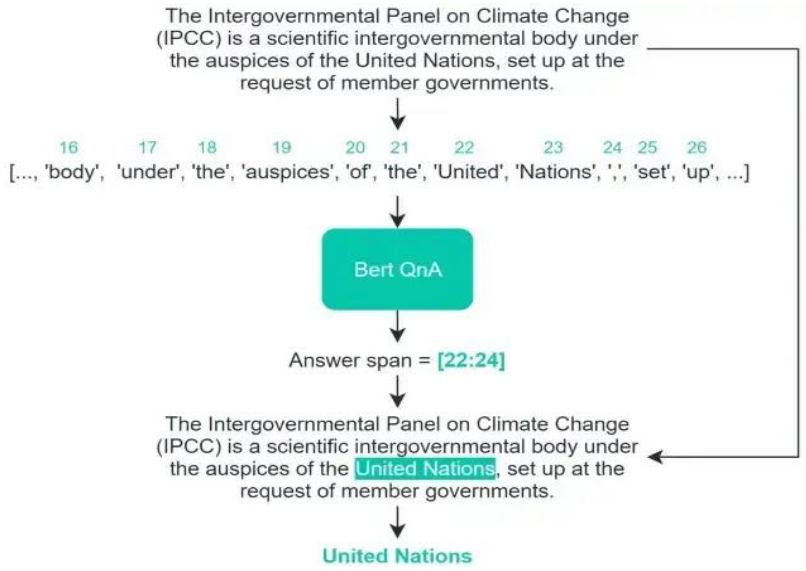
1. Model and tokenizer initialization
2. Query tokenization
3. Pipeline and Prediction

Model and Tokenizer Initialization - this is our very first step, here we will import transformers and initialize our model and tokenizer using deepset/bert-base-cased-squad2.

Input (Query) Data Tokenization - now it's time to feed it some text to convert into Bert-readable token IDs. Our Bert tokenizer is in-charge of converting human-readable text into Bert-friendly data, called token IDs. First, the tokenizer takes a string and splits it into tokens.

Pipeline and Prediction –

- Now we need to understood how our tokenizer is converting our human-readable strings into lists of token IDs, we can move to integrating this and asking questions but it is genuinely all we need to code a Q&A model and begin asking questions.
- However, let's break-down this pipeline function a little so we can understand what is actually happening.
- First, we are passing the question and the context. When feeding these into a Q&A model, a specific format is expected — which looks like:

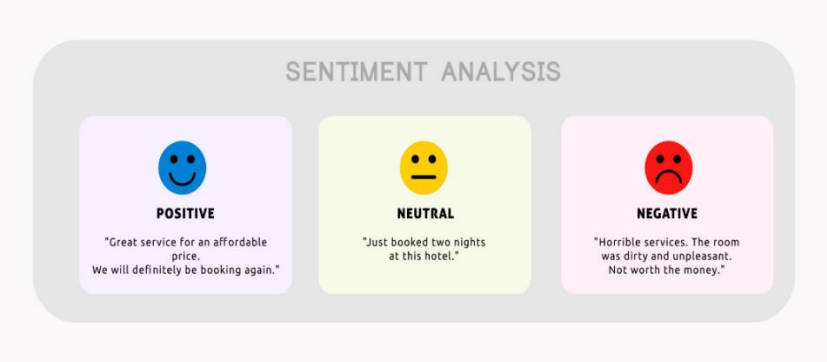


- At last, the pipeline will pull out the relevant token IDs according to Bert's start-end token index prediction.
- These token IDs will then be fed back into our tokenizer to be decoded into human-readable text and there we have it, our answer.
- This is how transformers work as Question answering system

7.Sentiment Analysis

SENTIMENT ANALYSIS

- Sentiment analysis (or opinion mining) is a natural language processing (NLP) technique used to determine whether data is **positive, negative or neutral**.
- Sentiment analysis is often performed on textual data to help businesses monitor brand and product sentiment in customer feedback, and understand customer needs.
- companies often develop sentiment analysis systems for customer experience management, social media monitoring, or workforce analytics platform to their own customers.



Types of Sentiment Analysis:

- ❖ Emotion Analysis
- ❖ Multilingual Sentiment Analysis
- ❖ Graded Sentiment Analysis
- ❖ Aspect-based Sentiment Analysis
- ❖ Intent Analysis

Sentiment analysis is aimed at determining the general emotional state of a text. One of these cases focuses on the polarity of a text (positive, negative, neutral) but it also goes beyond polarity to detect specific feelings and emotions (angry, happy, sad, etc), urgency (urgent, not urgent) and even intentions (interested v. not interested). Some types of sentiment analysis are explained below:

- ❖ **Emotion Analysis**

The type of emotion analysis in which emotion types (happiness, frustration, anger, and sadness) are classified is called **emotion detection.**

Example: the sentence "I connect to customer service too late, it's killing me" is a negative sentence, while the sentence "you are killing me" is positive.

- ❖ **Multilingual Sentiment Analysis**

It is the version of Sentiment Analysis systems that provides multi-language support.

- ❖ **Graded Sentiment Analysis**

If the precision of the mood is important, the categories can be further elaborated. A broader classification can be made, not just positive and negative:

- Very positive
- Positive
- Neutral
- Negative
- Very negative

This classification is often used in reviews where 5 stars are awarded.

- Very Positive = 5 stars
- Very Negative = 1 star

❖ **Aspect-based Sentiment Analysis**

Generally, when analyzing the emotions of the texts, the focus is on determining whether the comment/opinion is positive or negative. But we do not focus on what is positive or negative in this text.

Example: in the expression "I did not like the product at all, the size is too small", the user is not satisfied with the product and complains about its dimensions. In a normal sentiment analysis, this sentence is classified as negative, but in **aspect-based sentiment analysis**, the "the size is too small" part is also focused on.

❖ **Intent Analysis**

Intent analysis focuses on what the user wants to do. Understanding what the user wants to do will allow us to better guide him.

For example, being able to understand that a customer browsing an e-commerce site has a shopping intention also allows us to offer him the right products.

Benefits of sentiment analysis:

❖ **Sorting Data at Scale**

❖ Users make a lot of comments about brands, it is almost impossible to process them manually. Sentiment analysis enables businesses to automatically classify large amounts of raw data.

❖ **Real-Time Analysis**

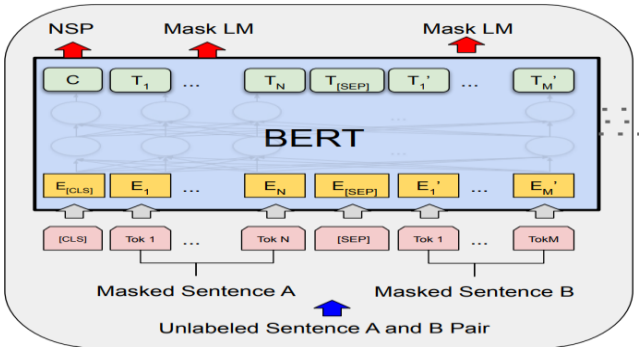
❖ Companies can learn the wishes of their customers by analyzing the social media comments about you in real time. They can identify the angry customer and ensure his satisfaction.

❖ **Discovering New Marketing Strategies**

❖ With more data and information gathered through sentiment analysis, the organizations could develop an effective marketing strategy.

SENTIMENT ANALYSIS MODELS

1 BERT:



BERT:

- Makes use of Transformer, an attention mechanism that learns contextual relations between words (or sub-words) in a text.
- In its vanilla form, Transformer includes two separate mechanisms — an encoder that reads the text input and a decoder that produces a prediction for the task.
- Since BERT's goal is to generate a language model, only the encoder mechanism is necessary.
- BERT is a bi-directional transformer for pre-training over a lot of unlabelled textual data to learn a language representation that can be used to fine-tune for specific machine learning tasks.
- While BERT outperformed the NLP state-of-the-art on several challenging tasks, its performance improvement could be attributed to the bidirectional transformer, novel pre-training tasks of Masked Language Model and Next Structure Prediction along with a lot of data and Google's compute power.

2. XLNET

- It is a large bidirectional transformer that uses improved training methodology, larger data and more computational power to achieve better than BERT prediction metrics on 20 language tasks.
- To improve the training, XLNet introduces permutation language modeling, where all tokens are predicted but in random order.
- This is in contrast to BERT's masked language model where only the masked (15%) tokens are predicted.
- This is also in contrast to the traditional language models, where all tokens were predicted in sequential order instead of random order.
- This helps the model to learn bidirectional relationships and therefore better handles dependencies and relations between words.
- In addition, Transformer XL was used as the base architecture, which showed good performance even in the absence of permutation-based training.
- XLNet was trained with over 130 GB of textual data and 512 TPU chips running for 2.5 days, both of which are much larger than BERT.

3 RoBERTa

- Introduced at Facebook, Robustly optimized BERT approach RoBERTa, is a retraining of BERT with improved training methodology, 1000% more data and compute power.
- To improve the training procedure, RoBERTa removes the Next Sentence Prediction (NSP) task from BERT's pre-training and introduces dynamic masking so that the masked token changes during the training epochs.
- Larger batch-training sizes were also found to be more useful in the training procedure.
- Importantly, RoBERTa uses 160 GB of text for pre-training, including 16GB of Books Corpus and English Wikipedia used in BERT.
- The additional data included CommonCrawl News dataset (63 million

articles, 76 GB), Web text corpus (38 GB) and Stories from Common Crawl (31 GB).

- This coupled with whopping 1024 V100 Tesla GPU's running for a day, led to pre-training of RoBERTa.

8. Zero-shot Classification

Zero-Shot Text Classification using HuggingFace Model

- Zero-shot text classification is a groundbreaking technique that allows for categorizing text into predefined labels without any prior training on those specific labels.
- This method is particularly useful when labelled data is scarce or unavailable. Leveraging the HuggingFace Transformers library, we can easily implement zero-shot classification using pre-trained models.
- Zero-shot classification relies on pre-trained language models that understand language context deeply.
- These models can be prompted with new tasks, such as classification, by providing text and candidate labels.
- The model evaluates the text against the labels and assigns probabilities to each label based on its understanding.
- This is achieved using semantic knowledge transfer, where the model leverages prior knowledge, typically through:
  - Word embeddings (e.g., Word2Vec, GloVe)
  - Pre-trained language models (e.g., GPT, BERT, CLIP)
  - Ontologies and knowledge graphs

Working of Zero-Shot Learning:

Zero-shot classification is typically based on:

1. Semantic Embeddings

- Each class is represented by a **vector embedding** that captures its meaning.
- This can be derived from text descriptions, word embeddings, or knowledge graphs.

2. Similarity Matching

- When a new instance arrives, the model compares its representation with the available class embeddings.
- The class with the **highest similarity score** is selected.

Mathematical Formulation

Let:

- X be the feature space.
- Y_{train} be the set of seen classes.
- Y_{test} be the set of unseen classes, where $Y_{train} \cap Y_{test} = \emptyset$

Given a new instance $x \in X$, the model assigns it to a class $y \in Y_{test}$ using a semantic similarity function $S(x,y)$, such that:

$$\hat{y} = \arg\max_{y \in Y_{\text{test}}} S(f(x), g(y))$$

where:

- f(x) maps an instance to an embedding space.
- g(y) maps class descriptions to the same embedding space.

The **Cosine Similarity** function is often used:

$$S(x, y) = \frac{f(x) \cdot g(y)}{\|f(x)\| \|g(y)\|}$$

Types of Zero-Shot Learning:

1. **Transductive Zero-Shot Learning (TZSL):** Uses unlabelled test data distribution for classification.
2. **Inductive Zero-Shot Learning (IZSL):** Relies solely on training data without accessing test distributions.

HuggingFace Transformers for zero-shot classification:

The HuggingFace Transformers library provides an easy-to-use interface for zero-shot classification.

One of the most popular models for this task is facebook/bart-large-mnli, which is based on the BART model and fine-tuned on the Multi-Genre Natural Language Inference (MNLI) dataset.

Example:

Consider a classifier trained only on **cats** and **dogs**.

If asked to classify a "lion," a traditional classifier would fail.

However, a Zero-Shot model could recognize that a lion is semantically similar to a cat and classify it accordingly.

9. Text Summarization using HuggingFace Model

- Text summarization involves reducing a document to its most essential content. The aim is to generate summaries that are concise and retain the original meaning.
- Summarization plays an important role in many real-world applications such as digesting long articles, summarizing legal contracts, highlights from research papers, etc.
- With the use of deep learning and pre-trained language models, summarization systems have become more accurate and context-aware. Hugging Face Transformers library provides easy access to powerful summarization models like T5.

Text Summarization process Using Transformers

There are two ways to summarize text using transformer:

Extractive Summarization: Extractive summarization involves identifying important sections from text and generating them verbatim which produces a

subset of sentences from the original text. Transformers improve this procedure by using text processing to extract features, which they then use to rank sentences according to these attributes. The primary actions consist of:

- **Text Processing:** Transformers examine the text to determine its context and the connections among its various sections.
- **Feature Extraction:** The text takes key words and phrases, along with other significant properties.
- **Sentence Ranking:** The order of sentences is determined by how closely they relate to the main idea of the document.
- **Summary Generation:** A logical summary is created by combining the sentences that scored highest.

Abstractive Summarization: Abstractive summarization uses natural language techniques to interpret and understand the important aspects of a text and generate a more "human" friendly summary. This summarizes a text in a manner similar to that of a person. Here, methods like encoder-decoder models are used, where:

- **Encoder** : Processes the input text to understand and extract its features.
- **Decoder** : Generates the summary by creating new sentences that encapsulate the essence of the original text.

- In this architecture, transformers can function as the encoder, the decoder, or both.
- Transformers are trained on enormous volumes of textual data for both extractive and abstractive summarization.
- Their in-depth training makes them especially adept at summarizing assignments since it teaches them intricate patterns and connections between words, sentences, and entire papers.

Benefits:

- **Contextual Understanding:** To comprehend the context of words, sentences, and documents, transformers make use of attention mechanisms.
- **Large Language Models:** Transformers have a profound grasp of linguistic relationships and patterns since they have been educated on enormous volumes of textual data.
- **Scalability:** Transformers are ideal for summarizing lengthy papers or massive volumes of text data because they can handle enormous amounts of text data in simultaneously.
- **End-to-End Training:** By training transformers on text summarizing tasks from beginning to end, we can tailor their performance to the particular task at hand.
- **State-of-the-Art:** Text summarization is just one of the many natural language processing tasks that Transformers have accomplished state-of-the-art results on. Steps to Summarize Text with Transformer-based Models.

Steps to summarize text with transformer-based model: Python

Step1: Install Transformers
Step2: Importing the pipeline Module from the transformers Library
Step3: Importing the textwrap Library
Step4: Importing the numpy Library
Step5: Importing the pandas Library
Step6: Importing the pprint Function from the pprint Library
Step7: Loading the Dataset into a DataFrame
Step8: Display the first few rows of the DataFrame to ensure it loaded correctly
Step9: Selecting a Business News Article from the DataFrame
Step10: Defining the Text Wrapping Function
Step11: Printing the Wrapped News Article
Step12: Creating the Summarization Pipeline
Step13: Selecting an Article and Generating a Summary
Step14: Printing the Summarized Text

These lines select and summarize an article from the 'entertainment' category in a similar manner as above.

UNIT V
Syllabus

Applications, Ethics, and Multilingual NLP

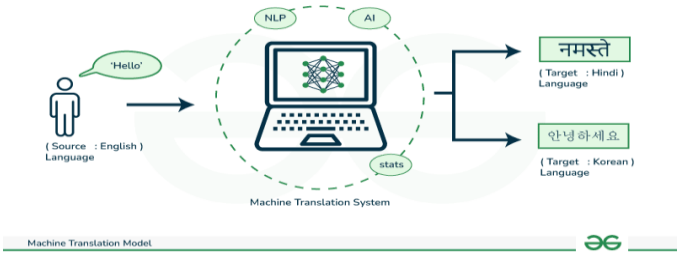
Machine Translation: Rule-based vs Neural MT, Chatbots and Conversational AI, Information Retrieval and Question Answering, Speech-to-Text and Text-to-Speech Overview, Multilingual NLP and Low-Resource Languages, Bias, Fairness, and Ethics in NLP.

STUDY MATERIAL

Prepared by **K.ANJANEYULU, M. Tech (CSE), MBA, PGDCA, M. Sc, B. Ed.,**
Assistant Professor,
Department of Computer Science & Engineering (AI&ML),
Sri Venkateswara College of Engineering & Technology,
RVS Nagar. Chittoor.
Phno: **(+91) – 9640985702, (+91) – 9398526101**
E-mail: **anji.karapakula@gmail.com**

Machine Translation

- Machine translation is a sub-field of computational linguistics that focuses on developing systems capable of automatically translating text or speech from one language to another.
- In Natural Language Processing (NLP), the goal of machine translation is to produce translations that are not only grammatically correct but also convey the meaning of the original content accurately.



Key approaches in Machine Translation:

In machine translation, the original text is decoded and then encoded into the target language through two step process that involves various approaches employed by language translation technology to facilitate the translation mechanism.

1. Rule-Based Machine Translation
2. Statistical Machine Translation
3. Neural Machine Translation (NMT)
4. Hybrid Machine Translation

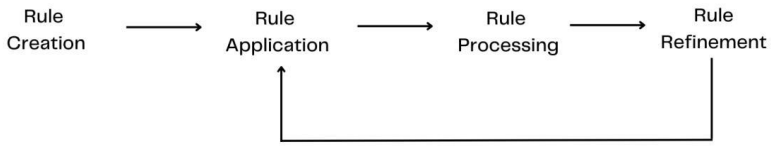
1. Rule-based Machine Translation

- Rule-based approach is one of the oldest NLP methods in which predefined linguistic rules are used to analyze and process textual data.
- Rule-based approach involves applying a particular set of rules or patterns to capture specific structures, extract information, or perform tasks such as text classification and so on.
- Some common rule-based techniques include regular expressions and pattern matches.

Steps in Rule-based approach in NLP:

1. **Rule Creation:** Based on the desired tasks, domain-specific linguistic rules are created such as grammar rules, syntax patterns, semantic rules or regular expressions.
2. **Rule Application:** The predefined rules are applied to the inputted data to capture matched patterns.
3. **Rule Processing:** The text data is processed in accordance with the results of the matched rules to extract information, make decisions or other tasks.

4. **Rule refinement:** The created rules are iteratively refined by repetitive processing to improve accuracy and performance. Based on previous feedback, the rules are modified and updated when needed.



Types of RBMT

There are three different types of rule-based machine translation systems:

1. **Direct Systems** (Dictionary Based Machine Translation) map input to output with basic rules.
2. **Transfer RBMT Systems** (Transfer Based Machine Translation) employ morphological and syntactical analysis.
3. **Interlingual RBMT Systems**:

RBMT systems can also be characterized as the systems opposite to Example-based Systems of Machine Translation (Example Based Machine Translation), whereas Hybrid Machine Translations Systems make use of many principles derived from RBMT.

Basic principles

The main approach of RBMT systems is based on linking the structure of the given input sentence with the structure of the demanded output sentence, necessarily preserving their unique meaning. The following example can illustrate the general frame of RBMT:

A girl eats an apple. Source Language = English; Demanded Target Language = German

Minimally, to get a German translation of this English sentence one needs:

1. A dictionary that will map each English word to an appropriate German word.
2. Rules representing regular English sentence structure.
3. Rules representing regular German sentence structure.

And finally, we need rules according to which one can relate these two structures together.

Accordingly, we can state the following **stages of translation**:

1st: getting basic part-of-speech information of each source word:
a = indef.article; girl = noun; eats = verb; an = indef.article; apple = noun
2nd: getting syntactic information about the verb "to eat":
NP-eat-NP; here: eat – Present Simple, 3rd Person Singular, Active Voice
3rd: parsing the source sentence:
(NP an apple) = the object of eat

Often only partial parsing is sufficient to get to the syntactic structure of the source sentence and to map it onto the structure of the target sentence.

4th: translate English words into German
a (category = indef.article) => ein (category = indef.article)
girl (category = noun) => Mädchen (category = noun)
eat (category = verb) => essen (category = verb)
an (category = indef. article) => ein (category = indef.article)
apple (category = noun) => Apfel (category = noun)
5th: Mapping dictionary entries into appropriate inflected forms (final **generation**):
A girl eats an apple. => Ein Mädchen isst einen Apfel.

Components of RBMT:

The RBMT system contains:

- **SL morphological analyzer** - analyses a source language word and provides the morphological information;
- **SL parser** - is a syntax analyzer which analyses source language sentences;
- **translator** - used to translate a source language word into the target language;
- **TL morphological generator** - works as a generator of appropriate target language words for the given grammatical information;
- **TL parser** - works as a composer of suitable target language sentences;
- **Several dictionaries** - more specifically a minimum of three dictionaries:
- **SL dictionary** - needed by the source language morphological analyzer for morphological analysis,
- **bilingual dictionary** - used by the translator to translate source language words into target language words,
- **TL dictionary** - needed by the target language morphological generator to generate target language words.

Advantages

No bilingual texts are required. This makes it possible to create translation systems for languages that have no texts in common, or even no digitized data whatsoever.

- **Domain independent**. Rules are usually written in a domain independent manner, so the vast majority of rules will "just work" in every domain, and only a few specific cases per domain may need rules written for them.
- **No quality ceiling**. Every error can be corrected with a targeted rule, even if the trigger case is extremely rare. This is in contrast to statistical systems where infrequent forms will be washed away by default.
- **Total control**. Because all rules are hand-written, you can easily debug a rule-based system to see exactly where a given error enters the system, and why.
- **Reusability**.

Neural Machine Translation (NMT):

- Neural Machine Translation (also known as Neural MT, NMT, Deep Neural Machine Translation, Deep NMT, or DNMT) is a state-of-the-art machine translation approach that utilizes **neural network techniques** to predict the likelihood of a set of words in sequence.
- This can be a text fragment, complete sentence, or with the latest advances an entire document.
- Neural Machine Translation typically produces much higher quality translations that Statistical Machine Translation approaches, with better fluency and adequacy.
- Neural machine translation uses only a fraction of the memory used by the traditional Statistical Machine Translation (SMT) models.
- Neural machine translation attempts to build and train a single, large neural network that reads a sentence and outputs a correct translation.
- Neural Machine Translation (NMT) differs from rule-based machine translation (RBMT) primarily in its approach: NMT uses large, single neural networks and deep learning to learn context and nuances across entire sentences, producing more accurate and fluent translations. In contrast, RBMT relies on explicitly programmed linguistic rules and dictionaries, which are costly to create and maintain and struggle with ambiguity and idiomatic expressions.

Pros and Cons of Rule-Based Machine Translation (RBMT):

- **Pros:**
  - **No large datasets:** Doesn't require massive bilingual text corpora, making it useful for low-resource languages.
  - **Predictable:** Translations can be highly predictable and consistent because they follow specific rules.
  - **Domain-specific:** Rules can be tailored for specialized terminologies or domains.
- **Cons:**
  - **Labor-intensive:** Requires linguistic experts to manually write, codify, and maintain linguistic rules.
  - **Poor fluency:** Often results in stiff and less natural-sounding translations.
  - **Limited scope:** Struggles with ambiguity, idiomatic expressions, and the nuances of human language.

Pros and Cons of Neural Machine Translation (NMT):

- **Pros:**
  - **Higher quality:** Produces significantly more accurate and fluent translations by capturing context and nuances.
  - **Better handling of idioms:** Excels at translating idiomatic language more naturally.
  - **Scalable:** Capable of processing large volumes of text quickly.
- **Cons:**
  - **Data-hungry:** Requires massive amounts of parallel training data, which is a bottleneck for low-resource languages.

- **Computationally expensive:** Demands significant processing power and memory for training and operation.
- **"Black box" problem:** The complex nature of neural networks can make it difficult to understand why a specific translation was generated.

Differences between RBMT and NMT

Feature	Rule-Based MT (RBMT)	Neural Machine Translation (NMT)
Core Approach	Explicit, expert-defined	Deep learning algorithms and
Context Handling	Processes words and	Processes entire sentences for
Fluency &	Lower fluency, prone to	High fluency and accuracy,
Data Requirements	Low, no large corpora	High, requires massive
Development Cost	High cost for rule creation	High computational cost for

2. Chatbots

Chatbots are computer programs that simulate human conversations to create better experiences for customers.

Some operate based on predefined conversation flows, while others use artificial intelligence and natural language processing (NLP) to decipher user questions and send automated responses in real-time.

Types of chatbots:

1. Menu or button-based chatbots
2. Rule-based chatbots
3. AI-powered chatbots
4. Voice chatbots
5. Generative AI chatbots
6. Hybrid chatbots

1. Menu or button-based chatbots

- Menu-based or button-based chatbots are the most basic kind of chatbot. Users interact with them by clicking on the button option from a scripted menu that best represents their needs.

- Essentially, these chatbots operate like a decision tree and are good for transactional tasks.
- These chatbots offer simple functionality and are useful for answering repetitive, straightforward questions, but they struggle with more nuanced requests due to their limited predefined answer options.

2. Rule-based chatbots

- Rule-based chatbots also known as decision-tree, menu-based, script-based, button-based, or basic chatbots are the most rudimentary type of chatbots. They communicate through pre-set rules (if the customer says "X," respond with "Y").
- The conversations are sometimes designed like a decision-tree workflow where users can select answers depending on their use case.
- These bots are similar to automated phone menus where the customer has to make a series of choices to reach the answers they're looking for.
- The technology is ideal for answering FAQs and addressing basic customer issues.

3. AI-powered chatbots

- AI-powered chatbots also called as AI customer service chatbots. They are also referred to as contextual chatbots or virtual agents.
- They use machine learning, natural language processing, or both to understand user intent and form responses.
- These bots can continuously learn from conversations with customers, so they're able to deliver more helpful responses as time goes on.
- Both types of chatbots provide a layer of friendly self-service between a business and its customers.

4. Voice chatbots

- A voice chatbot is another conversation tool that allows users to interact with the bot by speaking to it, rather than typing.
- Some voice chatbots can be more basic. Some users might find the interactive voice response (IVR) technology frustrating.
- However, this system is evolving with artificial intelligence and improving customer satisfaction.
- AI-driven voice chatbots can offer the same advanced functionalities as AI chatbots, but they are deployed on voice channels and use text-to-speech and speech-to-text technology.
- With the help of NLP and through integration with computer and telephony technologies, voice chatbots can now understand spoken questions, analyse users' business needs and provide relevant responses in a conversational tone.

5. Generative AI chatbots

- The next generation of chatbots with generative AI capabilities can offer even more enhanced functionality.
- They are fluent in understanding common language, can adapt to a user's style of conversation and use empathy when answering users' questions.

- ❖ While conversational AI chatbots can digest a user's questions or comments and generate a human-like response, generative AI chatbots can go a step further by generating new content as the output.
- ❖ This new content could look like high-quality text, images and sound that are based on LLMs they are trained on.
- ❖ New chatbot software can provide personalized experiences to users and help support teams reach more customers quickly.
- ❖ Chatbot interfaces with generative AI can recognize, summarize, translate, predict and create content in response to a user's query without the need for human interaction.

6. Hybrid chatbots

- ❖ A hybrid chatbot is a conversational AI system that combines rule-based logic with machine learning capabilities.
- ❖ The combination of both can deliver a versatile user experience that handles a range of tasks varying in difficulty due to the integration of AI technology.
- ❖ Rule-based chatbots function on a set of predefined rules and scripts and therefore provide structure, while AI has learning potential for more complex interactions.
- ❖ The hybrid chatbot provides the best of both systems in one single system and delivers a vast user experience that is straightforward and personalized.

Note: Customer service teams handling 20,000 support requests on a monthly basis can save more than 240 hours per month by using chatbots.

Chatbots working:

- ➢ The earliest chatbots were essentially interactive FAQ programs, which relied on a limited set of common questions with pre-written answers.
- ➢ Unable to interpret natural language, these FAQs generally required users to select from simple keywords and phrases to move the conversation forward. Such, traditional chatbots are unable to process complex questions and answer simple questions that haven't been predicted by developers.
- ➢ Over time, chatbot algorithms became capable of more complex rules-based programming and even natural language processing, enabling customer queries to be expressed in a conversational way.
- ➢ This gave rise to a new type of chatbot, contextually aware and armed with machine learning to continuously optimize its ability to correctly process and predict queries through exposure to more and more human language.
- ➢ Modern AI chatbots now use natural language understanding (NLU) to discern the meaning of open-ended user input, overcoming anything from typos to translation issues.
- ➢ Advanced AI tools then map that meaning to the specific "intent" the user wants the chatbot to act upon and use conversational AI to formulate an appropriate response.

- These AI technologies leverage both machine learning and deep learning, different elements of AI, with some nuanced differences to develop an increasingly granular knowledge base of questions and responses informed by user interactions.

Typical use cases include:

- Timely, always-on assistance for customer service or human resource issues.
- Personalized recommendations in an e-commerce context.
- Promoting products and services with chatbot marketing.
- Defining of fields within forms and financial applications.
- Intaking and appointment scheduling for healthcare offices.
- Automated reminders to for time- or location-based tasks.

conversational AI

Conversational AI is a broader term that refers to AI-driven communication technology such as chatbots and virtual assistants (e.g., Siri or Amazon Alexa). Conversational AI platforms use data, machine learning (ML), and NLP to recognize vocal and text inputs, mimic human interactions, and facilitate conversational flow.

Conversational AI refers to technologies that can recognize and respond to speech and text inputs.

In customer service, this technology is used to interact with buyers in a human-like way.

The interaction can occur through a bot in a messaging channel or through a voice assistant on the phone.

Some examples include:

- **Online customer support**
- **Accessibility**
- I**nternet of things (IoT) devices**
- **Computer software**

Feature	Traditional Chatbot	Conversational AI
Intelligence	Rule-based, keyword-driven	AI-powered, NLU, ML, Deep Learning
Context	Minimal or no context understanding	Understands and remembers conversation context
Conversation Flow	Rigid, single-turn interactions	Dynamic, multi-turn, and adaptive

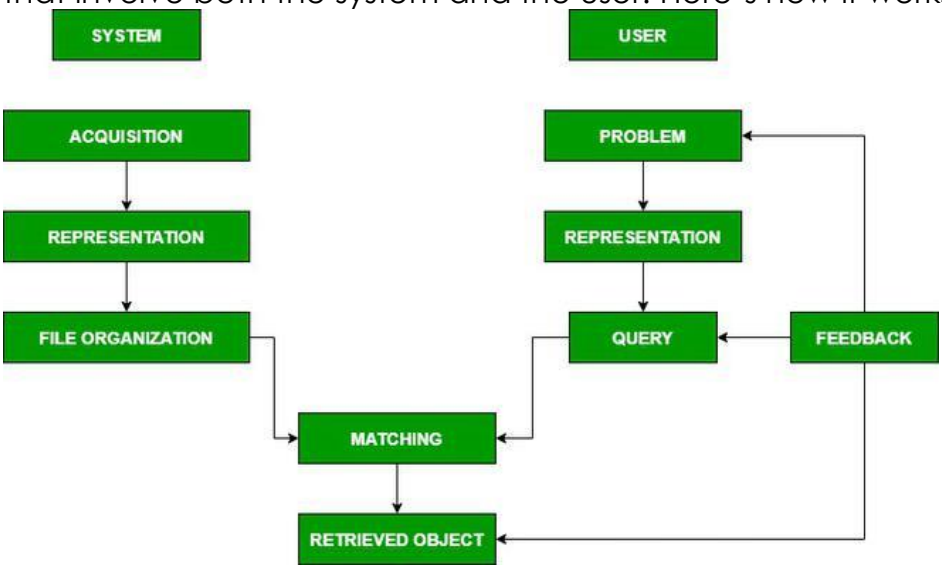
Personalization	Limited or none	High degree of personalization
Scope	Basic, structured tasks	Complex queries, problem-solving, process automation
Interaction	Primarily text-based chat	Text and voice

3. Information Retrieval

- **Information Retrieval (IR)** helps to find relevant information from large collections of documents. It can be defined as **a software program that deals with the organization, storage, retrieval and evaluation of information from documents**.
- It is like a smart librarian who doesn't give you direct answers but tells you where to find the right book like this IR system scans them and pulls out the ones that match your query.
- When we search for something **Information Retrieval (IR) model** helps find the most relevant documents and ranks them based on your query. It works by comparing your query with documents in the system using a **matching function**. This function gives each document a **retrieval status value (RSV)** which helps rank the most relevant results first. To do this IR systems represent documents using **descriptors** i.e most important keywords from **vocabulary (V).**

Components of Information Retrieval/ IR Model

The **Information Retrieval (IR) model** can be broken down into key components that involve both the system and the user. Here's how it works in a simple flow:



1. User Side (Search Process)

- **Problem Identification:** A student wants to learn about machine learning and types a query into a search engine.
- **Representation:** The user converts their need into a search query using keywords or phrases like instead of asking "How do machines learn?" the student types "machine learning basics" into Google and the problem is converted into a query (keywords or phrases).
- **Query:** The user submits the search query into IR system.
- **Feedback:** User can refine or modify the search based on the retrieved results.

2. System Side (Retrieval Process)

- **Acquisition:** The system collects and stores a large number of documents or data sources. It can includes web pages, books, research papers or any text-based information.
- **Representation:** Each document in the system is analyzed and represented in a structured way using keywords (terms). Example: If the document talks about "machine learning" it is tagged with relevant terms like "AI, deep learning, algorithms, models" to help retrieval.
- **File Organization:** The documents are **indexed and stored** efficiently so the system can quickly find relevant ones. Like organizing a library so books can be found easily based on topics.
- **Matching:** The system **compares the user's search query with stored documents** to find the best matches. It uses **matching functions** that rank documents based on **relevance.**
- **Retrieved Object:** The system returns the most **relevant documents** to the user. These documents are ranked so the most useful ones appear **at the top**.

3. Interaction Between User & System

The user reviews the retrieved results and may provide **feedback** to refine the search. The system then processes the updated query and retrieves better results.

Types of Information Retrieval (IR) Model

An information model (IR) model can be classified into the following three models −

Classical IR Model

It is the simplest and easy to implement IR model. This model is based on mathematical knowledge that was easily recognized and understood as well. Boolean, Vector and Probabilistic are the three classical IR models.

Non-Classical IR Model

It is completely opposite to classical IR model. Such kind of IR models are based on principles other than similarity, probability, Boolean operations. Information logic model, situation theory model and interaction models are the examples of non-classical IR model.

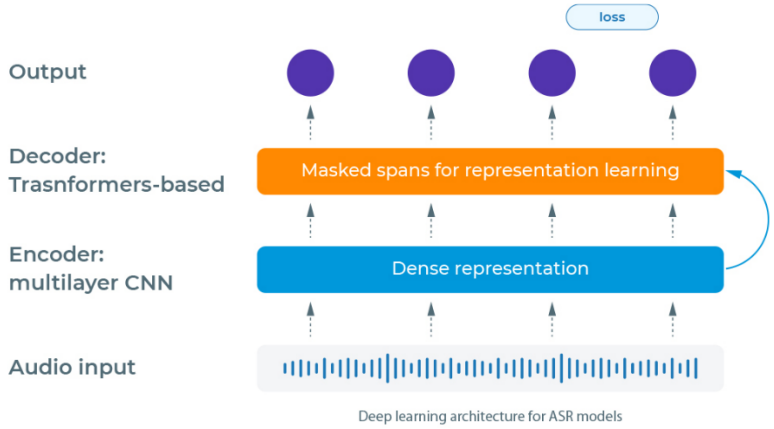
Alternative IR Model

It is the enhancement of classical IR model making use of some specific techniques from some other fields. Cluster model, fuzzy model and latent semantic indexing (LSI) models are the example of alternative IR model.

4. SPEECH TO TEXT

- Speech to text is the process of converting spoken words into a text transcript. Sometimes referred to as voice to text, it is available mostly as a software-based service (SaaS).
- It typically combines artificial intelligence-powered speech recognition technology, also known as automatic speech recognition, with transcription.
- A computer program picks up audio in the form of sound wave vibrations and uses linguistic algorithms to convert the audio input into digital characters, words and phrases.
- Machine learning, deep learning and large language models such as OpenAI's Generative Pre-Trained Transformer (GPT) have made speech to text software more advanced and efficient.
- Generative AI can be integrated with speech to text software to create assistants that can help customers over a phone call, or interact with voice-enabled apps.
- Generative AI can also convert text back to speech, otherwise known as text to speech, in a realistic, natural-sounding voice.
- Automatic Speech Recognition (ASR), or Speech to Text, is an NLP task that converts audio inputs into text.
- It is helpful for many applications, including automatic caption generation for videos, dictation to generate reports and other documents, or creating transcriptions of audio recordings.
- To perform this task, modern models use transformers-based deep learning models, and out of those, we have:
  1. **Wav2Vec 2.0**, created and shared by a Facebook researcher on wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations by Alexei Baevski, Henry Zhou, Abdelrahman Mohamed, Michael Auli
  2. **HuBER**T, also proposed by Facebook on HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units by Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, Abdelrahman Mohamed

Both Wav2Vec and HuBERT models consist of an encoder-decoder architecture. They encode the audio (converted in arrays of float numbers) into a dense representation and then decode this representation using attention-based (Transformers) models of NLP to generate the text.



Deep learning architecture for ASR models

While Wav2Vec 2.0 transforms the audio using the quantization technique with a Gumbel Softmax sampling to determine the candidate words, the HuBERT model uses the K-means algorithm to cluster the audio inputs and create embeddings of them that are used in the prediction step.
As for the loss function, the Wav2Vec 2.0 uses the CTC loss together with a Diversity loss, and the HuBERT model uses cross-entropy.

Uses of speech to text

There are various use cases for speech to text software:

1. Call center insight and agent assist
2. Real-time transcription and translation services
3. Voice recognition
4. Voice typing and dictation apps
5. Content monitoring

There are numerous applications of Speech Recognition some major applications are:

- It is very useful for making projects for physically disabled people.
- Designing a talking Bot
- Language Translator using Speech
- Offensive speech detection
- Smart Gadgets working on voice commands
- Military Equipment

5.Text-to-Speech

- Text-to-speech is a form of speech synthesis that converts any string of text characters into spoken output.
- Text to speech (TTS) is a type of technology that converts text on a digital interface into natural-sounding audio.
- It can also be referred to as "read aloud" technology, computer-generated speech or speech synthesis.

- Most companies offer text to speech technology as an application programming interface (API).
- Text-to-Speech (TTS) models can be used in any speech-enabled application that requires converting text to speech imitating human voice.
- Now, artificial intelligence-powered voice generators are enabling text to speech software to mimic human speech better. Opening up a wave of new use cases like customer service call answering, AI-generated podcasts,

text to speech working:

Deep learning techniques allow speech synthesis models to parse through more data and better understand the relationship between words and their acoustic feature. All of this makes the AI voice sound more natural. Converting text to speech is a multi-step process that involves both linguistic analysis and speech synthesis.

The main components of text to speech are:

1. Linguistic analysis
2. Speech synthesis

Linguistic analysis

Deep neural networks in the model are given audio datasets and corresponding transcriptions in English and sometimes other languages.

This helps the system understand how words match up with speech as well as accents, pitch, volume, tone, rhythm and more.

After it receives a text input, the text to speech model analyzes the words, punctuation and sentence structure.

It can expand abbreviations and expressions, calculate the duration of words, find the matching pronunciations and plot out the prosody of phrases and sentences.

Speech synthesis

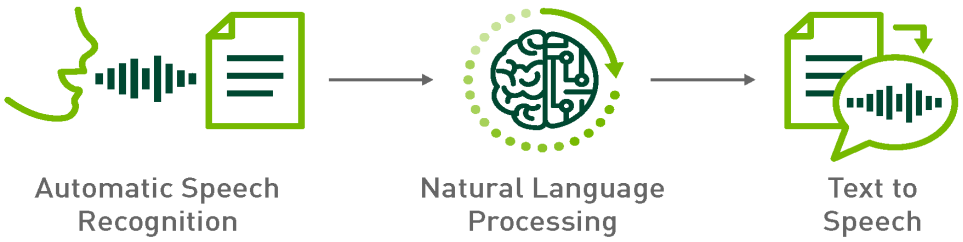
After the text gets analyzed, the model then uses a two-step process to turn it into a voice output.

Step 1: The model transforms the text into time-aligned features such as a spectrogram, which is used to map the variation of frequencies over time.

Step 2: A voice encoding (vocoder) network can turn the time-aligned features into audio waveforms, which computers can convert into natural sounding speech. Certain text to speech models allows users to alter volume, pitch, speed, and choose between different languages, accents and speaking styles.

Many devices like smartphones have text to speech systems built in. Text to speech is also available as a software program, a browser extension, a web-based tool or downloadable apps.

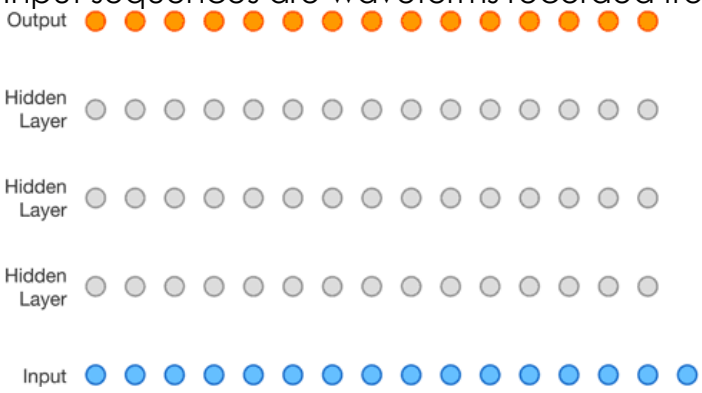
Telemarketers now employ these systems to replace human callers with conversational robots that can carry on realistic conversations to the point that a human operator is no longer needed.



Models for TTS:

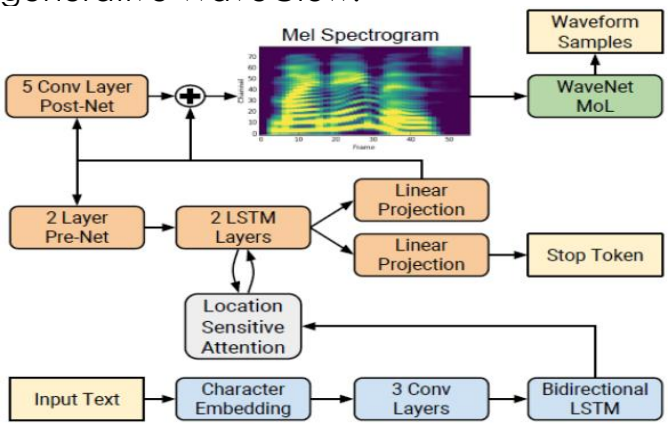
Popular deep learning models for TTS include Wavenet, Tacotron 2, and WaveGlow.

WaveNet: In 2006, Google WaveNet introduced deep learning techniques with a new approach that directly modeled the raw waveform of the audio signal one sample at a time.
Its model is probabilistic and autoregressive, with the predictive distribution for each audio sample conditioned on all previous ones. WaveNet is a fully convolutional neural network with the convolutional layers having various dilation factors that allow its receptive field to grow exponentially with depth. Input sequences are waveforms recorded from human speakers.



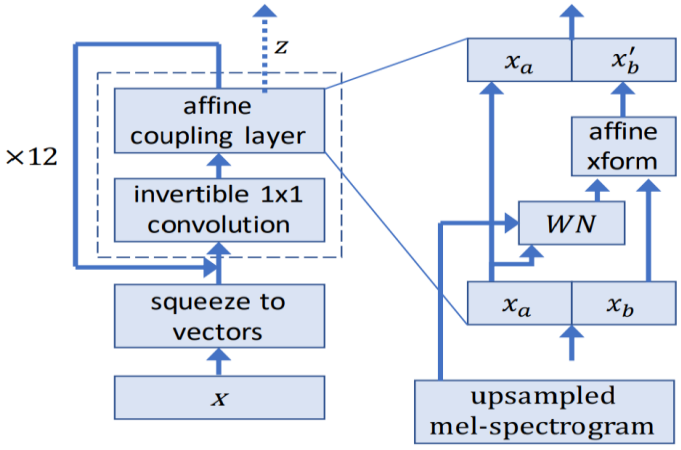
Tacotron 2 is a neural network architecture for speech synthesis directly from text using a recurrent sequence-to-sequence model with attention. The encoder (blue blocks in the figure below) transforms the whole text into a fixed-size hidden feature representation. This feature representation is then consumed by

the autoregressive decoder (orange blocks) that produces one spectrogram frame at a time. In the NVIDIA Tacotron 2 and WaveGlow for PyTorch model, the autoregressive WaveNet (green block) is replaced by the flow-based generative WaveGlow.



WaveGlow is a flow-based model that consumes the mel spectrograms to generate speech.

During training, the model learns to transform the dataset distribution into spherical Gaussian distribution through a series of flows. One step of a flow consists of an invertible convolution, followed by a modified WaveNet architecture that serves as an affine coupling layer. During inference, the network is inverted and audio samples are generated from the Gaussian distribution.



Applications of text to speech

Here are some of the main applications for the technology.

- Audio content
- Education
- Chatbots and virtual assistants

- Navigation
- Multilingual communication and language learning
- Media and entertainment
- Healthcare

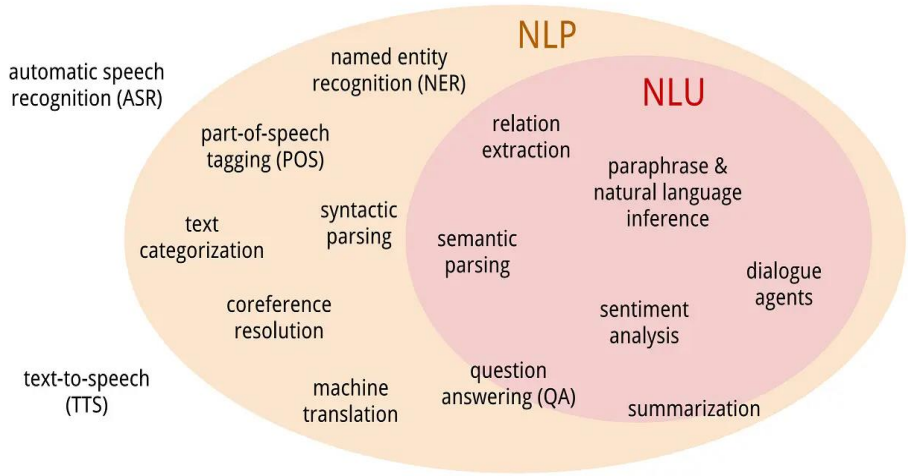
6.Multilingual NLP

Multilingual NLP tackles the challenge of developing systems that work across multiple languages, especially those with limited resources.
It's crucial for promoting language equality and enabling access to technology for underrepresented communities.

Challenges of Multilingual NLP

- Language Diversity and Variations

Top images from around the web for Language Diversity and Variations



- Multilingual NLP aims to develop NLP systems that can handle multiple languages, particularly low-resource languages with limited available data and resources
- Challenges in multilingual NLP include dealing with language diversity, variations in syntax (word order and sentence structure), morphology (word formation and inflection), and semantics (meaning) across languages
- Language-specific preprocessing and feature engineering are often needed to address these variations effectively

- Examples of language diversity include differences in writing systems (alphabets, characters), word order (Subject-Verb-Object, Subject-Object-Verb), and grammatical features (gender, case marking)

  Examples of low-resource languages include Quechua (indigenous language of South America), Yoruba (African language), and Hmong (Asian language)

- Multilingual NLP techniques can help bridge the language divide, facilitate cross-lingual information retrieval and machine translation, and support applications such as sentiment analysis and named entity recognition in multiple languages

Utilizing Linguistic Resources

- Leveraging linguistic resources, such as bilingual dictionaries, parallel corpora (texts aligned across languages), and typological databases (information about language features), can provide valuable information for improving NLP performance in low-resource languages
- Techniques like zero-shot learning and few-shot learning can be employed to adapt high-resource language models to low-resource languages with minimal or no labeled data
  - Zero-shot learning enables the application of models trained on high-resource languages directly to low-resource languages without the need for labeled data in the target language
  - Few-shot learning techniques, such as cross-lingual meta-learning and cross-lingual data augmentation, can effectively adapt models to low-resource languages with only a small amount of labeled data

Transfer Learning for Low-Resource NLP

Multilingual Embeddings

- Multilingual embeddings, such as fastText and MUSE, capture semantic similarities across languages and enable knowledge transfer from high-resource to low-resource languages
- These embeddings align word vectors from different languages into a shared semantic space, allowing for cross-lingual comparisons and knowledge transfer
- Examples of multilingual embedding models include fastText (supports 157 languages) and MUSE (Multilingual Unsupervised and Supervised Embeddings)

Pre-trained Multilingual Language Models

- Pre-trained multilingual language models, like mBERT (Multilingual BERT) and XLM-R (Cross-lingual Language Model), can be fine-tuned on low-resource languages with limited labeled data, leveraging the models' pre-trained knowledge to improve performance on downstream tasks

- These models are trained on large amounts of multilingual text data and learn language-agnostic representations that can be transferred to low-resource languages
- Fine-tuning these models on task-specific data in low-resource languages can significantly improve performance compared to training from scratch

Multilingual Data Augmentation Strategies

- Multilingual data augmentation strategies, like multilingual fine-tuning and multilingual multi-task learning, can leverage data from multiple languages to improve performance in low-resource languages
- Multilingual fine-tuning involves fine-tuning a pre-trained multilingual model on labeled data from multiple languages simultaneously
- Multilingual multi-task learning trains a single model to perform multiple tasks across different languages, sharing knowledge and representations
- Data augmentation techniques should be carefully designed to preserve the linguistic properties and maintain the quality of the generated examples to avoid introducing noise or biases

7.Bias, Fairness, and Ethics in NLP

Bias in NLP

- Bias in natural language processing (NLP) refers to the tendency of an NLP model to favour or discriminate against a particular group of people based on their race, ethnicity, gender, age, or other characteristics.
- Bias can occur in various ways throughout the development and deployment of NLP models, including data collection, data preprocessing, and algorithmic design.
- One of the main sources of bias in NLP is biased data. If the training data used for an NLP model is obtained from a specific group of individuals, the model may learn to favor their language, dialect, and cultural nuances.
- This can result in biased outputs that perpetuate stereotypes, inappropriate language, and discrimination against certain groups.
- Additionally, biases can be introduced during data preprocessing and algorithmic design.
- For example, the selection of certain features or the choice of certain parameters may lead to unintended biases in the model's output.
- Bias in NLP can have serious consequences, leading to discrimination, social injustice, and unequal treatment of certain groups.
- It is, therefore, essential to address and overcome bias in NLP to ensure that the technology is used responsibly and fairly.
- To overcome bias in NLP, it is crucial to ensure that the training data is representative of the entire population.
- This can be achieved by collecting data from diverse sources and populations. Fair data preprocessing techniques and algorithms that consider biases in the data must also be implemented. Regular monitoring

and testing of NLP models can help identify and correct any biases that may arise.

Fairness in NLP

- Fairness in natural language processing (NLP) pertains to the just and equal treatment of all individuals and groups without discrimination.
- This means that an NLP model should not amplify or perpetuate existing biases, stereotypes, or assumptions about certain groups.
- Instead, it should treat all individuals equally, regardless of their race, ethnicity, gender, age, or other characteristics.
- Ensuring fairness in NLP is crucial to prevent discrimination and promote equality. Fairness can be achieved by collecting and analyzing data on the performance of the model across various groups.
- This can help identify any biases or disparities that may arise and allow for corrective actions to be taken.
- Another way to ensure fairness in NLP is by using transparent and explainable models that can be easily audited.
- This means that the decision-making process of the model should be understandable and transparent to both developers and end-users.
- This can help build trust and accountability, as individuals can understand how the model arrived at its decisions and can identify any biases or disparities in the process.
- Overall, ensuring fairness in NLP is essential to promote social justice and prevent discrimination.
- It requires a commitment to collect and analyze data, use transparent and explainable models, and take corrective actions when necessary.
- Ultimately, promoting fairness in NLP can help build trust in the technology and promote its responsible use for the benefit of all.

Privacy in NLP

- Privacy is a crucial ethical consideration in natural language processing (NLP), as NLP models may collect, process, and store sensitive data, such as personal information, financial data, and health records.
- The misuse of this data can lead to serious privacy violations and harm to individuals.
- To ensure privacy in NLP, it is essential to adopt appropriate data protection and security measures.
- This can include data encryption, secure data storage, and access controls. These measures can help safeguard the data from unauthorized access, theft, or misuse.
- Obtaining informed consent from individuals before collecting and processing their data is another crucial aspect of ensuring privacy in NLP.
- This means that individuals should be fully informed about the data that is being collected, the purpose of the data collection, and how the data will be used. They should also have the option to opt out of data collection or to request the deletion of their data.

- Another important consideration for privacy in NLP is the anonymization of data. Anonymizing data means removing any personally identifying information from the data before processing it.
- This can help protect the privacy of individuals and prevent the misuse of their personal data.
- Overall, ensuring privacy in NLP is essential to protect the rights and dignity of individuals and to prevent the misuse of their personal data.
- It requires adopting appropriate data protection and security measures, obtaining informed consent from individuals, and anonymizing data when necessary.
- By promoting privacy in NLP, we can help build public trust in the technology and promote its responsible use for the benefit of all.

Ethics in NLP

Natural Language Processing (NLP) ethics is a critical and evolving area of consideration as technology advances. Here are some important ethical considerations in NLP:

Explainability: Due to the black-box nature of some advanced NLP models, understanding their decision-making processes can be difficult. Transparency necessitates efforts to make models more interpretable and explainable.

Openness: Transparency is enhanced by sharing information about data sources, training methods, and model architectures.

Responsibility: Developers and organisations that use NLP must accept responsibility for the impact of their technology. This includes addressing and resolving post-deployment issues.

Legal Implications: It is critical to understand and follow the legal frameworks governing data protection and privacy.

Accessibility: Making sure NLP applications work for all users, even those with varying linguistic backgrounds or skill levels.

Cultural sensitivity: Cultural sensitivity is avoiding imposing one culture's viewpoint on another and taking into account subtle cultural differences in language.

Security:

To stop misuse or exploitation, potential vulnerabilities in NLP systems should be found and fixed.

END OF UNIT – V